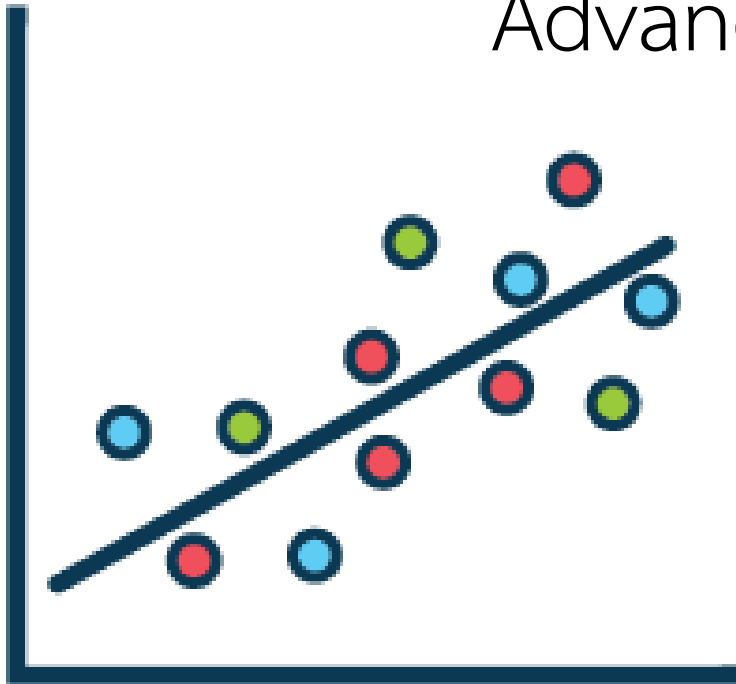




# Machine Learning : Advanced in Experiment and Ridge Regression



อ.ดร.ปัญญานต์ อ้นพงษ์

ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

aonpong\_p@su.ac.th

# Outline



- พื้นฐานการทดลองเกี่ยวกับ Machine Learning
  - Dataset Preparation
  - Training set, Validation set, Test set
  - K-Fold Cross Validation
  - Confusion Matric, Precision, Recall, Accuracy, Loss
- Regression (การถดถอย)
  - Linear Regression
  - Logistic Regression
  - Regression ประเภทอื่น ๆ ที่น่าสนใจ

# พื้นฐานการทดลองเกี่ยวกับ Machine Learning



Dataset Preparation



Design the model



Train the model



Test the model



Evaluation

- Download
- Data Cleaning
- แยกส่วนข้อมูลที่มีออกเป็น 3 ส่วน training set, validation set, and test set
- อาจทำ k-fold cross validation
- เลือกโมเดลที่เหมาะสมกับงาน
- พัฒนาโมเดลให้เข้ากับลักษณะของข้อมูล
- นำโมเดลที่ออกแบบมาฝึกฝน (Train) ด้วย Training set และ Validation set
- ปรับค่าพารามิเตอร์ต่าง ๆ ให้ได้ผลลัพธ์ที่รับได้ (Acc สูง หรือ Loss ต่ำ) ทดลองกับ Val.
- นำโมเดลที่ฝึกฝน (Train) แล้ว มาปรับค่าพารามิเตอร์ต่าง ๆ ให้ได้ผลลัพธ์ที่รับได้ (Acc สูง หรือ Loss ต่ำ) โดยทดลองกับ Test set
- วัดประสิทธิภาพของโมเดล

# Dataset Preparation

- ชุดข้อมูลหรือ Dataset คือส่วนประกอบชิ้นสำคัญของการสร้างโมเดล
- การได้ชุดข้อมูลที่ดี ส่งผลให้โมเดลมีประสิทธิภาพกว่าการได้ชุดข้อมูลที่ไม่ดี
- ในโลกการศึกษา ชุดข้อมูลมักจะดีกว่าชุดข้อมูลในโลกแห่งความเป็นจริง
- ชุดข้อมูลอาจถือได้ว่าเป็นส่วนประกอบที่หายากที่สุดในการสร้างโมเดลก็ได้
- แหล่งชุดข้อมูลที่แนะนำ Kaggle.com (สามารถดาวน์โหลดมาใช้ทดลองด้วยตนเองได้)



# Dataset Preparation: แนะนำ Kaggle.com



The image shows a collage of the Kaggle website interface. The main focus is the 'Datasets' section, which includes a search bar, a 'New Dataset' button, and a list of trending datasets. The datasets listed are:

- Car Price Prediction Challenge** by Deep Contractor (Updated 3 hours ago, Usability 8.8, 439 KiB, 1 File (CSV))
- Face mask detection** by SankNN (Updated 3 hours ago, Usability 6.9, 739 MiB, 18623 Files (other))
- NASA - Nearest Earth Objects** by Sameep Vani (Updated 20 days ago, Usability 10.0, 2 Files (CSV), 7 MiB)
- Airlines Dataset to predict a delay** by Jims Chacko (Updated 15 days ago, Usability 10.0, 1 File (CSV), 6 MiB)
- Mobile Price Classification** by Abhishek Sharma (Updated 4 years ago, Usability 7.1, 2 Files (CSV), 72 KiB)
- World University Rankings 2023** by Jatin (Updated 25 days ago, Usability 9.4, 1 File (CSV), 59 KiB)
- Driving Behavior** by Ion Cojocaru (Updated a month ago, Usability 10.0, 2 Files (CSV), 201 KiB)

The interface also features a 'New Notebook' button, a search bar for public notebooks, and a 'Trending' section with various data visualizations and datasets like 'Top 10% Rank with very good score', 'Top 100 YouTube Channels', and 'EDA Big Basket Strong Areas'.

# Dataset Preparation: แนะนำ Kaggle.com



The image shows a screenshot of the Kaggle website. The main header includes the Kaggle logo, a search bar, and links for Sign In and Register. The left sidebar contains navigation links: Home, Competitions, Datasets (selected), Code, Discussions, Courses, and More. The main content area is titled "Datasets" and includes a search bar, a "New Dataset" button, and a "Trending Datasets" section. The trending datasets list includes:

- Car Price Prediction Challenge** by Deep Contractor (Updated 3 hours ago, Usability 8.8, 439 KiB, 1 File (CSV))
- Face mask detection** by SankNN (Updated 3 hours ago, Usability 6.9, 739 MiB, 18623 Files (other))
- NASA - Nearest Earth Objects** by Sameep Vani (Updated 20 days ago, Usability 10.0, 2 Files (CSV), 7 MiB)
- Airlines Dataset to predict a delay** by Jims Chacko (Updated 15 days ago, Usability 10.0, 1 File (CSV), 6 MiB)
- Mobile Price Classification** by Abhishek Sharma (Updated 4 years ago, Usability 7.1, 2 Files (CSV), 72 KiB)
- World University Rankings 2023** by Jatin (Updated 25 days ago, Usability 9.4, 1 File (CSV), 59 KiB)
- Driving Behavior** by Ion Cojocaru (Updated a month ago, Usability 10.0, 2 Files (CSV), 201 KiB)

On the right side, there is a "New Notebook" button and a "Search public notebooks" section with filters for Python, R, Beginner, NLP, Random Forest, GPU, and TPU. Below this, there is a "Trending" section showing various notebooks with their respective scores and update times.

# Dataset Preparation: Data Cleaning

- โดยทั่วไป ข้อมูลที่ดาวน์โหลดหรือเก็บมาอาจมีข้อมูลบางบรรทัดหรือบางช่องที่ไม่สมบูรณ์ ด้วยสาเหตุบางประการ

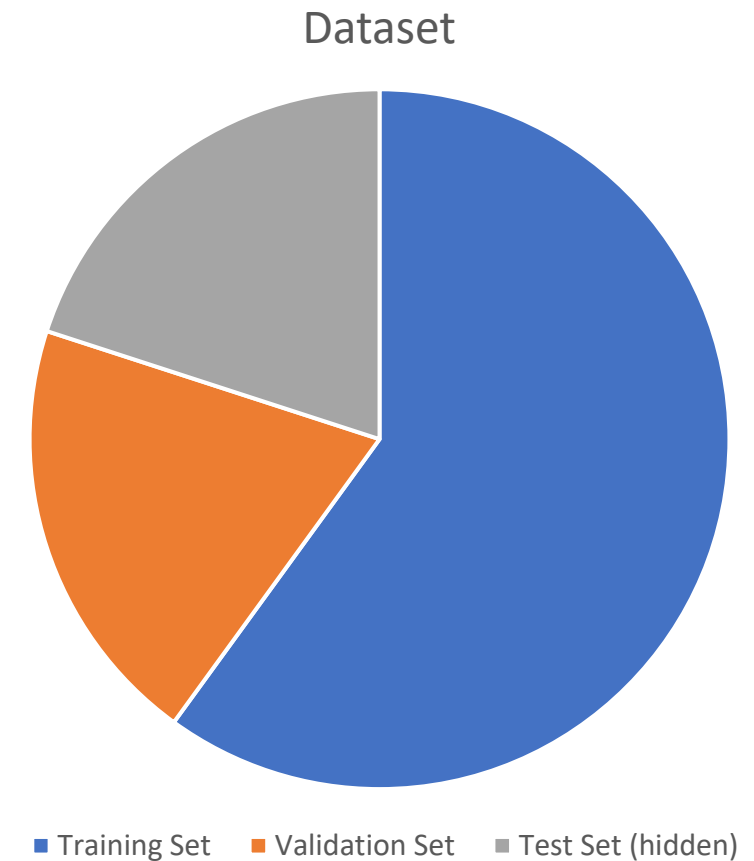
|             | R01-023   | R01-024   | R01-006   | R01-153   | R01-031  | R01-032   | R01-033   | R01-034  | R01-035   | R01-037   | R01-038  |
|-------------|-----------|-----------|-----------|-----------|----------|-----------|-----------|----------|-----------|-----------|----------|
|             | R01-141   | R01-142   | R01-144   | R01-145   | R01-146  |           |           |          |           |           |          |
| 1/2-SBSRNA4 | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| A1BG        | NA        | 2.52851   | 1.713994  | 3.143938  | 1.79508  | 2.41091   | 2.538406  | NA       | 10.386501 | 1.82622   | NA       |
| A1BG-AS1    | NA        | NA        | NA        | 0.646213  | NA       | NA        | NA        | NA       | NA        | NA        | 0.73464  |
| A1CF        | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| A2LD1       | 2.03438   | 0.436761  | 1.60103   | 3.366031  | 0.994382 | 2.130685  | 0.842759  | 1.835353 | 0.662647  | 0.646078  | 0.       |
| A2M         | 59.5255   | 31.7306   | 40.8846   | 93.2069   | 27.921   | 116.234   | 9.54511   | 93.646   | 25.0512   | 18.4653   | 19.      |
| 17          | 35.0792   | 142.104   | 233.773   | 153.046   | 192.524  | 58.2058   |           |          |           |           |          |
| A2ML1       | NA        | 2.374324  | 0.477936  | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| A2MP1       | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| A4GALT      | 2.475232  | 2.588888  | 1.90887   | 1.47992   | 1.82878  | 0.548072  | NA        | 1.52343  | 0.662936  | NA        | 2.61645  |
| 85241       | 3.263207  | NA        | 1.46139   | NA        | 2.7111   | 2.869876  | 1.16306   | 3.44072  | 6.36594   |           |          |
| A4GNT       | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| AA06        | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| AAA1        | NA        | NA        | 0.563646  | 23.554684 | NA       | NA        | 8.26926   | NA       | NA        | NA        | NA       |
| AAAS        | 2.50285   | 5.33951   | 3.94623   | 5.20715   | 4.99749  | 3.38188   | 2.32976   | 4.62022  | 2.47675   | 3.38809   | 6.       |
| 4           | 4.61238   | 4.75882   | 4.58766   | 5.77244   |          |           |           |          |           |           |          |
| AACS        | 2.447989  | 2.75109   | 2.757213  | 6.46604   | 4.354905 | 1.68304   | 4.14082   | 5.47595  | 2.52088   | 2.775422  | 6.       |
| 43          | 4.659579  | 6.36701   | 10.834514 | 6.70961   | 9.80428  | 3.68711   | 2.957987  | 1.50746  | 2.42778   | 10.50286  |          |
| AACSP1      | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| AADAC       | 1.37559   | 2.390977  | 4.538896  | NA        | NA       | 5.82912   | NA        | 1.46685  | NA        | NA        | 0.636969 |
| AADACL2     | 7.428797  | NA        | 2.844523  | NA        | 1.213328 | NA        | NA        | 0.744146 | NA        | NA        | 3.232238 |
| AADACL3     | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| AADACL4     | NA        | NA        | NA        | NA        | NA       | NA        | NA        | NA       | NA        | NA        | NA       |
| AADAT       | 3.013143  | 1.496432  | NA        | NA        | 0.691553 | 2.62826   | NA        | 6.769798 | 0.619719  | 4.359393  | 1.687345 |
| AAGAB       | 10.290372 | 8.55964   | 4.44126   | 9.36847   | 4.06271  | 4.69195   | 4.22151   | 9.20141  | 3.61207   | 6.80435   | 4.       |
| 0.84227     | 15.48193  | 7.00282   | 7.0136    | 9.31564   | 10.39528 | 8.68258   | 9.19264   | 12.41824 | 6.67453   | 13.10146  |          |
| AAV1        | 37.42241  | 27.581001 | 4.054714  | 21.177466 | 20.20451 | 45.770751 | 33.670513 | 0.077267 | 28.037501 | 16.120038 | 23.      |

# Dataset Preparation: Data Cleaning

- ในที่นี้ ถ้า Row ใด (ตัวอย่างข้อมูลใด) มีข้อมูลที่ขาดหาย ให้ลบข้อมูลแถวนั้นทิ้งได้เลย
- ข้อมูลจะหายไป ทำให้มีข้อมูลในการฝึกน้อยลง ดังนั้นควรระมัดระวังด้วย
- วิธีนี้ไม่ได้ดีที่สุด แต่ง่ายที่สุดและยอมรับได้ ถ้ายังคงมีข้อมูลที่ใช้ได้เหลืออยู่ในปริมาณมาก
- ถ้าใครศึกษาเพิ่มเติมแล้วพบวิธีที่ดีกว่า สามารถนำมาใช้ทำ Assignment ได้เช่นกัน

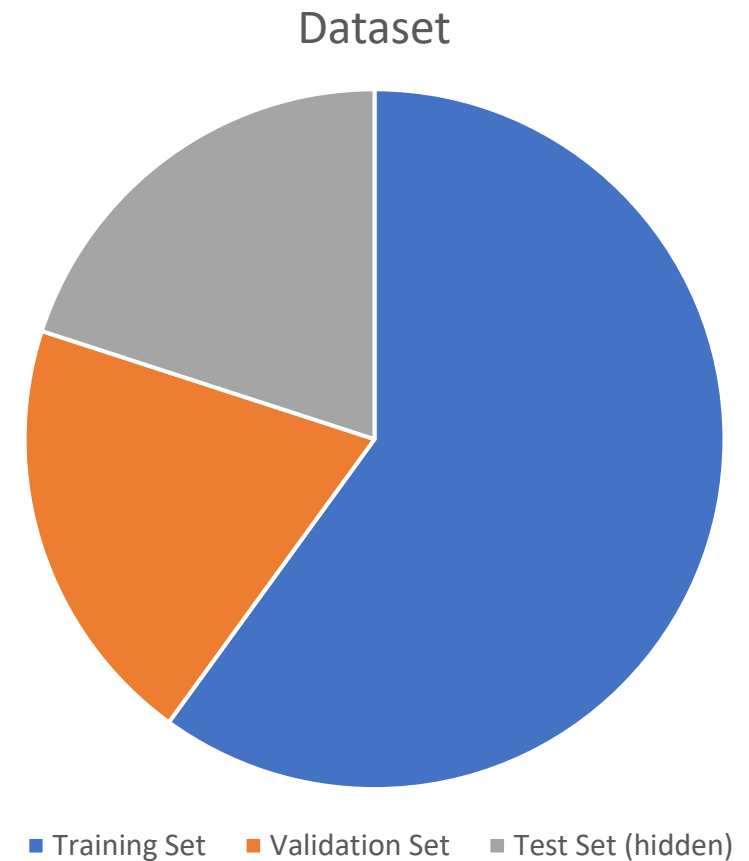
# Dataset Preparation: Training, Val, Test set

- ในการทดลอง ข้อมูลจะถูกแบ่งออกเป็น 3 ส่วน (มักจะมาจากชุดข้อมูลแหล่งเดียวกัน แต่ก็ไม่ใช่ว่าเรื่องบังคับ) ได้แก่ Training Set, Validation Set และ Test Set โดยทั้งหมดนี้จะต้องมีเฉลยคำตอบที่คาดหวังด้วย
- ในการฝึกฝน (Train) โมเดล เราจะใส่ Training Set เข้าไปในโมเดล ทั้งตัวข้อมูลและเฉลย และปรับค่า  $w$  ต่างๆ การทำงานนี้อาจมีการวนทำงานหลายรอบ ขึ้นอยู่กับชนิดของโมเดลที่เลือกใช้

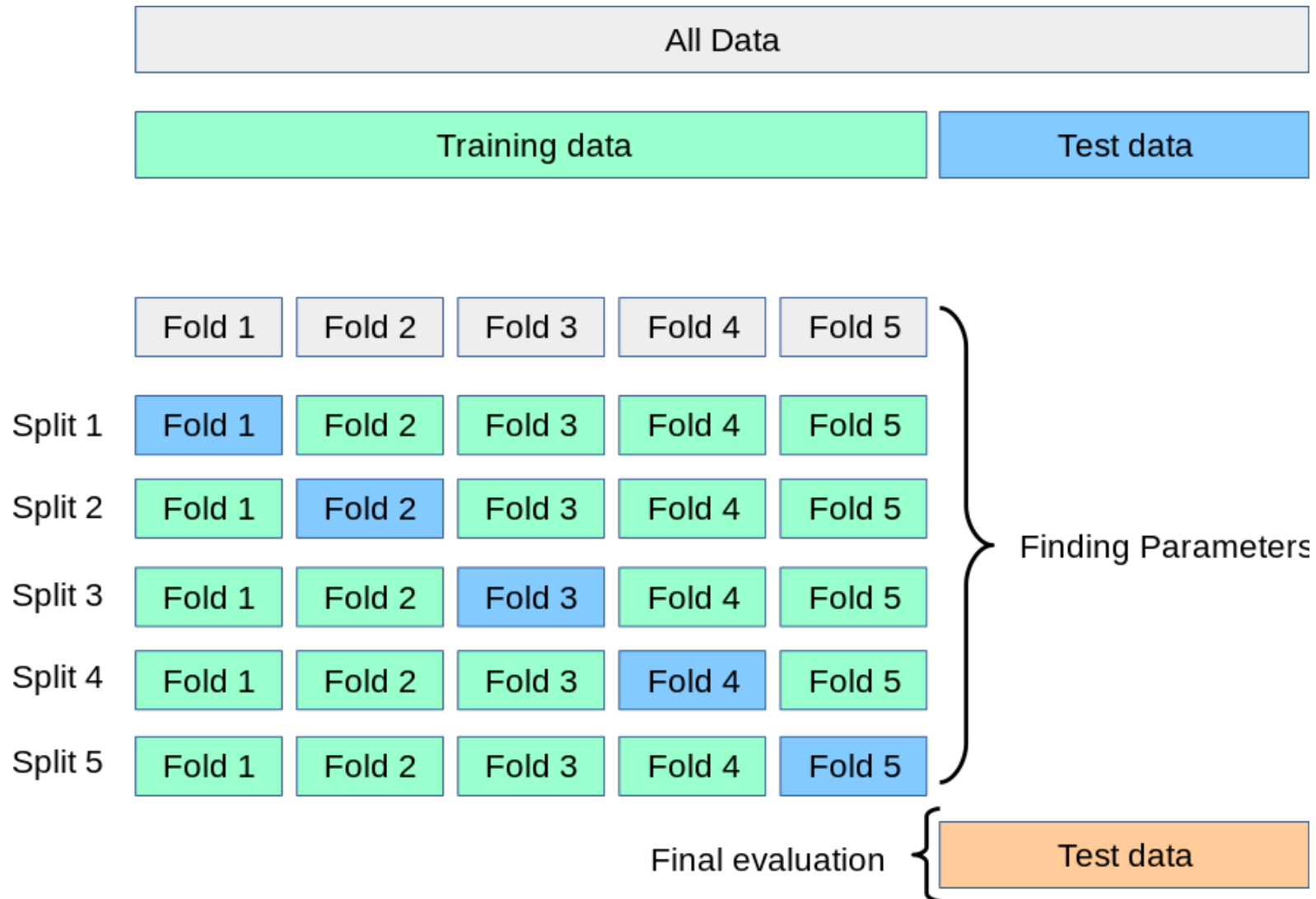


# Dataset Preparation: Training, Val, Test set

- การประเมินประสิทธิภาพเบื้องต้น จะทำโดยการนำโมเดลที่เทรนในแต่ละรอบ มาอินพุตด้วยข้อมูล Validation set ทั้งตัวข้อมูลและเฉลย ในด้านนี้ การฝึกฝนในครั้งที่มีประสิทธิภาพกับ Validation set จะถูกเลือกและนำไปใช้
- แต่การใช้ Validation set เปรียบเสมือนว่าโมเดลได้เห็นข้อมูลนี้ในระหว่างการปรับค่าโมเดลไปแล้ว เมื่อทุกอย่างเรียบร้อย เราจึงประเมินค่าด้วย Test set ที่โมเดลไม่เคยเห็นมาก่อนอีกครั้งหนึ่ง



# Dataset Preparation: k-fold cross validation



# Dataset Preparation: Model Evaluation

- True Positive (TP) คือ โมเดลบอกว่า + ความจริงเป็น + (บอกถูกต้อง)
- True Negative (TN) คือ โมเดลบอกว่า - ความจริงเป็น - (บอกถูกต้อง)
- False Positive (FP) คือ โมเดลบอกว่า + แต่ความจริงเป็น - (บอกผิด)
- False Negative (FN) คือ โมเดลบอกว่า - แต่ความจริงเป็น + (บอกผิด)

# Dataset Preparation: Confusion matrix

|                              | Condition positive | Condition negative |
|------------------------------|--------------------|--------------------|
| Predicted condition positive | True positive      | False positive     |
| Predicted condition negative | False negative     | True negative      |

ตัวอย่าง Confusion matrix ที่มีมากกว่า 2 คลาส

|      |      |      |      |      |
|------|------|------|------|------|
| 0.77 | 0.11 | 0.03 | 0.00 | 0.09 |
| 0.05 | 0.83 | 0.02 | 0.02 | 0.07 |
| 0.10 | 0.19 | 0.59 | 0.03 | 0.08 |
| 0.00 | 0.28 | 0.24 | 0.46 | 0.03 |
| 0.22 | 0.43 | 0.11 | 0.00 | 0.23 |

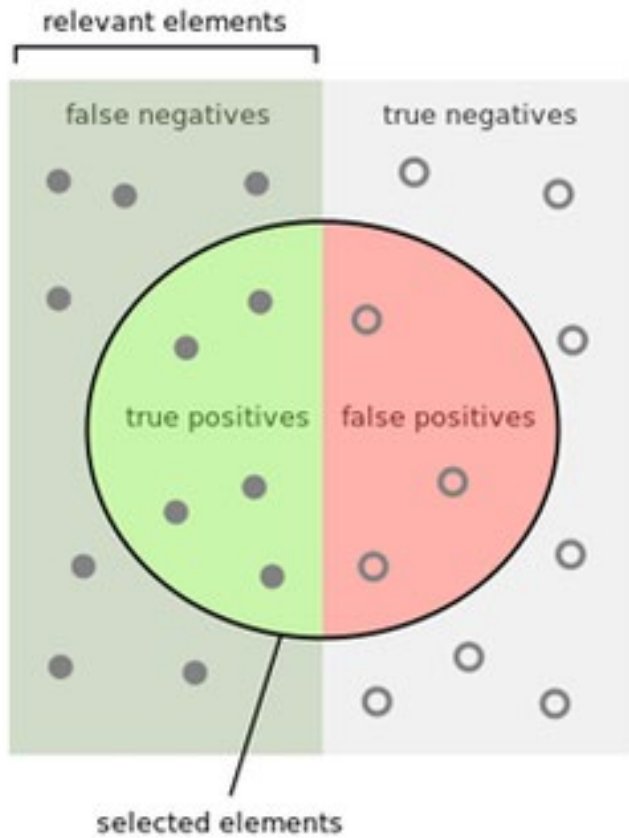
# Dataset Preparation: Confusion matrix

|                              | Condition positive | Condition negative |
|------------------------------|--------------------|--------------------|
| Predicted condition positive | True positive      | False positive     |
| Predicted condition negative | False negative     | True negative      |

ตัวอย่าง Confusion matrix ที่มี 2 คลาส

|        |     | Predicted |     |
|--------|-----|-----------|-----|
|        |     | Yes       | No  |
| Actual | Yes | 123       | 20  |
|        | No  | 33        | 161 |

# Dataset Preparation: Model Evaluation



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

# Dataset Preparation: Model Evaluation



|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

# Dataset Preparation: Model Evaluation



| Class    | Without Dimension Reduction |        | LDA       |        | PCA       |        |
|----------|-----------------------------|--------|-----------|--------|-----------|--------|
|          | Precision                   | Recall | Precision | Recall | Precision | Recall |
| tribal   | 0.6110                      | 0.5    | 0.6667    | 0.35   | 0.667     | 0.47   |
| beach    | 0.8330                      | 0.46   | 0.5       | 0.25   | 0.778     | 0.37   |
| towers   | 0.6666                      | 0.39   | 0.6667    | 0.37   | 0.667     | 0.37   |
| bus      | 0.722                       | 0.4    | 0.333     | 0.27   | 0.667     | 0.31   |
| dinosaur | 1                           | 0.97   | 1         | 0.77   | 1         | 0.97   |

# Dataset Preparation: Model Evaluation

- Loss เป็นอีกเมตริกซ์หนึ่งที่ใช้ในการวัดประสิทธิภาพของโมเดล
- แตกต่างจาก Accuracy, Precision และ Recall คือ Loss จะไม่ได้วัดค่าจากผลลัพธ์สุดท้าย แต่จะวัดความแตกต่างระหว่างค่าที่คาดหวัง และค่าที่ได้อย่างละเอียด
- นั่นทำให้ Loss บางประเภทสามารถวัดประสิทธิภาพของ Regression ได้ด้วย
- วิธีการวัดค่า Loss มีหลายวิธี (สมการ) เราเรียกสมการเหล่านี้ว่า Loss Function

# Dataset Preparation: Binary Cross Entropy Loss

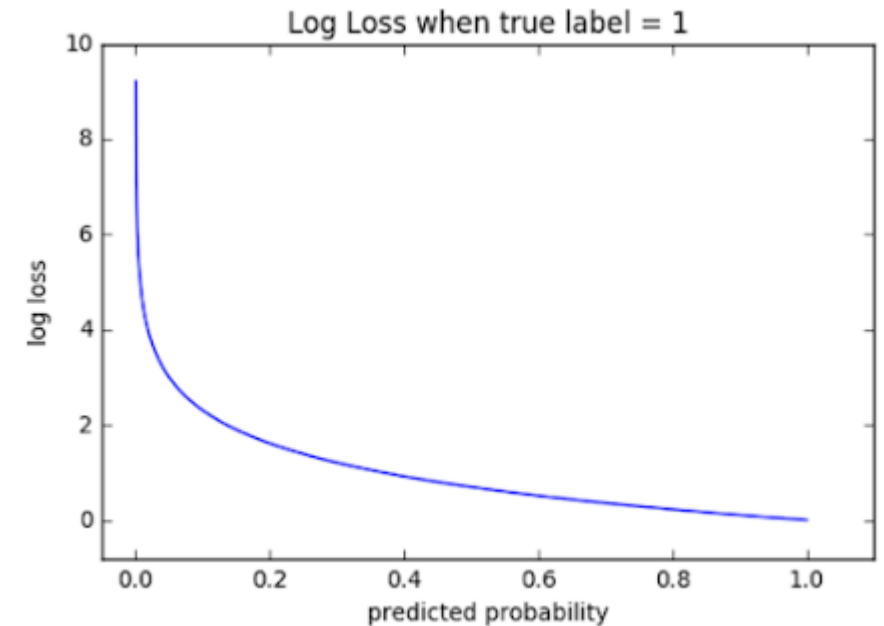
BCE มักถูกนำไปใช้ในงาน Classification เช่น Neural Network และอื่นๆ

เมื่อใช้ BCE กับงานที่เป็น 2-class จะใช้สมการนี้

$$L = -\frac{1}{m} \sum_{i=1}^m (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

เมื่อใช้ BCE กับงานที่มากกว่า 2-class จะใช้สมการนี้

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

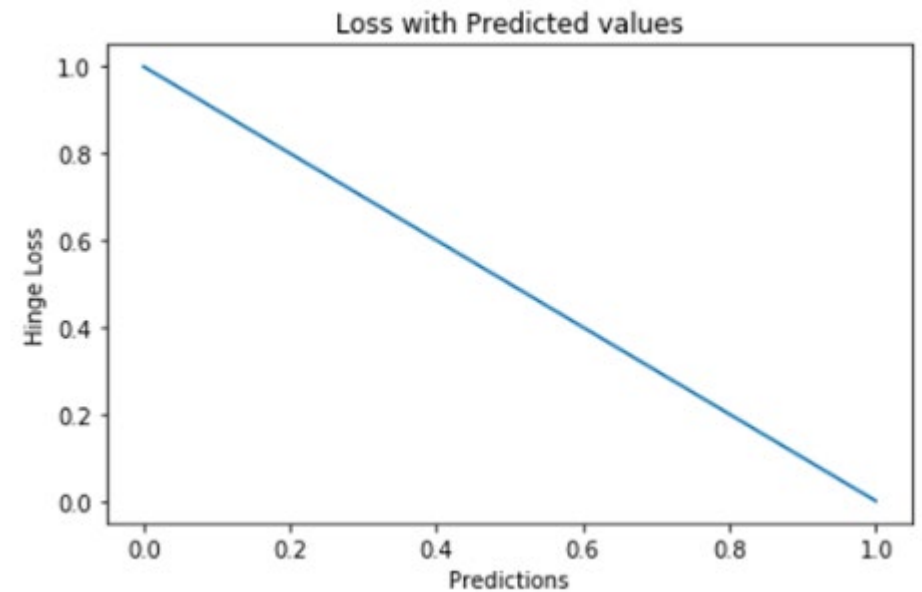


# Dataset Preparation: Hinge Loss



Hinge Loss มักถูกใช้งาน Classification ที่ใช้ SVM

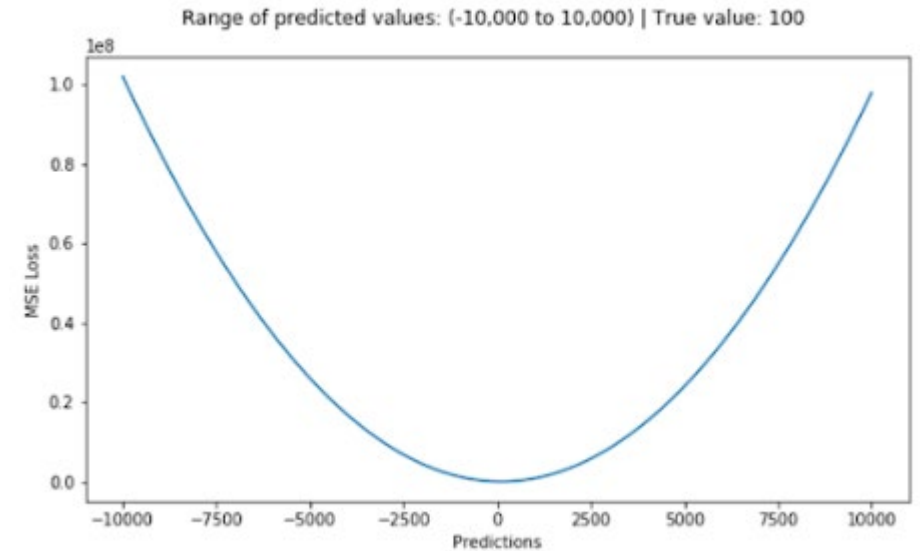
$$L = \max(0, 1 - y * f(x))$$



# Dataset Preparation: Mean Square Error Loss

MSE มักถูกใช้ในงาน Regression

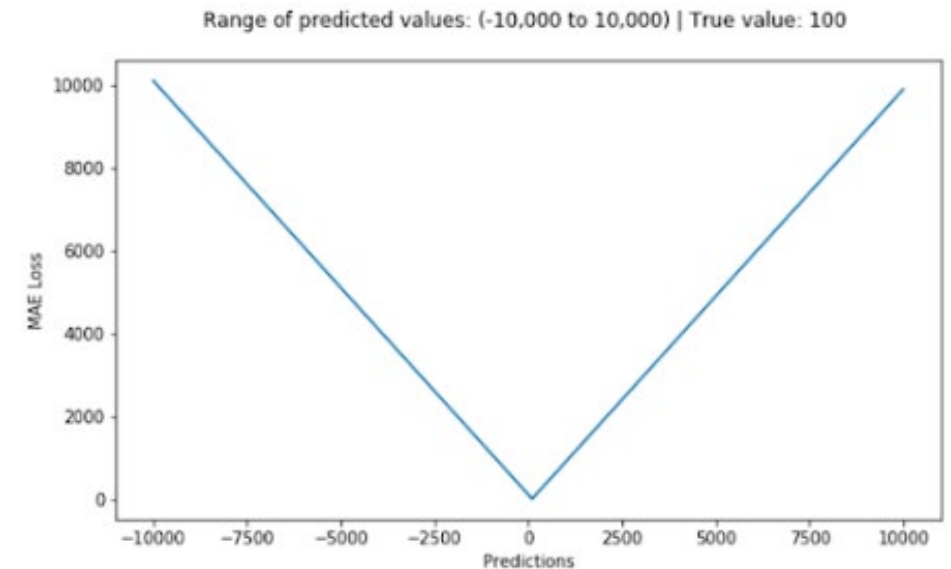
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$



# Dataset Preparation: Mean Absolute Error Loss

MAE (บางครั้งเรียกว่า L1-Loss) มักถูกใช้ในงาน Regression ที่ไม่สนใจทิศทางของข้อมูล

$$\text{MAE} = \frac{\sum_{i=1}^n |y_i - x_i|}{n}$$



# Dataset Preparation: Loss ชนิดอื่นๆ ที่น่าสนใจ



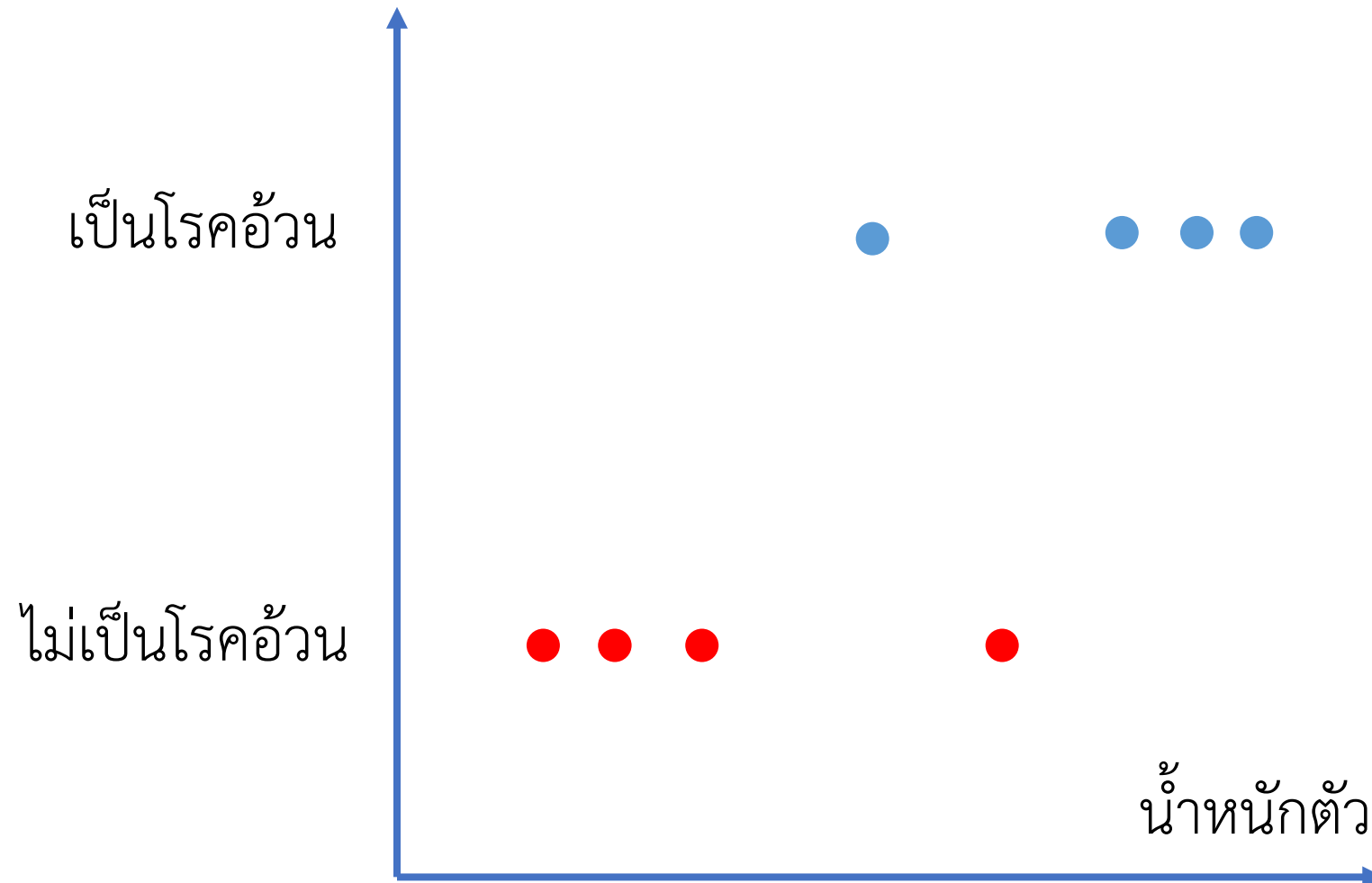
- Huber Loss (Smooth Mean Absolute Error)
- Log-Cosh Loss
- Quantile Loss

# Outline

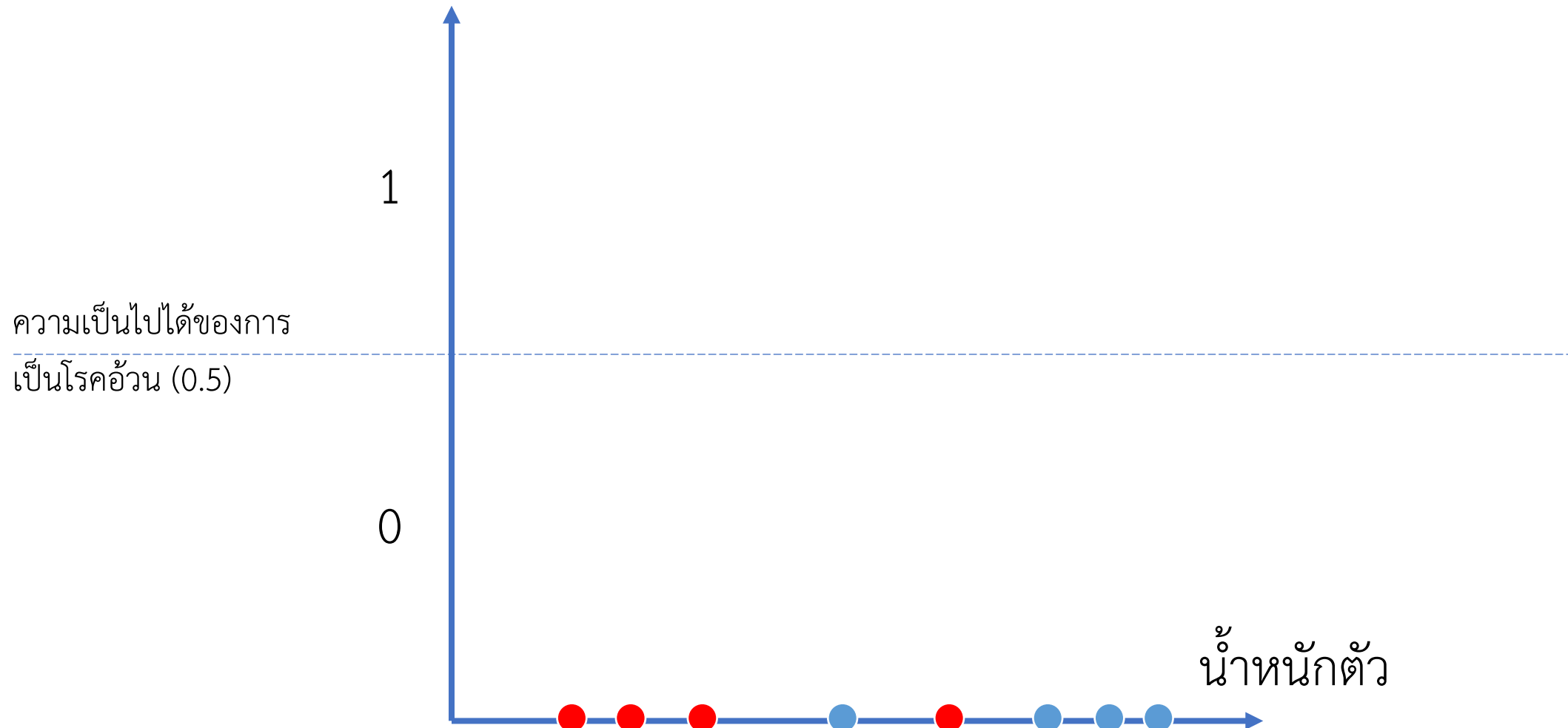


- พื้นฐานการทดลองเกี่ยวกับ Machine Learning
  - Dataset Preparation
  - Training set, Validation set, Test set
  - K-Fold Cross Validation
  - Confusion Matric, Precision, Recall, Accuracy, Loss
  - Receiver Operator Characteristic (ROC)
- Regression (การถดถอย)
  - Ridge Regression (L2)
  - Lasso Regression (L1)
  - Elastic Net Regression

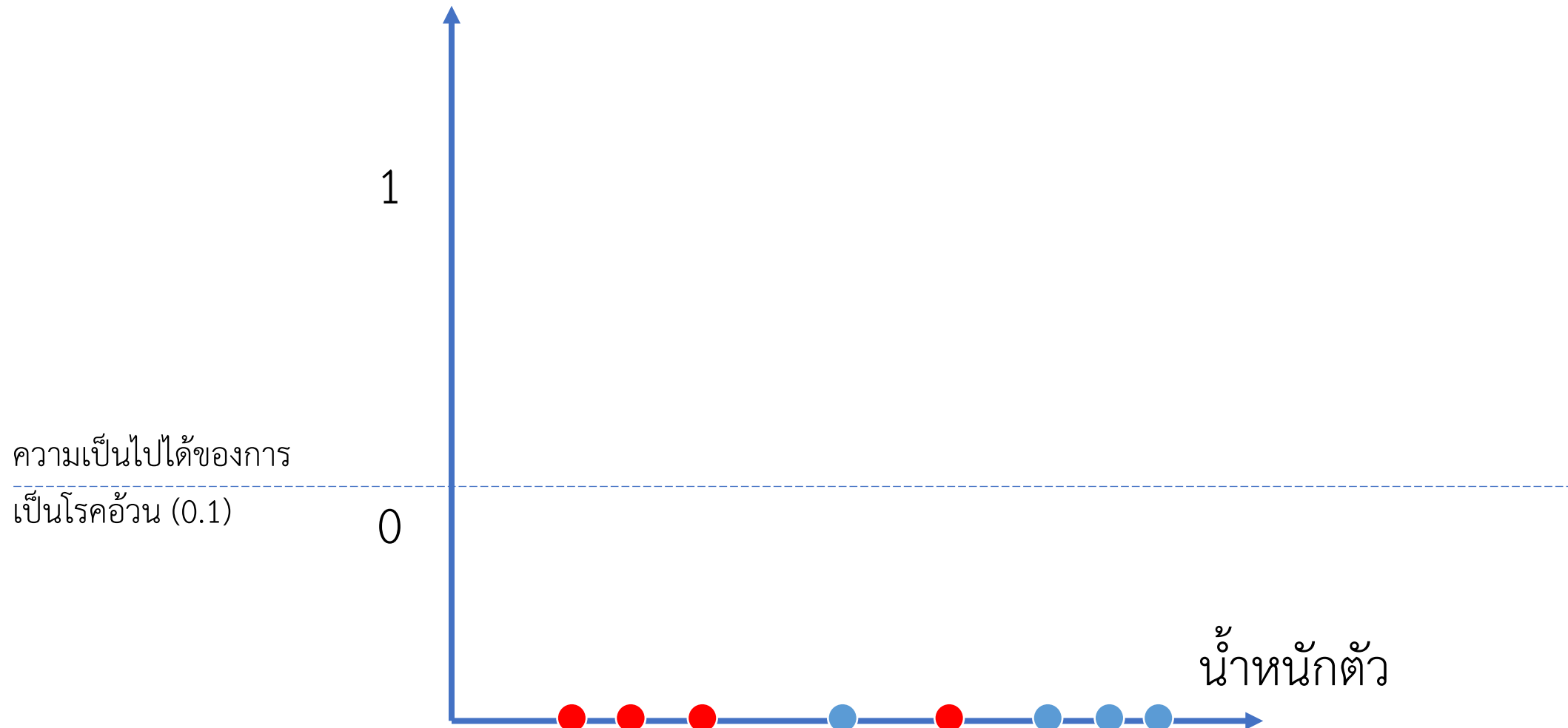
# Receiver Operator Characteristic (ROC)



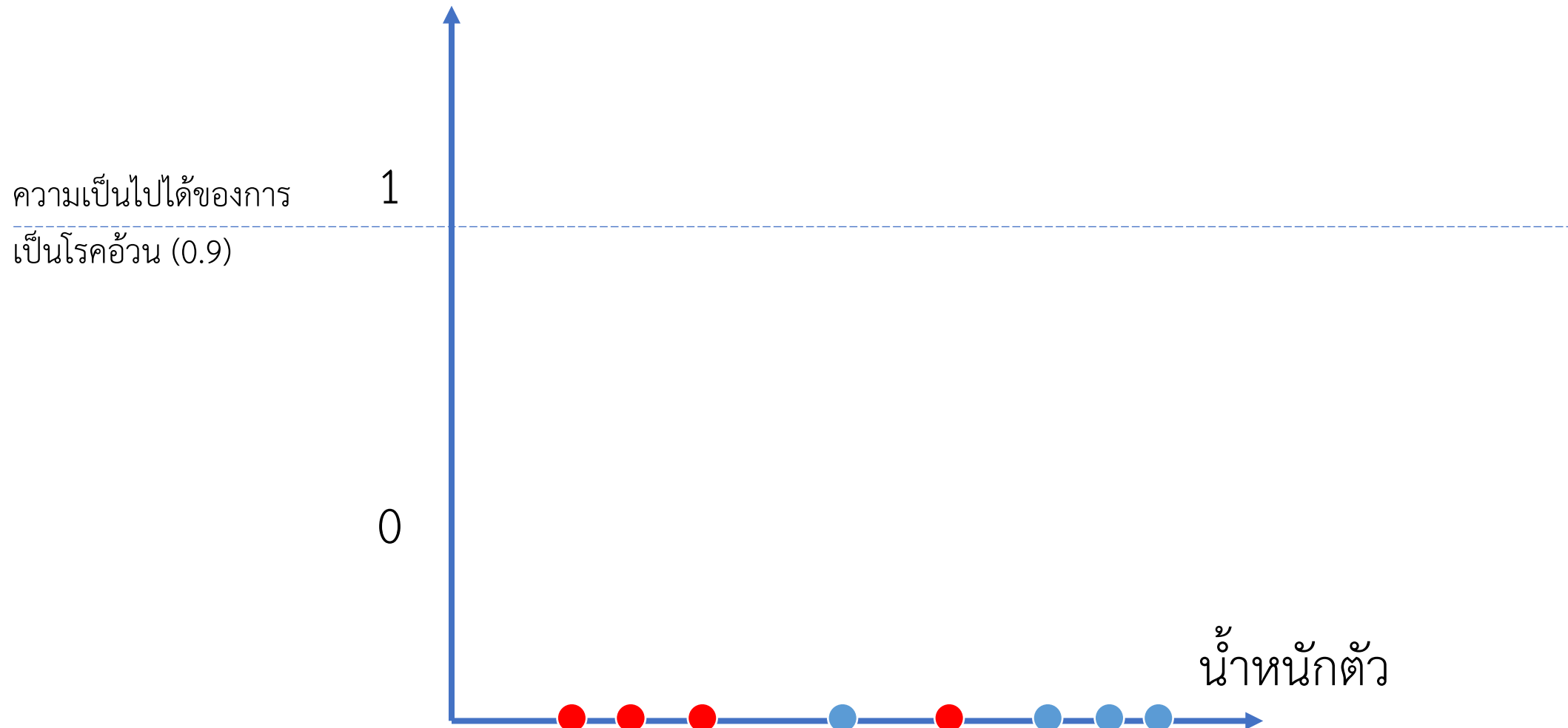
# Receiver Operator Characteristic (ROC)



# Receiver Operator Characteristic (ROC)



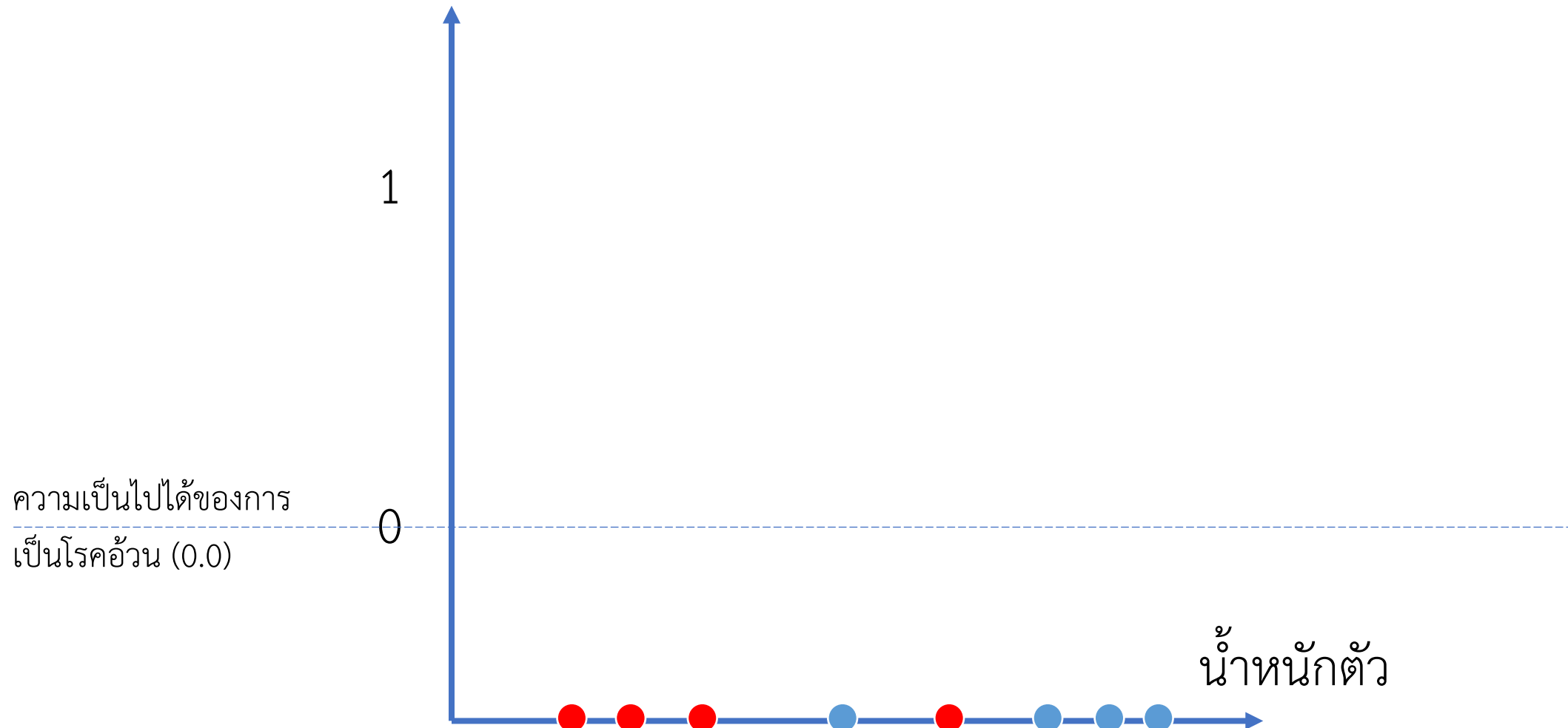
# Receiver Operator Characteristic (ROC)



# Receiver Operator Characteristic (ROC)

- ในที่นี้ เราไม่สามารถรู้ได้ว่า Threshold ที่เหมาะสม คือค่าเท่าไร
  - แม้จะรู้ Threshold ที่เหมาะสมจากการทดลองเทรนดู แต่ในตอนที่เทส เราก็ไม่รู้อยู่ดีว่าต้องกำหนด Threshold เป็นเท่าไรจึงจะเหมาะสมกับข้อมูล
- แต่เรารู้ว่า โมเดลที่สามารถทำงานได้ดีแม้ว่า Threshold จะเป็นเท่าไรก็ตาม คือโมเดลในอุดมคติ เพราะสามารถแบ่งแยกได้ทุกเงื่อนไขเสมอ
  - กล่าวคือ โมเดลมีความเสถียรสูงสุดนั่นเอง
  - (แต่ในความเป็นจริงเป็นแบบนี้ไม่ได้)

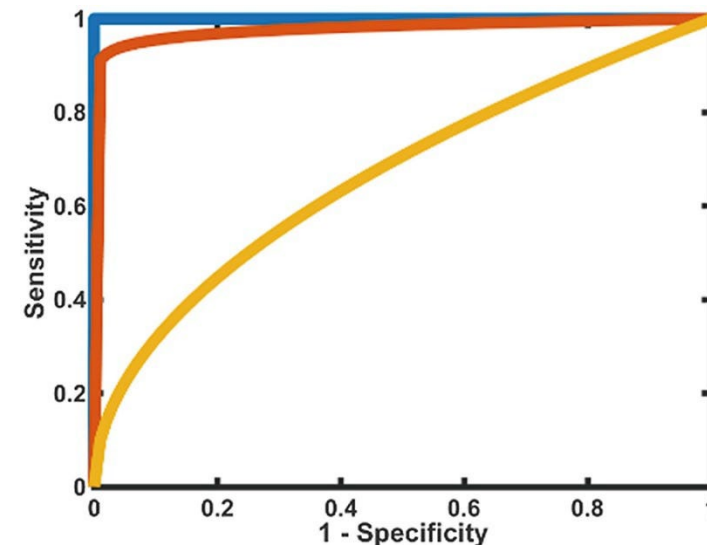
# Receiver Operator Characteristic (ROC)



# Receiver Operator Characteristic (ROC)

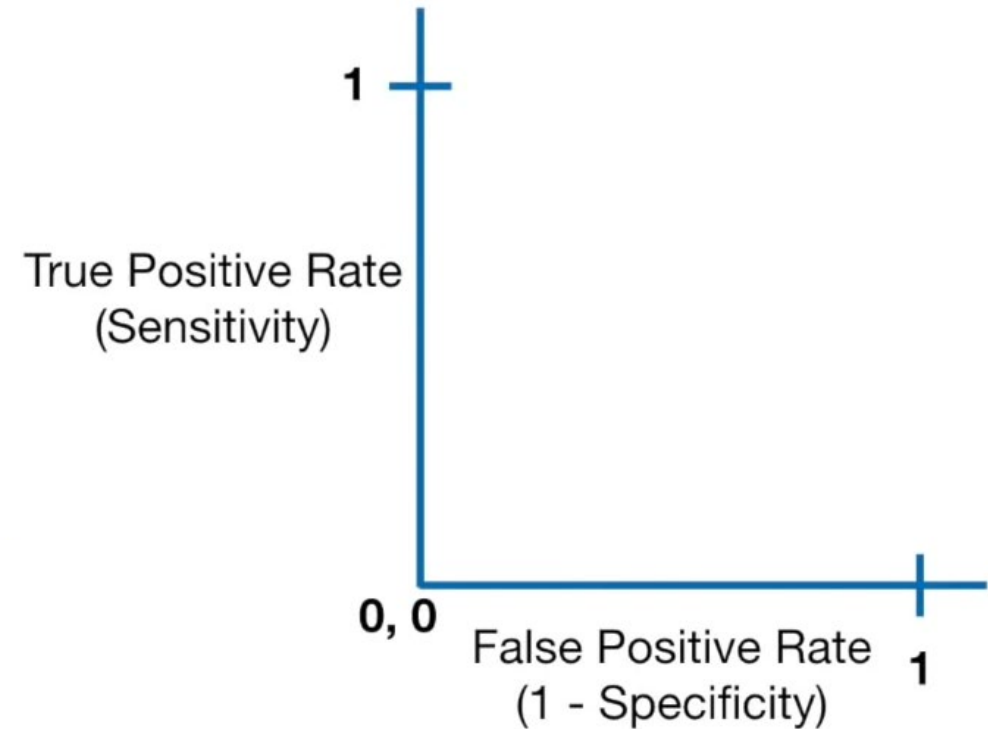
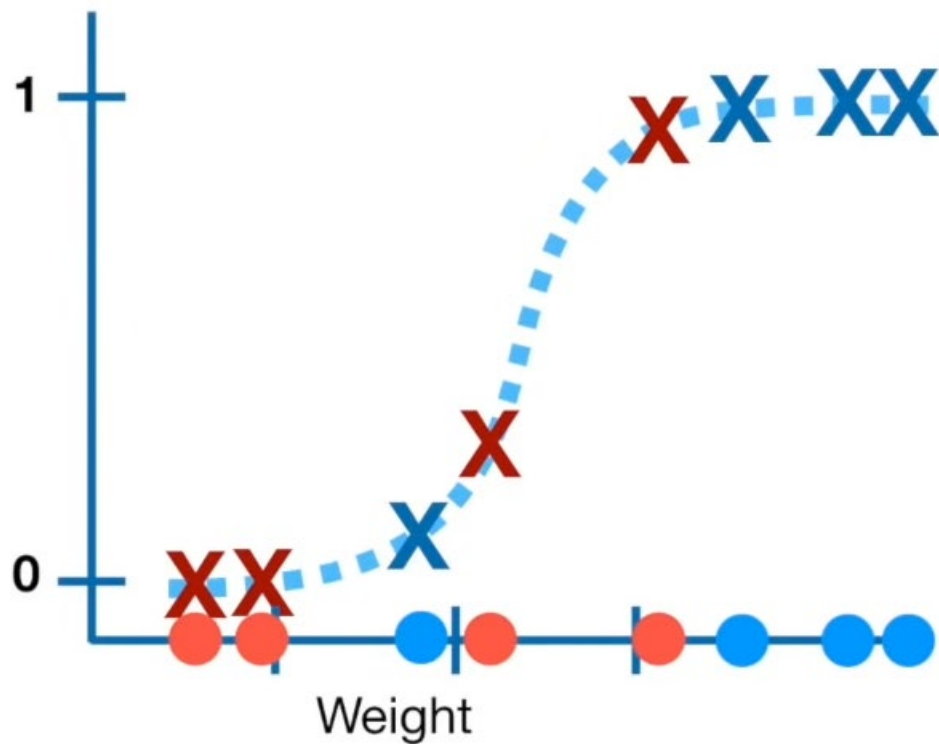
- ROC คืออะไร?
  - มันคือกราฟแสดงความสัมพันธ์ระหว่าง Sensitivity กับ 1-Specificity ที่โมเดลยอมให้เกิดขึ้นได้
  - ถ้าแปลให้ง่าย ๆ ก็คือ ความสัมพันธ์ระหว่าง Sensitivity กับ 1-Specificity เมื่อปรับ Threshold เป็นค่าต่าง ๆ นั้นเอง (Threshold 1 ค่า ก็จะได้ 1 จุดบนกราฟ)

|              |          | Predicted Class                            |                                                            |                                                          |
|--------------|----------|--------------------------------------------|------------------------------------------------------------|----------------------------------------------------------|
|              |          | Positive                                   | Negative                                                   |                                                          |
| Actual Class | Positive | True Positive (TP)                         | False Negative (FN)<br><b>Type II Error</b>                | <b>Sensitivity</b><br>$\frac{TP}{(TP + FN)}$             |
|              | Negative | False Positive (FP)<br><b>Type I Error</b> | True Negative (TN)                                         | <b>Specificity</b><br>$\frac{TN}{(TN + FP)}$             |
|              |          | <b>Precision</b><br>$\frac{TP}{(TP + FP)}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{(TN + FN)}$ | <b>Accuracy</b><br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |



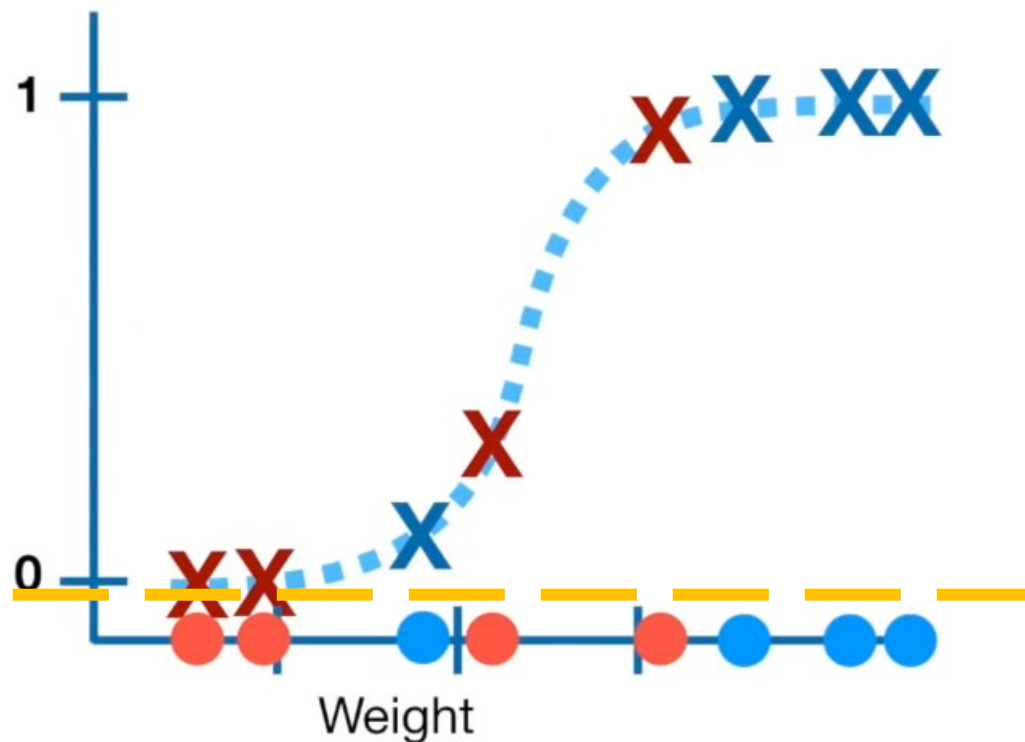
# Receiver Operator Characteristic (ROC)

- วิธีการสร้างกราฟ ROC



# Receiver Operator Characteristic (ROC)

- วิธีการสร้างกราฟ ROC
  - กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้



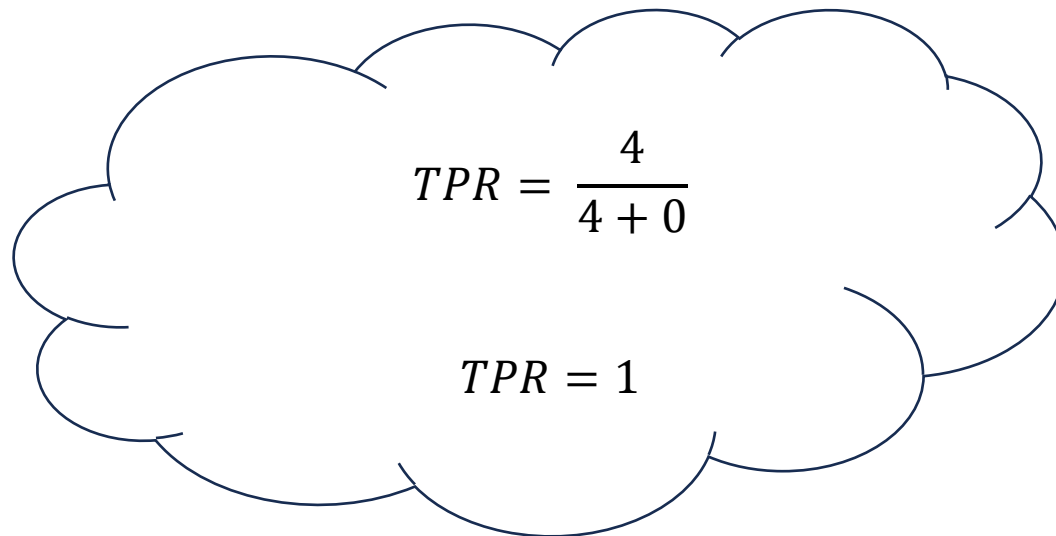
|           |         | Actual |         |
|-----------|---------|--------|---------|
|           |         | อ่าน   | ไม่อ่าน |
| Predicted | อ่าน    | 4      | 4       |
|           | ไม่อ่าน | 0      | 0       |

# Receiver Operator Characteristic (ROC)

- วิธีการสร้างกราฟ ROC

- กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
- คำนวณ Sensitivity (True Positive Rate)

- $TPR = \frac{TP}{TP+FN}$


$$TPR = \frac{4}{4 + 0}$$
$$TPR = 1$$

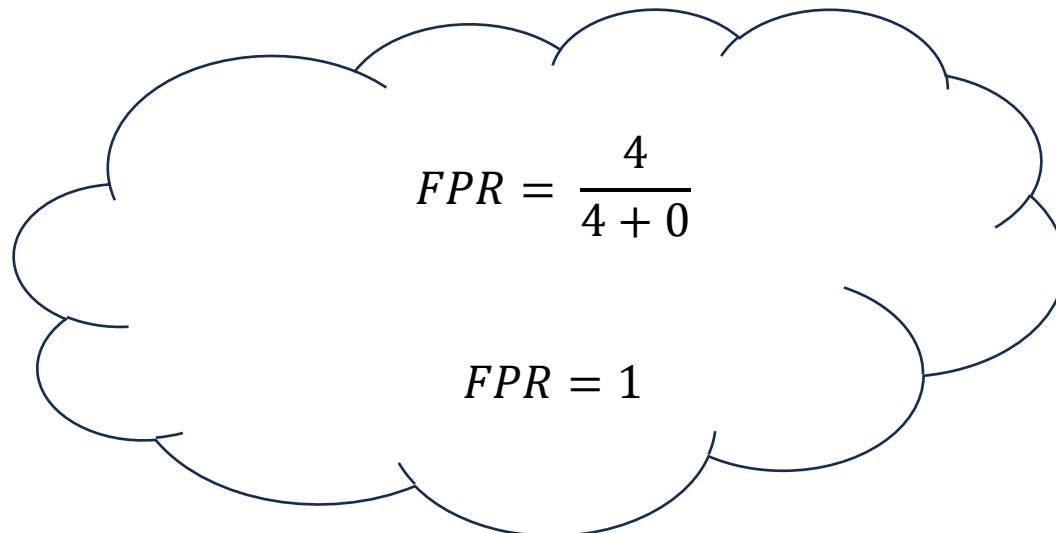
|           |         | Actual |         |
|-----------|---------|--------|---------|
|           |         | อ่าน   | ไม่อ่าน |
| Predicted | อ่าน    | 4      | 4       |
|           | ไม่อ่าน | 0      | 0       |

# Receiver Operator Characteristic (ROC)

## • วิธีการสร้างกราฟ ROC

- กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
- คำนวณ Sensitivity (True Positive Rate)
- คำนวณ 1-Specificity (False Positive Rate)

$$FPR = \frac{FP}{FP+TN}$$



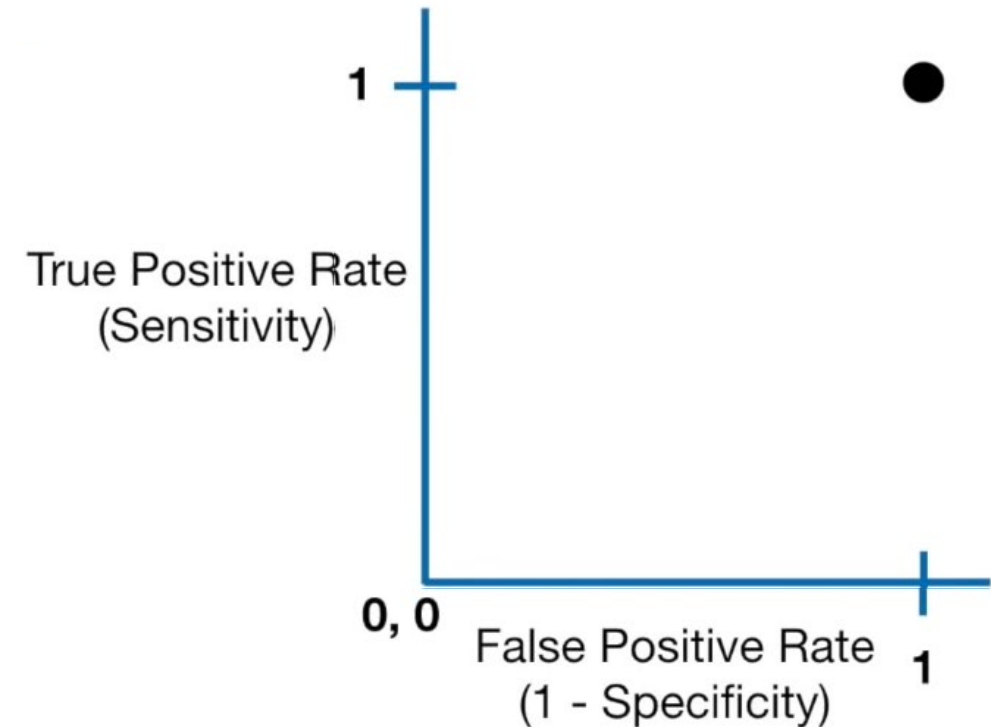
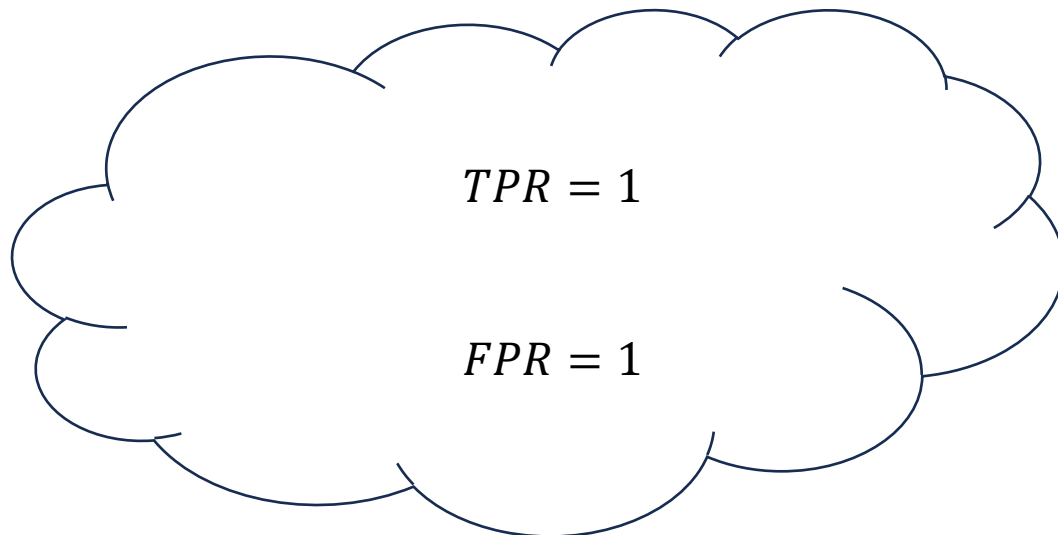
$$FPR = \frac{4}{4 + 0}$$

$$FPR = 1$$

|           |         | Actual |         |
|-----------|---------|--------|---------|
|           |         | อ่าน   | ไม่อ่าน |
| Predicted | อ่าน    | 4      | 4       |
|           | ไม่อ่าน | 0      | 0       |

# Receiver Operator Characteristic (ROC)

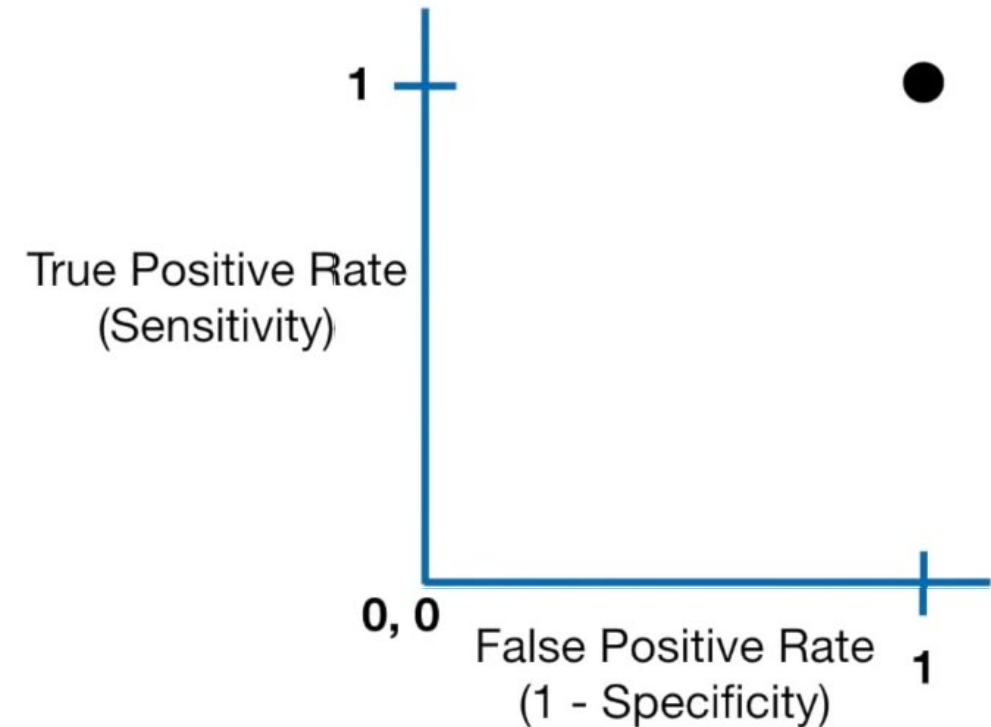
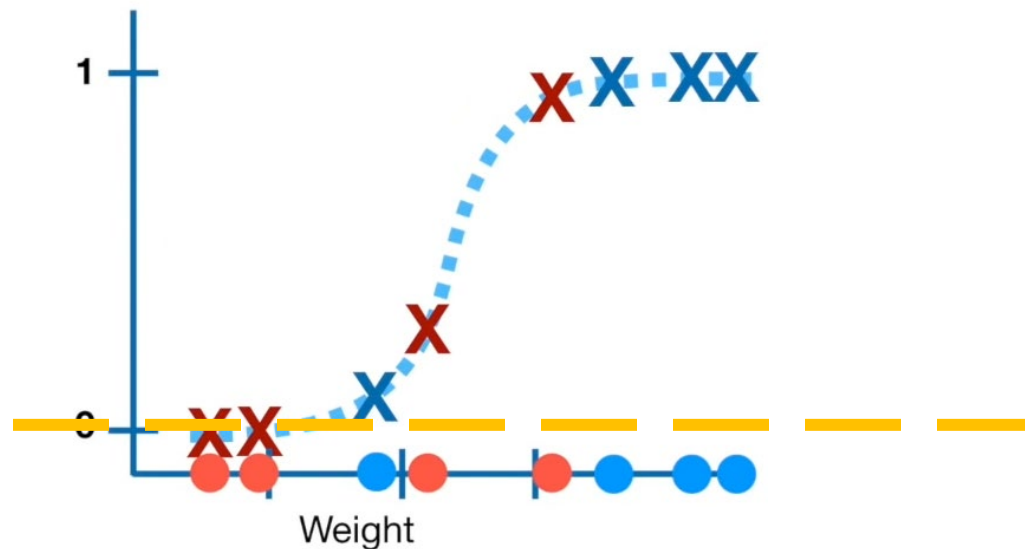
- วิธีการสร้างกราฟ ROC
  - กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
  - คำนวณ Sensitivity (True Positive Rate)
  - คำนวณ 1-Specificity (False Positive Rate)
  - Plot ลงกราฟ



# Receiver Operator Characteristic (ROC)

## • วิธีการสร้างกราฟ ROC

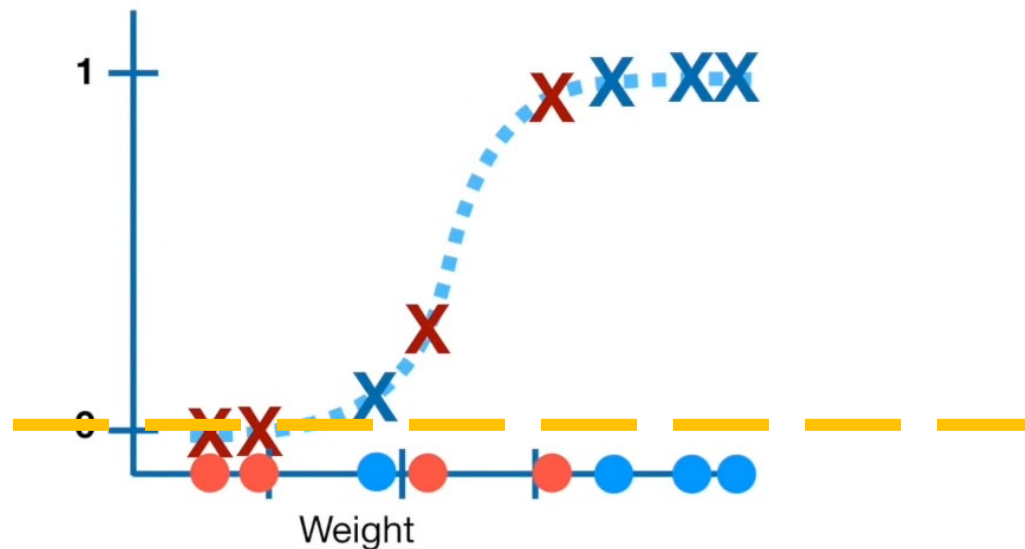
- กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
- คำนวณ Sensitivity (True Positive Rate)
- คำนวณ 1-Specificity (False Positive Rate)
- Plot ลงกราฟ
- เพิ่ม Threshold ให้สูงขึ้น และสร้าง Confusion Matrix ใหม่ และทำแบบเดิมซ้ำ



# Receiver Operator Characteristic (ROC)

## • วิธีการสร้างกราฟ ROC

- กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
- คำนวณ Sensitivity (True Positive Rate)
- คำนวณ 1-Specificity (False Positive Rate)
- Plot ลงกราฟ
- เพิ่ม Threshold ให้สูงขึ้น และสร้าง Confusion Matrix ใหม่ และทำแบบเดิมซ้ำ



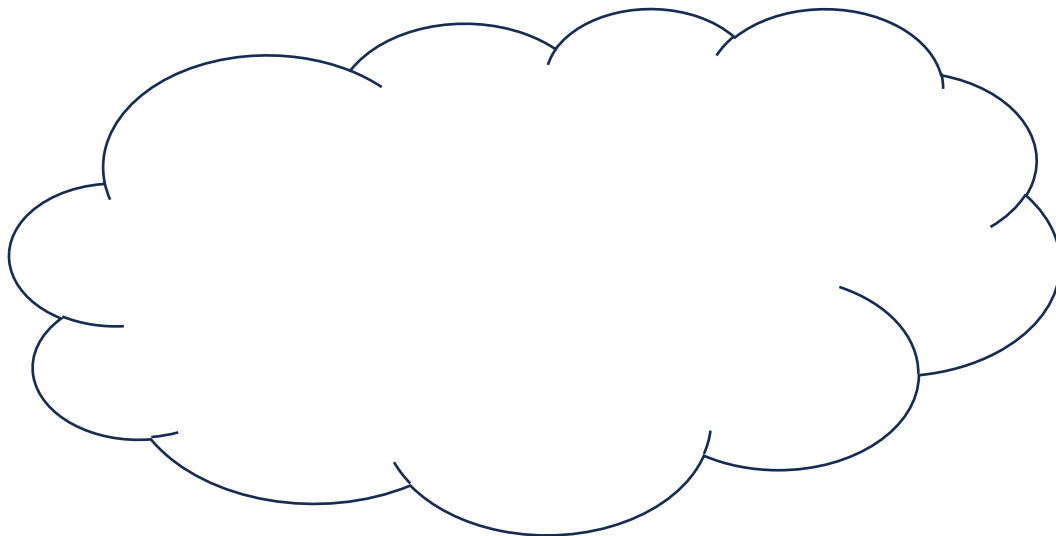
|           |         | Actual |         |
|-----------|---------|--------|---------|
|           |         | อ่าน   | ไม่อ่าน |
| Predicted | อ่าน    |        |         |
|           | ไม่อ่าน |        |         |

# Receiver Operator Characteristic (ROC)

- วิธีการสร้างกราฟ ROC

- กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
- คำนวณ TPR และ FPR

$$\bullet \quad TPR = \frac{TP}{TP+FN} \quad FPR = \frac{FP}{FP+TN}$$

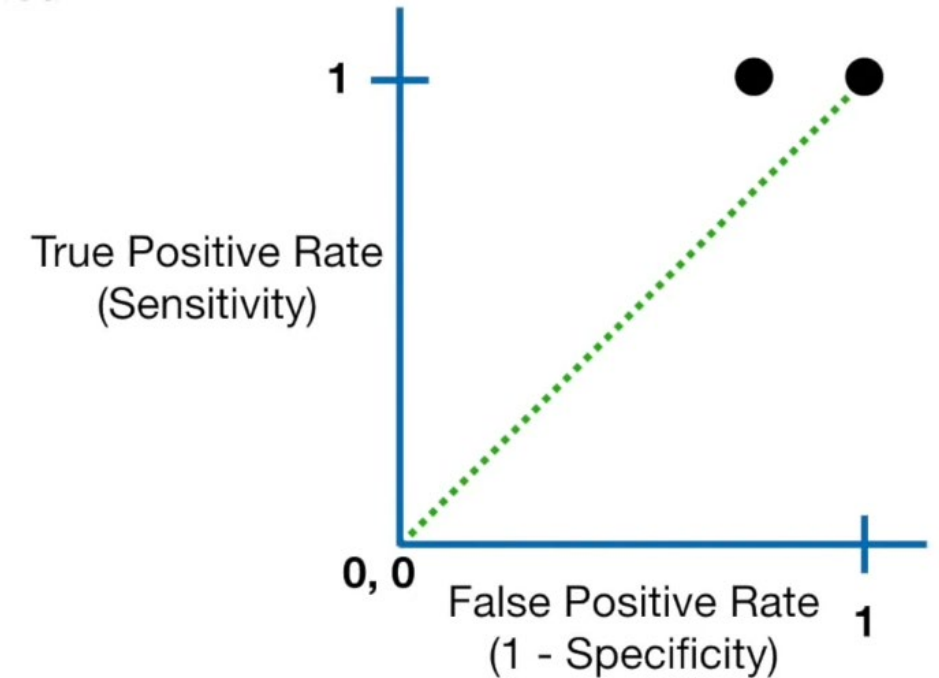
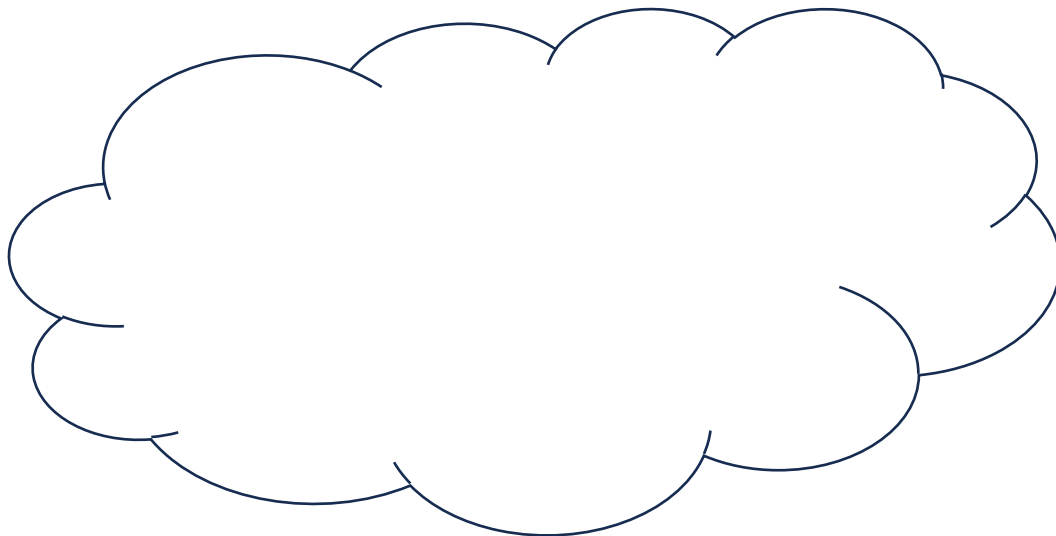


|           |         | Actual |         |
|-----------|---------|--------|---------|
|           |         | อ่าน   | ไม่อ่าน |
| Predicted | อ่าน    | 4      | 4       |
|           | ไม่อ่าน | 0      | 0       |

# Receiver Operator Characteristic (ROC)

- วิธีการสร้างกราฟ ROC

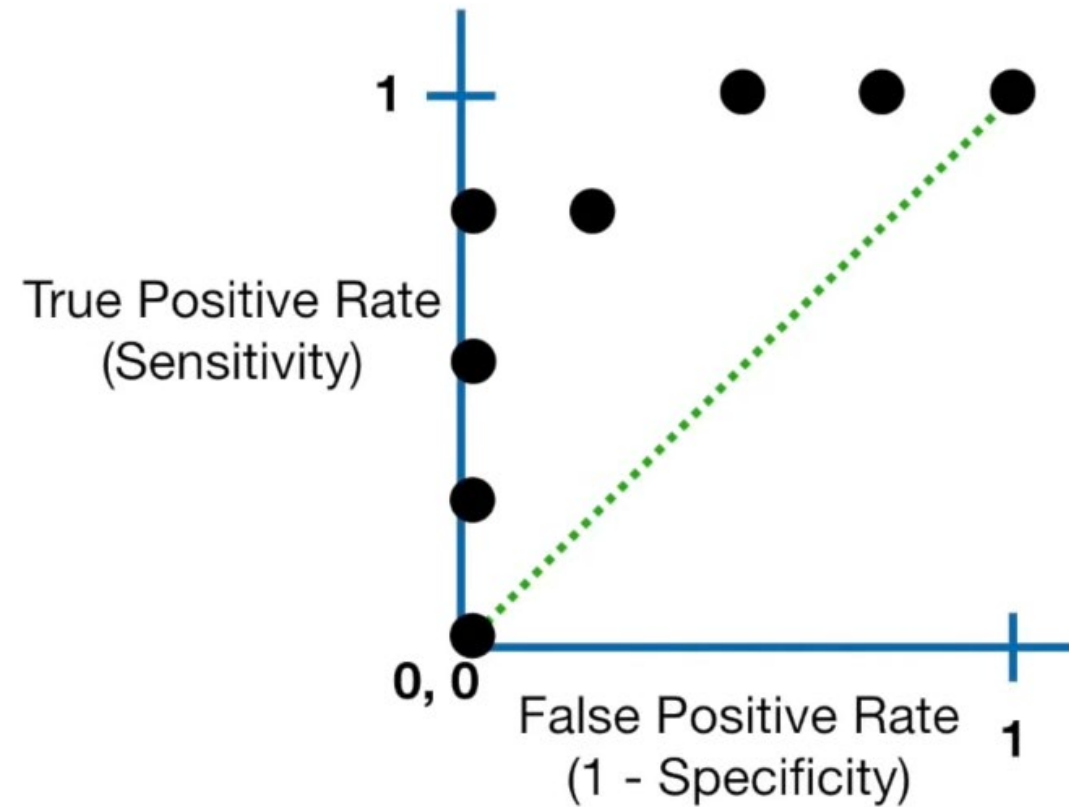
- กำหนด Threshold ต่ำ ๆ และสร้าง Confusion matrix ในกรณีนี้
- คำนวณ TPR และ FPR
  - $TPR = \frac{TP}{TP+FN}$        $FPR = \frac{FP}{FP+TN}$
- Plot ลงกราฟ



# Receiver Operator Characteristic (ROC)



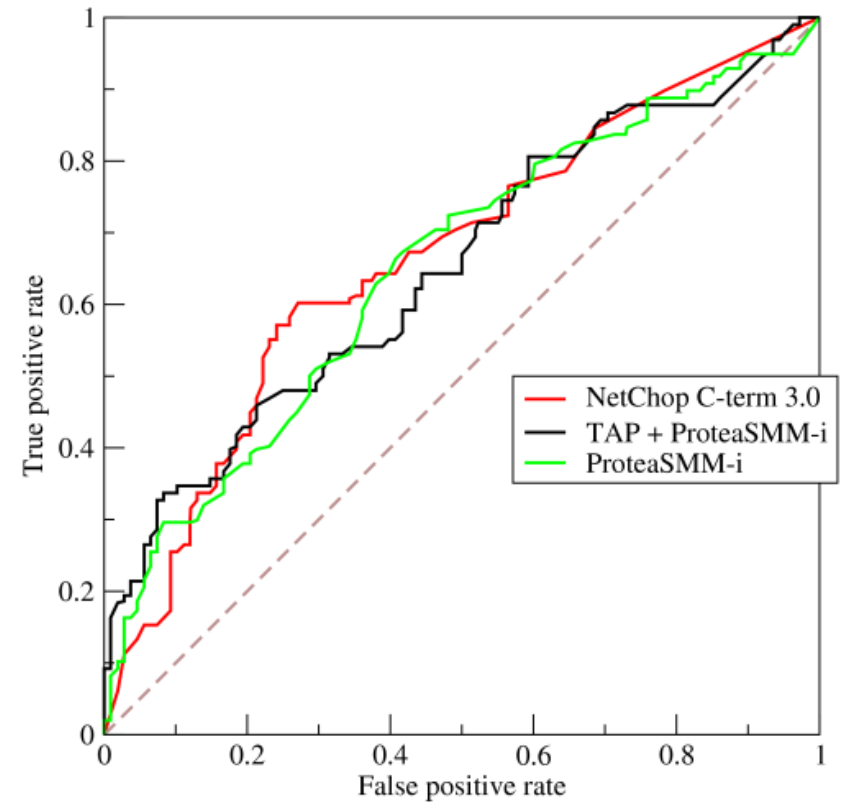
- วิธีการสร้างกราฟ ROC



# Receiver Operator Characteristic (ROC)



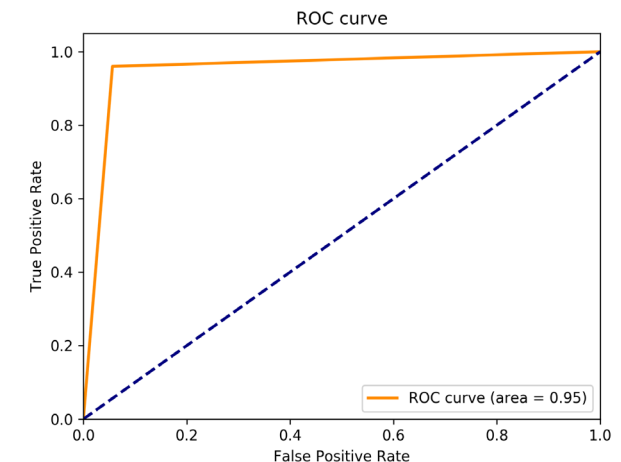
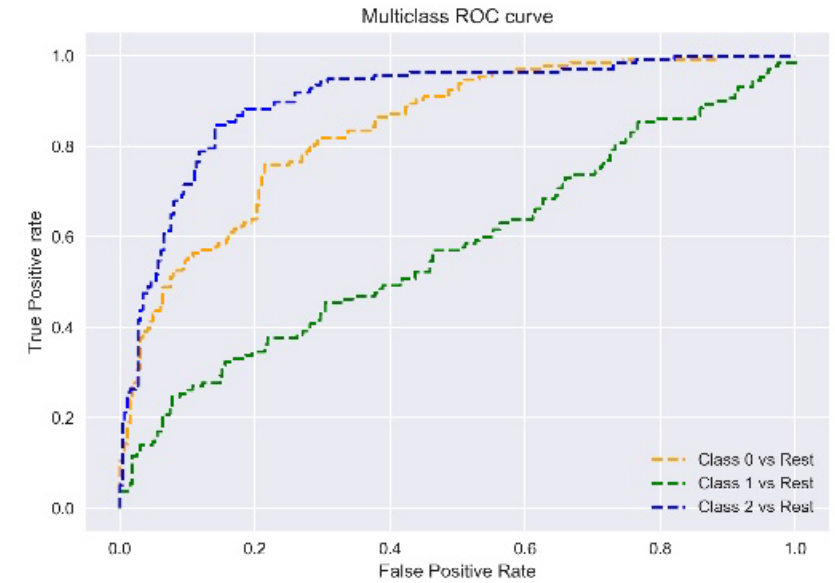
- นอกจาก ROC แล้ว อีกตัวที่จะพบได้คือ AUC
  - เพราะ ROC เป็นกราฟ เวลาเปรียบเทียบมันเลยยาก
    - เหตุผลคล้ายกับ Confusion matrix
  - นักวิจัยจึงนำเสนอสิ่งที่เรียกว่า AUC (Area Under the Curve)
    - กราฟเส้นไหนให้ AUC เยอะกว่า ก็น่าจะเสถียรมากกว่า



# Receiver Operator Characteristic (ROC)



- Fact ของ ROC
  - ROC ของโมเดลที่เพอร์เฟค จะต้องชิดมุมซ้ายบน (แทบเป็นไปไม่ได้)
  - ยิ่งชิดมุมซ้ายบนมากกว่า คือดีกว่า
  - โมเดลที่มั่วมาและใช้ไม่ได้ จะให้ผลลัพธ์ขีดเส้นทแยง ( $AUC = 0.5$ )
    - ไม่ต่างจากเดาสุ่ม
  - โมเดลที่ AUC ต่ำกว่า 0.5 มี 2 กรณี
    - โมเดลที่เดาสุ่ม แล้วยังเดาผิด
    - ทำกราฟผิด หรือคำนวณ TPR หรือ FPR ผิด
  - ถ้ามีหลายคลาส ก็ต้องทำแยกเส้นกัน เหมือนภาพทางขวา
  - ถ้าเป็นโมเดลที่ไม่ได้ให้ผลลัพธ์เป็นความน่าจะเป็น จะทำ ROC ได้กราฟน้อยเหลื่อม



# Outline



- พื้นฐานการทดลองเกี่ยวกับ Machine Learning
  - Dataset Preparation
  - Training set, Validation set, Test set
  - K-Fold Cross Validation
  - Confusion Matric, Precision, Recall, Accuracy, Loss
- Regression (การถดถอย)
  - Ridge Regression (L2)
  - Lasso Regression (L1)
  - Elastic Net Regression

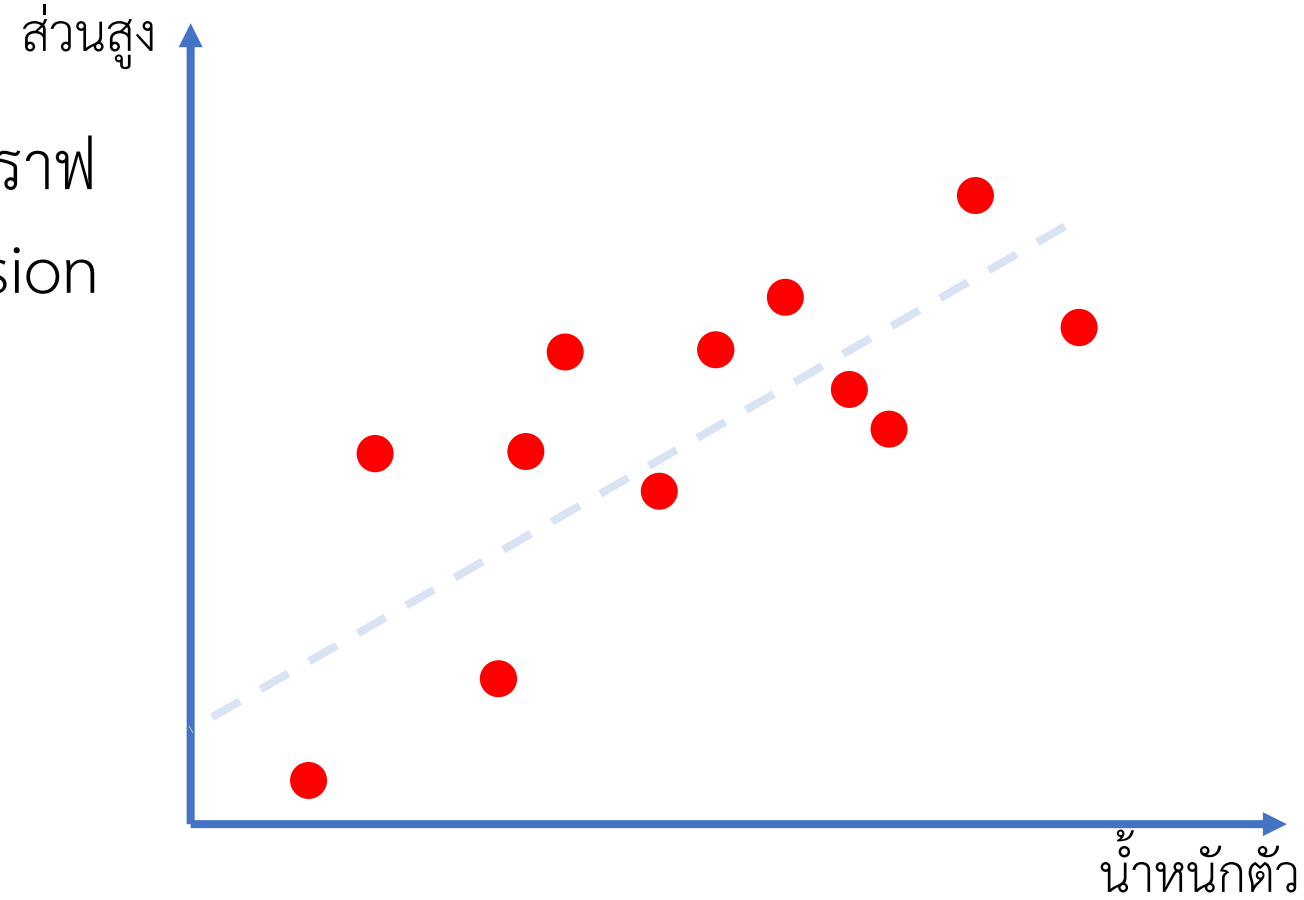
# Ridge Regression

- ในบทนี้ เราจะเน้นไปที่การพัฒนาโมเดลที่เราได้เรียนมาแล้วก่อนหน้านี้ ไม่ใช่โมเดลใหม่
- Ridge Regression สามารถใช้พัฒนาประสิทธิภาพของ Linear Regression ได้ในกรณีส่วนใหญ่
- ด้วยความที่เราใช้ Ridge Regression ในการพัฒนา Regression เป็นหลัก โมเดลนี้จึงสามารถใช้ในงาน Regression ได้ (ต้องได้อยู่แล้วโดยธรรมชาติ)

# Ridge Regression



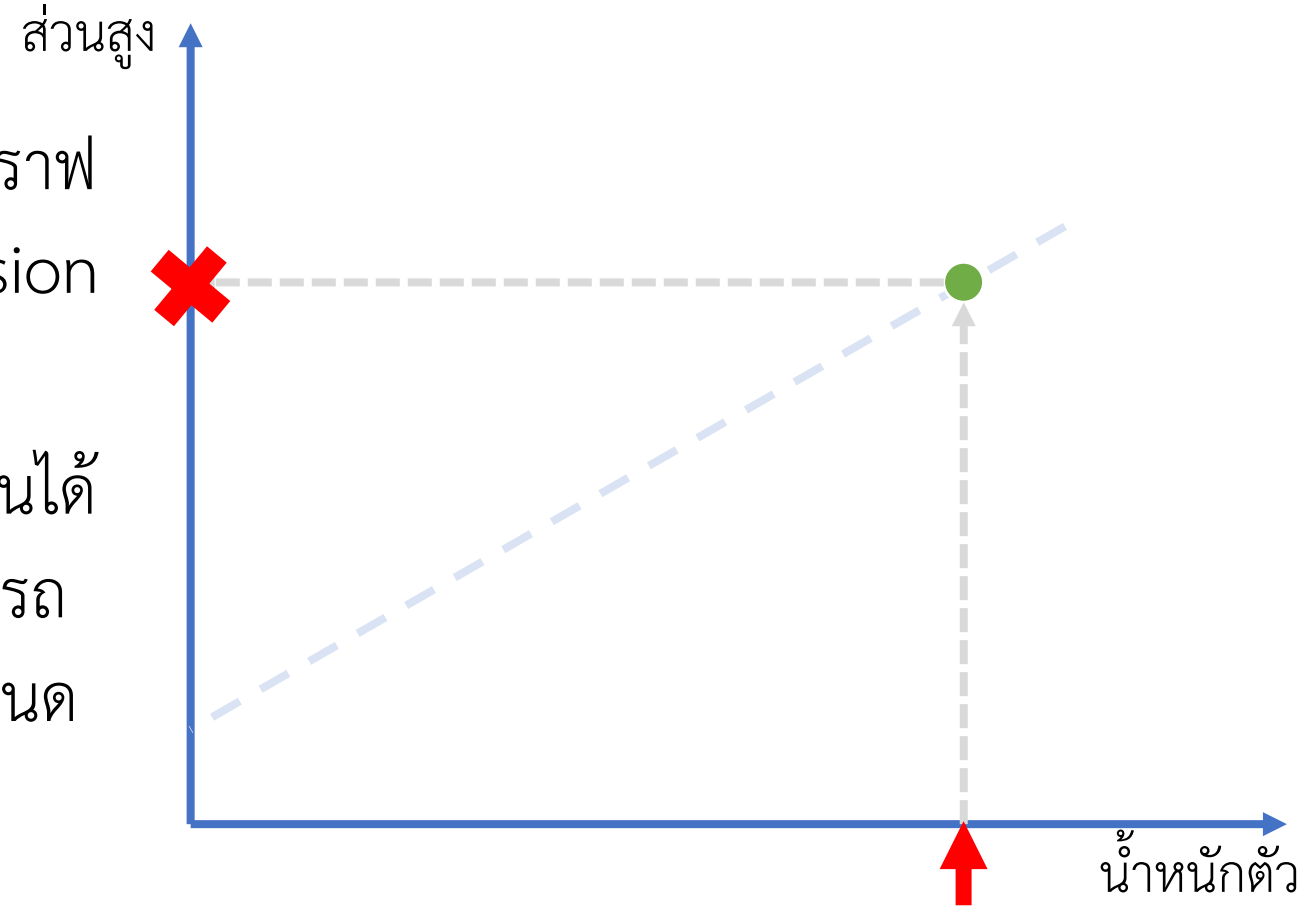
- กลับไปที่ Linear Regression
  - ถ้าเป็นตอนนี้ เราคงเข้าใจที่มาของกราฟตัวอย่างที่สร้างจาก Linear Regression ได้ไม่ยาก



# Ridge Regression

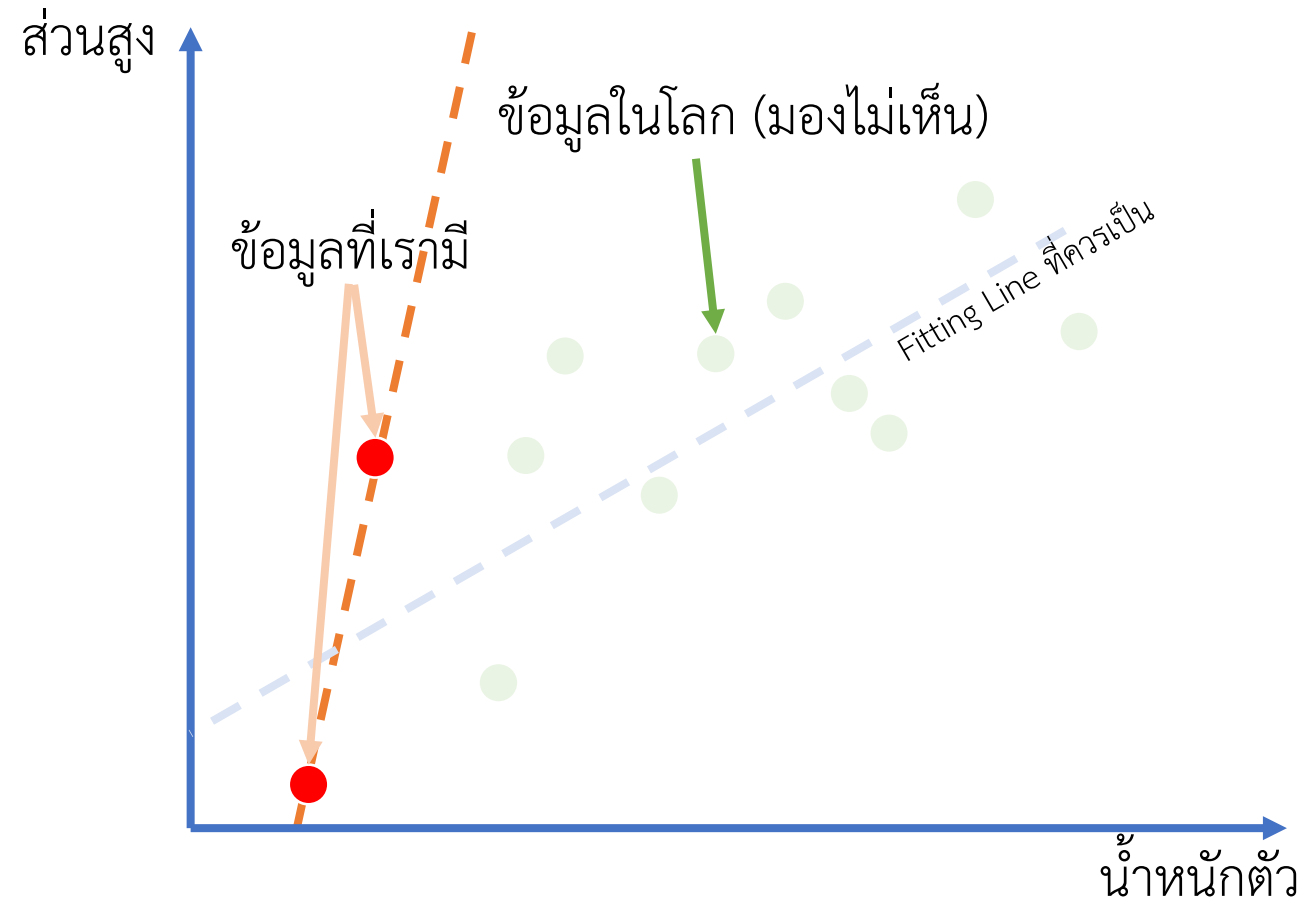


- กลับไปที่ Linear Regression
  - ถ้าเป็นตอนนี้ เราคงเข้าใจที่มาของกราฟตัวอย่างที่สร้างจาก Linear Regression ได้ไม่ยาก
  - เมื่อเทรนโมเดลจาก Training set จนได้ Fitting Line ที่เหมาะสม เราก็สามารถประมาณหาส่วนสูงจากน้ำหนักที่กำหนดได้



# Ridge Regression

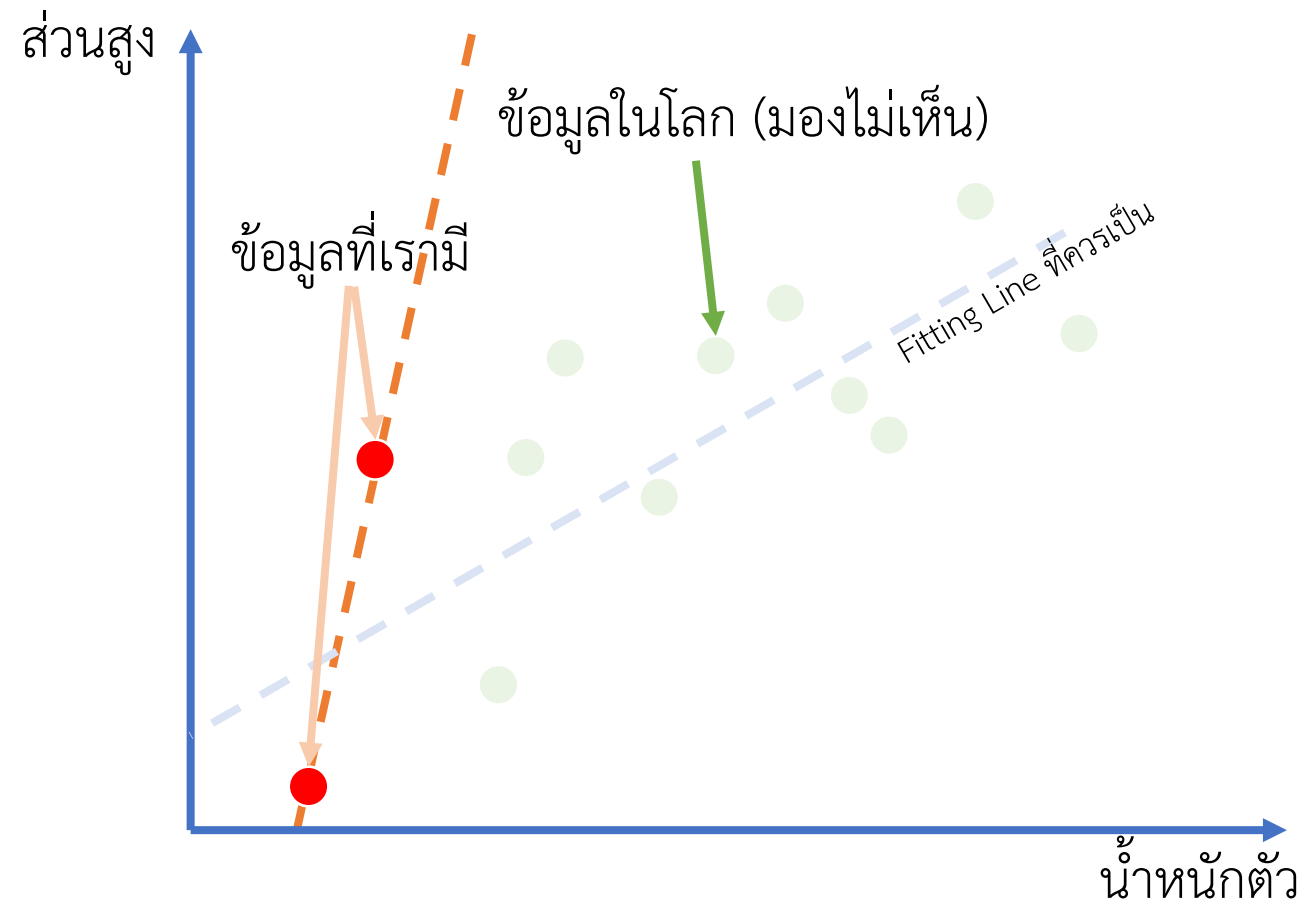
- กลับไปที่ Linear Regression
  - ที่นี้ลองจินตนาการถึงความเป็นจริงว่า ข้อมูลที่เรามี มักจะมีจำนวนน้อยมากเมื่อเทียบกับข้อมูลในโลกจริง (เช่นเราอาจสุ่มประชากรตัวอย่างมา 1000 คน เหมือนจะเยอะแล้ว แต่ในโลกเกือบ 7000 ล้านคน) เมื่อเราทำการศึกษาจริงในดาต้าเซตที่เล็กกว่า ข้อมูลจริงมาก ๆ ก็จะทำให้เกิดเหตุการณ์ต่อไปนี้



# Ridge Regression



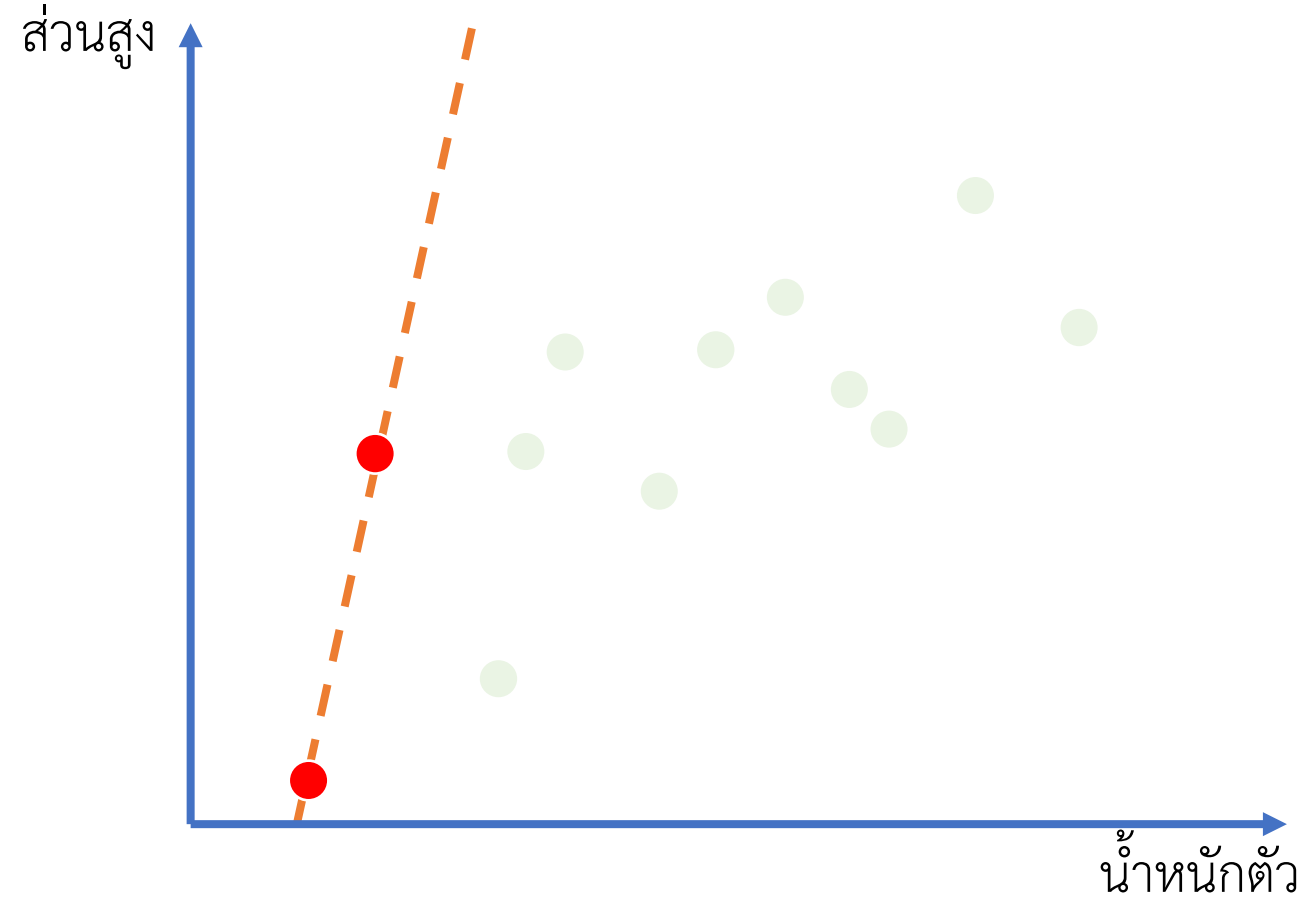
- กลับไปที่ Linear Regression
  - จากกราฟคือ Bias ต่ำมากๆ แต่ Variance สูงมากๆ
  - แบบนี้เรียกได้ว่า Overfit อย่างชัดเจน
  - เกิดขึ้นบ่อยด้วย



# Ridge Regression



- กลับไปที่ Linear Regression
  - เมื่อเราใช้กระบวนการ Least square เมื่อมีข้อมูลดังตัวอย่าง เราจะได้ว่า Residual ของ Training set นี้เป็น 0
  - Ridge Regression ถูกพัฒนาเพื่อแก้ไขปัญหานี้

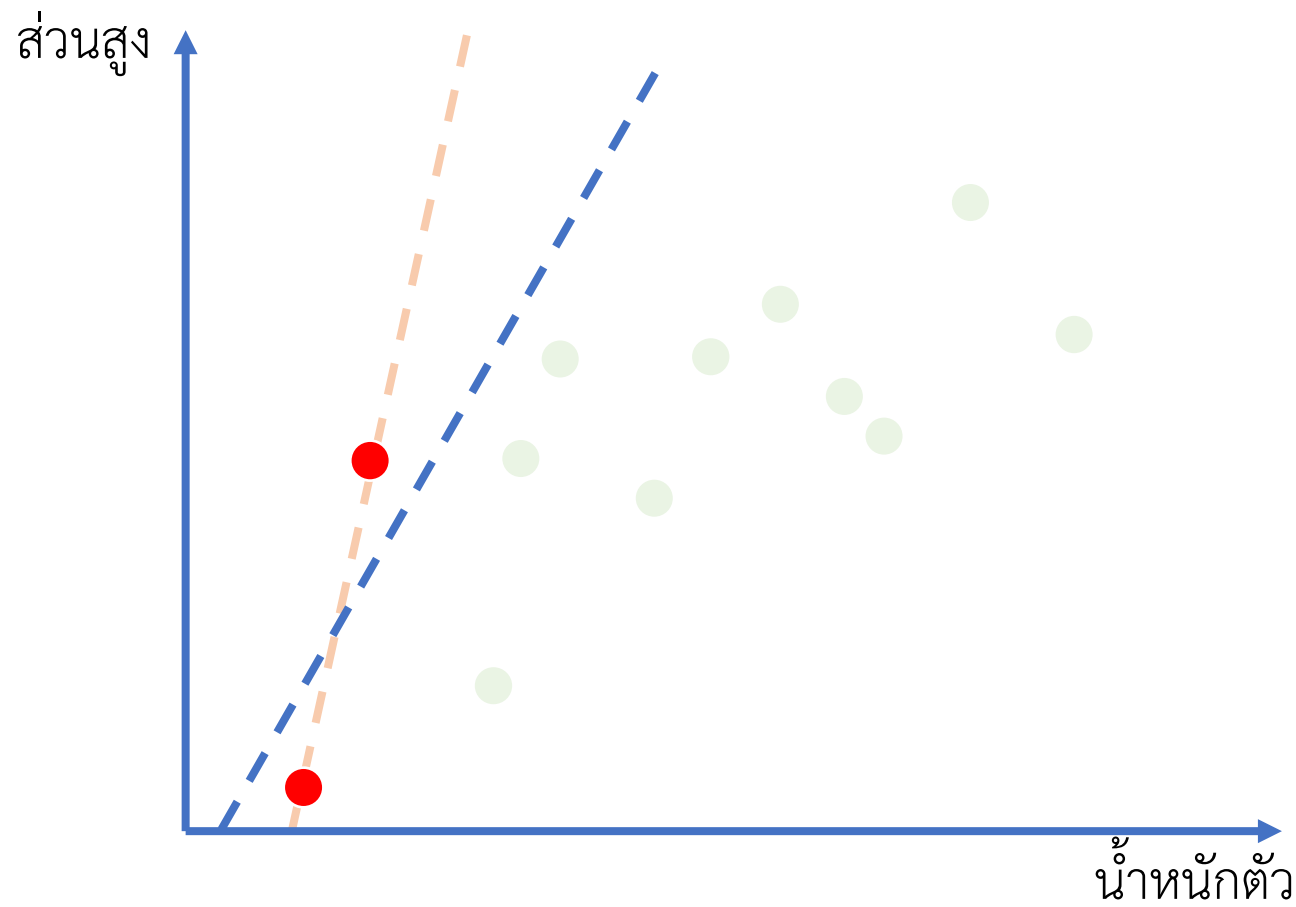


# Ridge Regression



- Ridge Regression

- เป็นวิธีการที่ยอมให้เรามี Residual เพิ่มขึ้นเล็กน้อย (เพิ่ม Bias) เพื่อลด Variance
- ของจริงจะไม่พุ่งทะยานแบบนี้ แต่ตัวอย่างยกมาให้เห็นได้ชัด
- จากตัวอย่างจะเห็นว่ายังไม่พิต แต่ก็มี Variance ลดลงอย่างมีนัยยะ
- ต่อไปจะขุดถึงที่มาและวิธีการลด slope (แน่นอนว่าไม่ได้ลดลงไปด้วยตัวเลขโดดเฉยๆแน่นอน)



# Ridge Regression

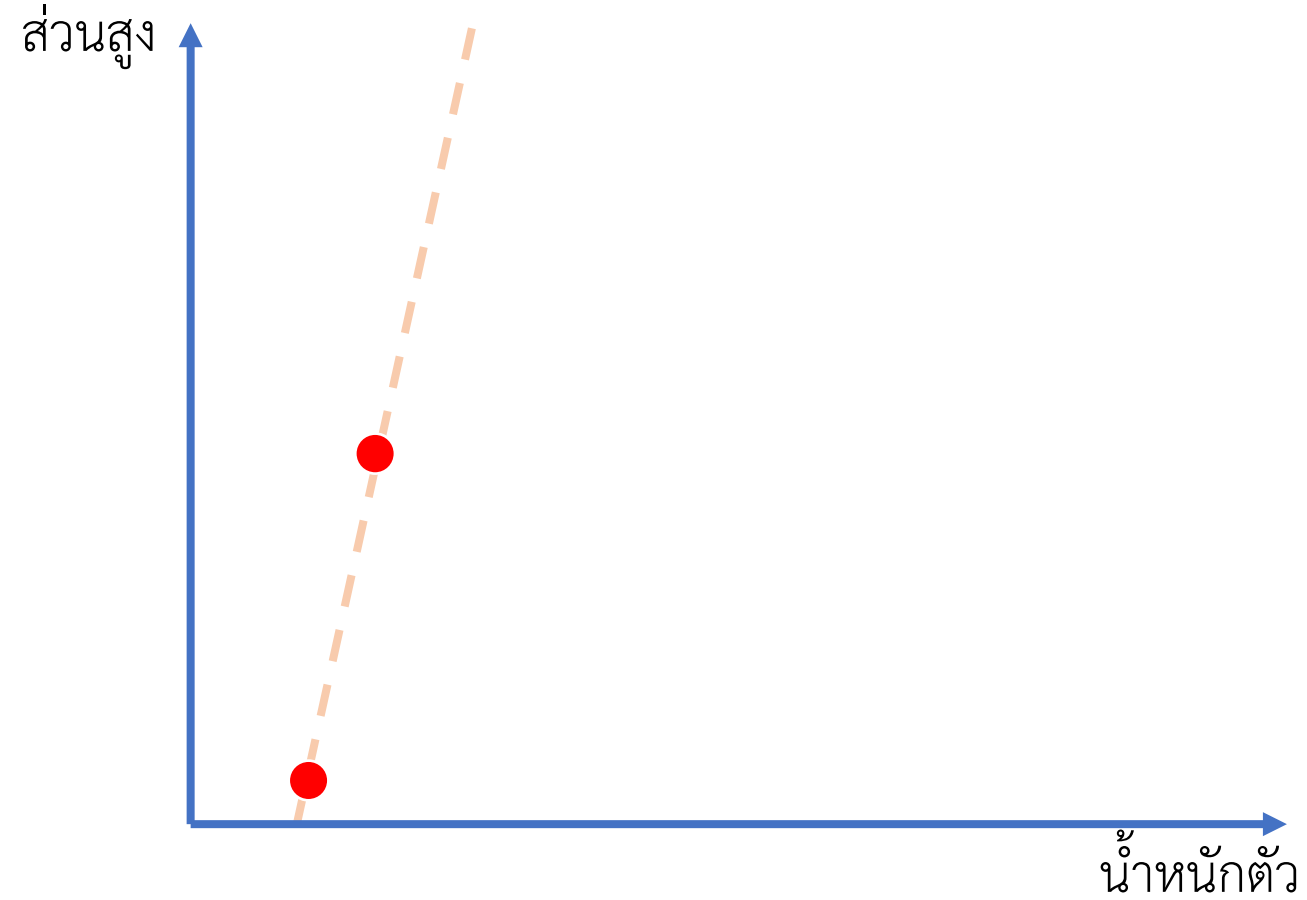


- Ridge Regression
  - การใช้ Least square ใน Linear regression

ส่วนสูง = จุดตัดแกน  $y$  + (slope)(น้ำหนัก)

เราจะลดค่าของ SSR (Sum of Square Residuals) ให้น้อยที่สุด

$\min(SSR)$



# Ridge Regression



- Ridge Regression

- แต่การทำงานของ Ridge regression จะต่างไป

สมการของ Linear Regression (Least squared)

$$\min(SSR)$$

สมการของ Ridge Regression

$$\min(SSR + \lambda(slope^2))$$

↑  
*Penalty term*

# Ridge Regression



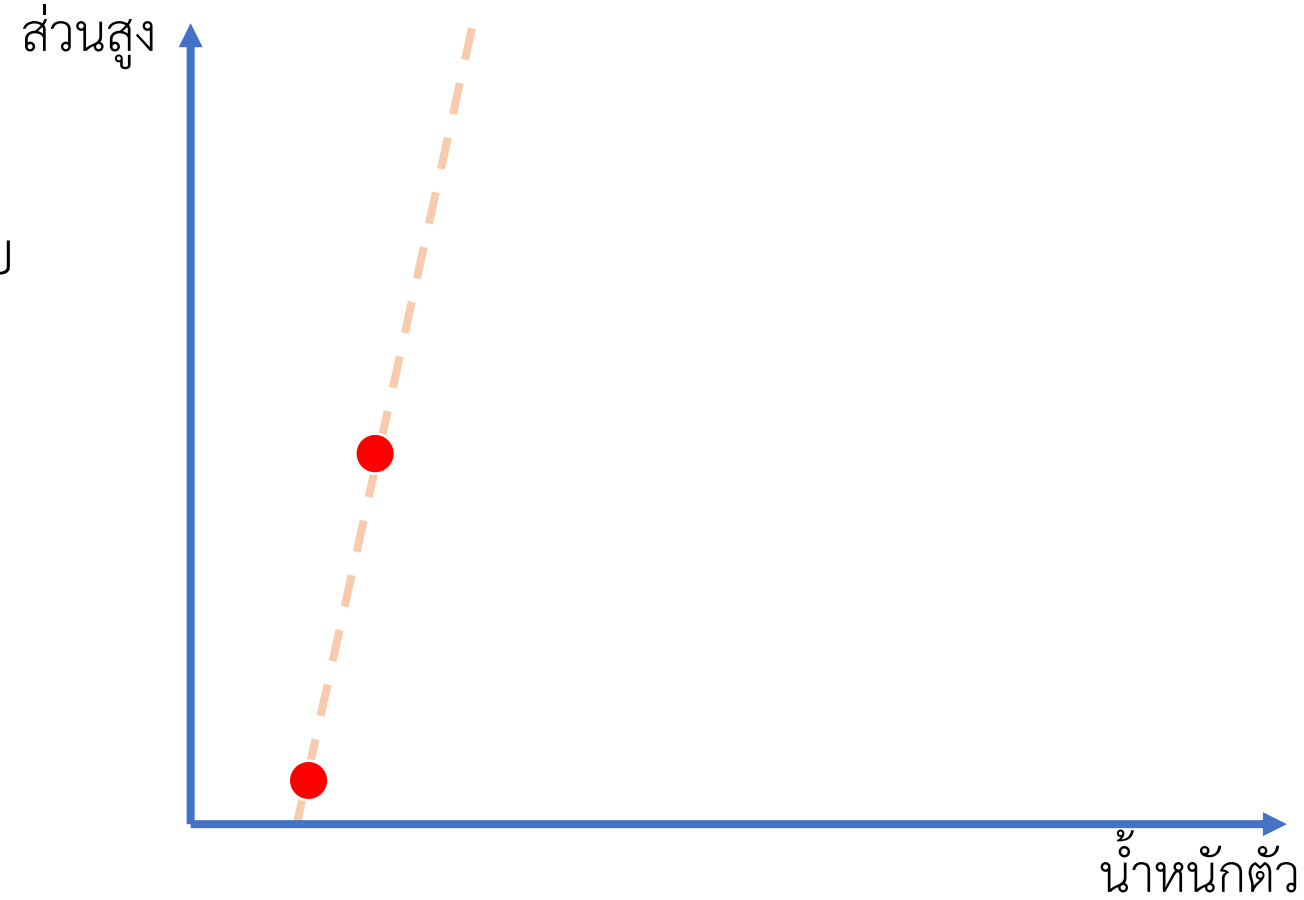
- Ridge Regression

$$\min(SSR + \lambda(slope^2))$$

จากกราฟ SSR เป็น 0 เพราะ slope แนบสนิทกับ  
ข้อมูลทั้งหมด (2 ตัว)

สมมติ  $\lambda = 1$  (เดี๋ยวจะกล่าวถึงค่านี้ภายหลัง)

$$0 + 1(1.3^2) = 1.69$$



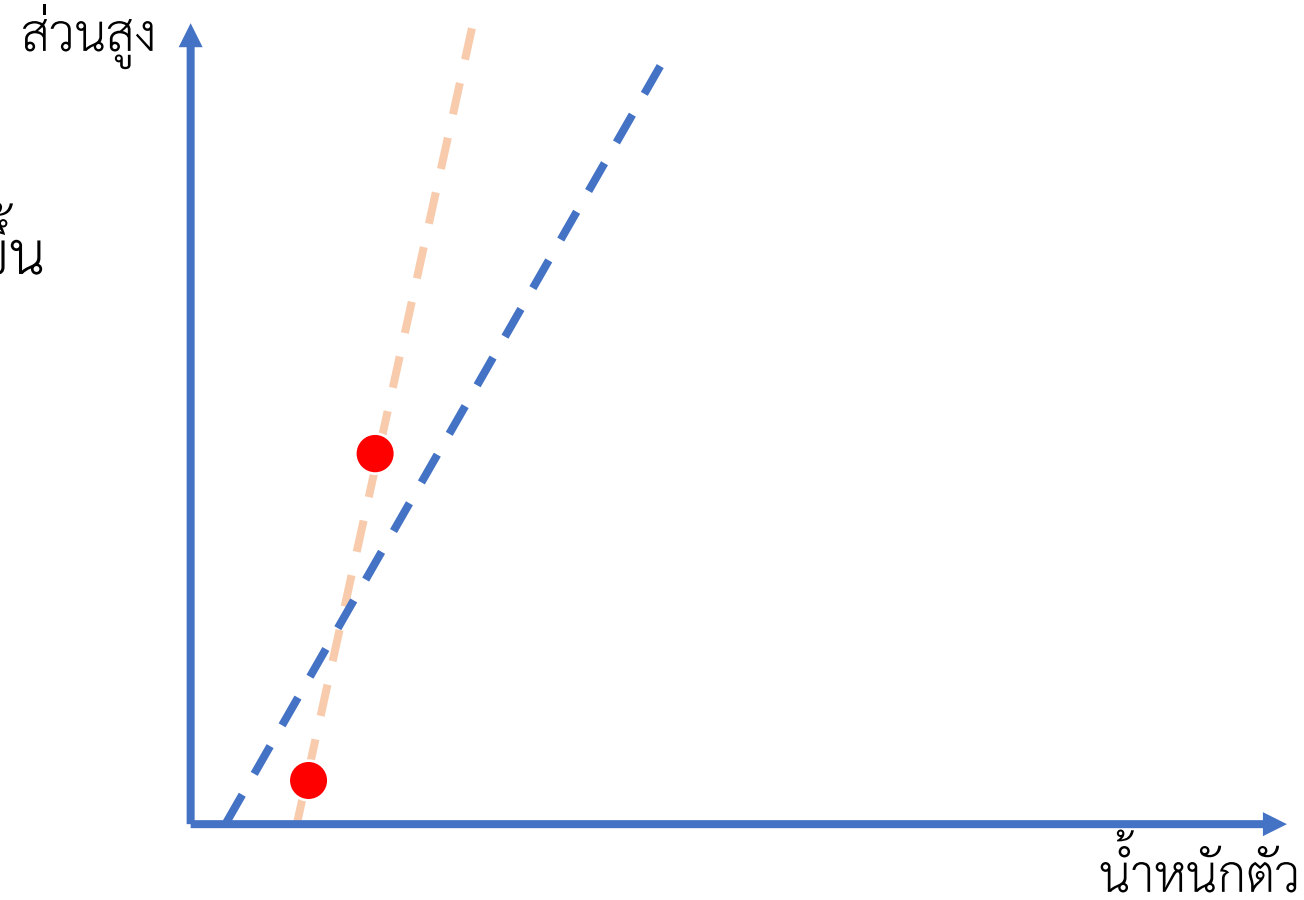
# Ridge Regression

- Ridge Regression

$$\min(SSR + \lambda(slope^2))$$

คราวนี้ลองลด slope ลง โดยยอมให้มี Bias เพิ่มขึ้น เล็กน้อย

$$\begin{aligned} &0.3^2 + 0.1^2 + 1(0.8^2) \\ &= 0.9 + 0.01 + 0.64 \\ &= 0.74 \end{aligned}$$



# Ridge Regression

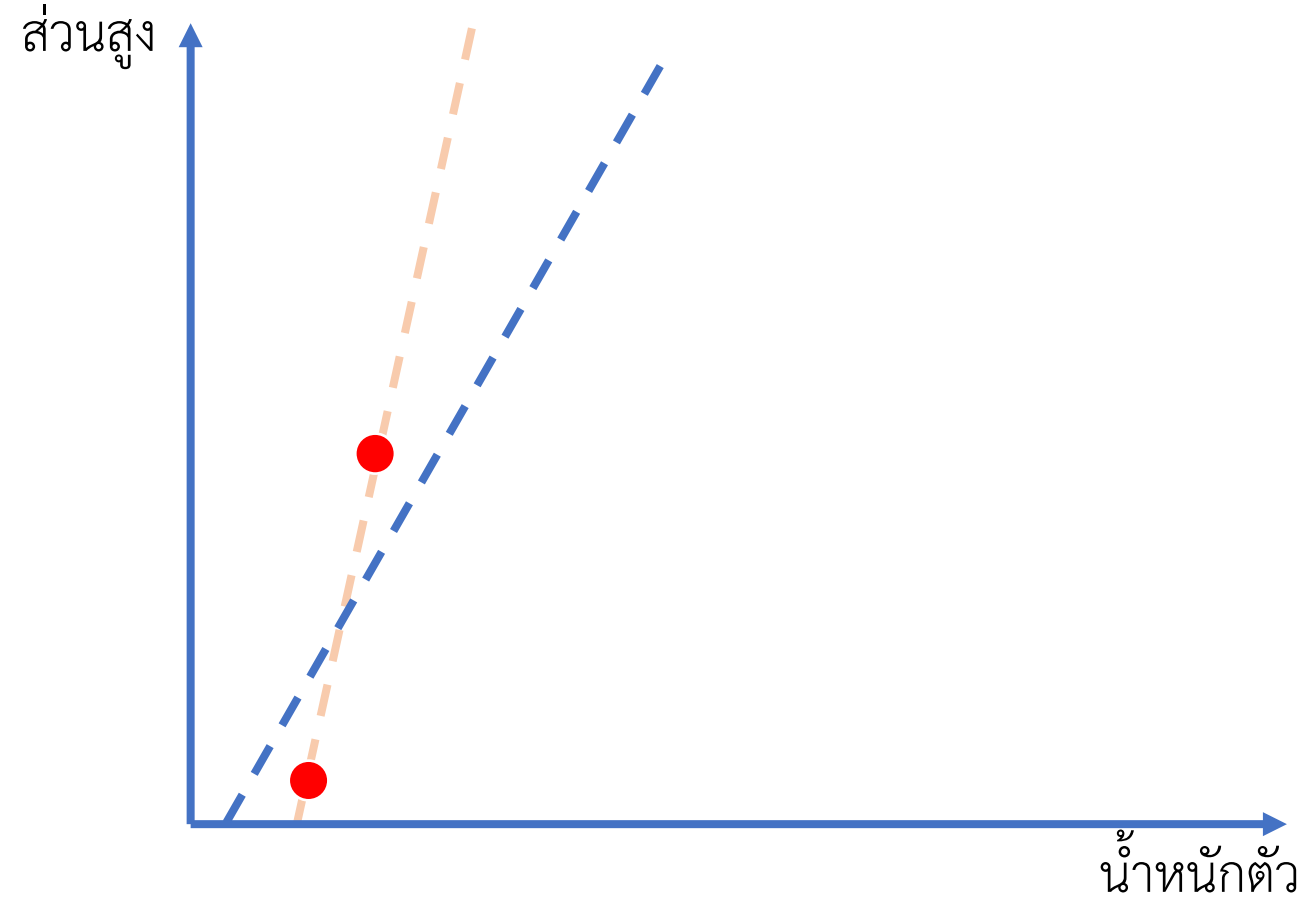
- Ridge Regression

$$\min(SSR + \lambda(slope^2))$$

จะพบว่า เมื่อมี penalty term ทำให้โมเดลยอมสร้าง Bias เล็กน้อย เพื่อให้ความ sensitive ของโมเดลที่สูงเกินไปลดต่ำลงได้

$$0 + 1(1.3^2) = 1.69$$

$$0.3^2 + 0.1^2 + 1(0.8^2) = 0.74$$



# Ridge Regression: ค่าของ $\lambda$

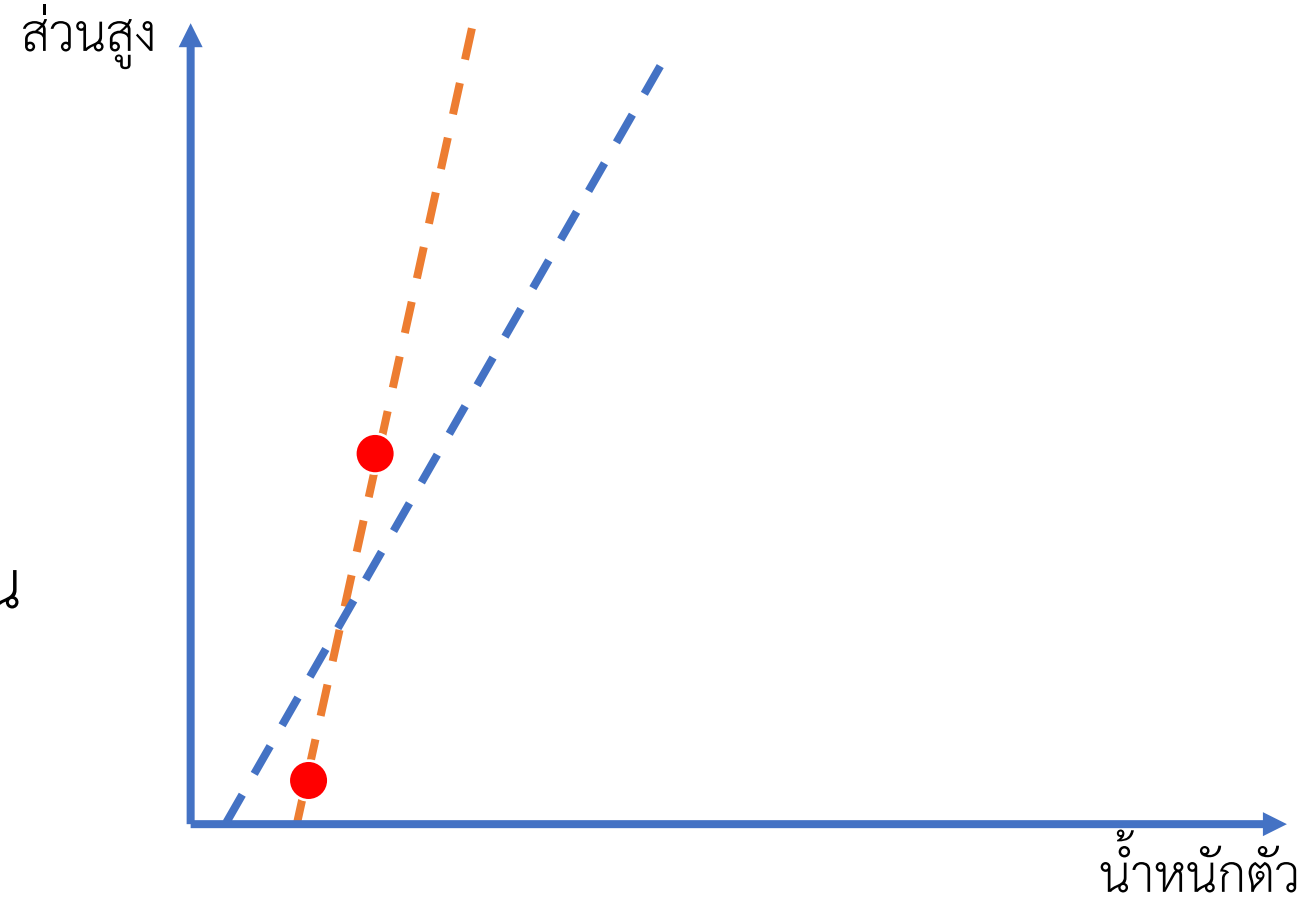


- Ridge Regression

$\lambda$  จะเป็นค่าอะไรก็ได้ตั้งแต่ 0 ไปจนถึง  $+\infty$

ทดลองเปลี่ยนค่า  $\lambda$

ถ้า  $\lambda$  มีค่าเป็น 1 การทำงานของ penalty term จะส่งผลให้ slope มีค่าน้อยลงจาก Linear Regression อย่างที่ได้ลองคิดไปก่อนหน้านี้



# Ridge Regression: ค่าของ $\lambda$

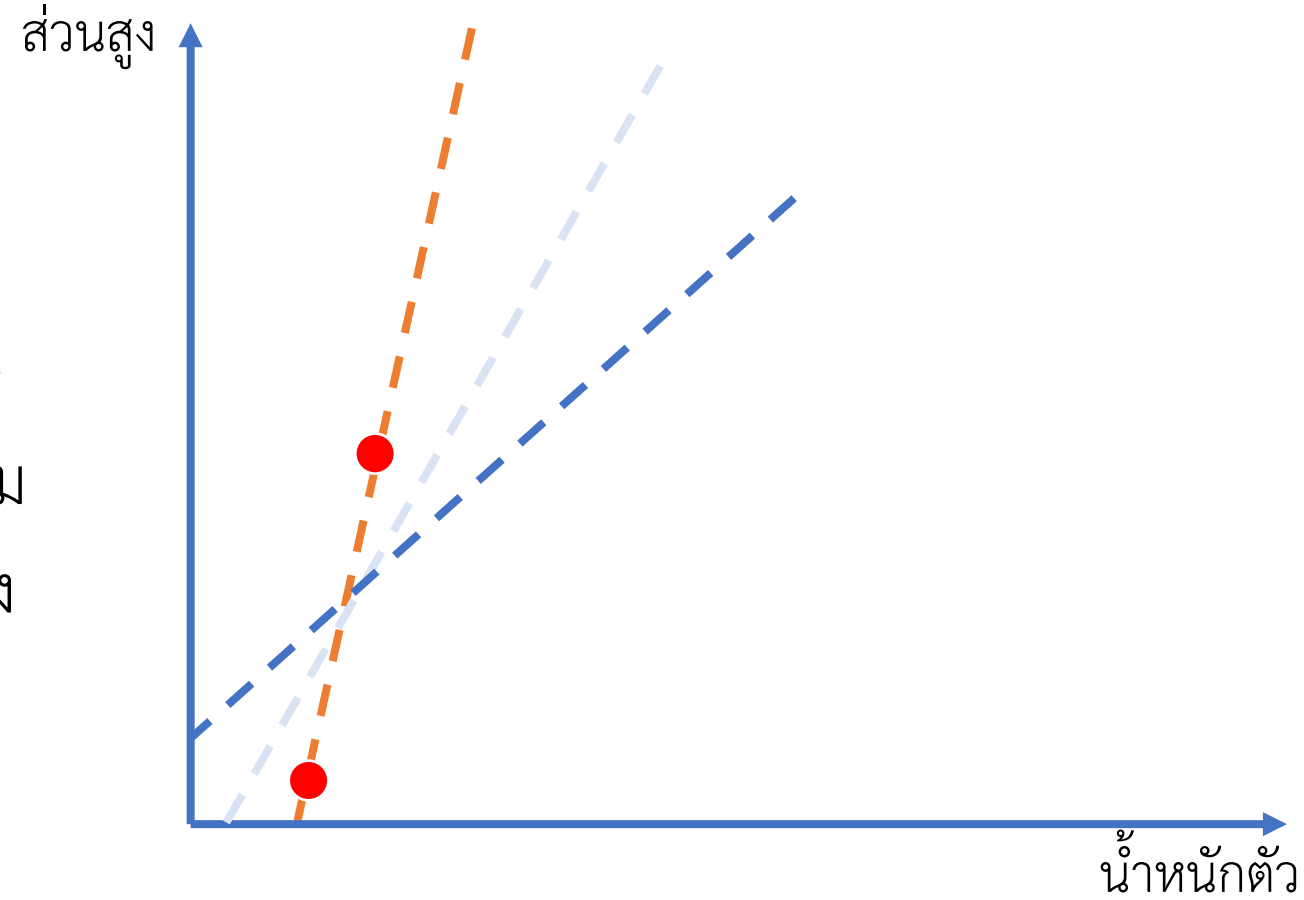


- Ridge Regression

$\lambda$  จะเป็นค่าอะไรก็ได้ตั้งแต่ 0 ไปจนถึง  $+\infty$

ทดลองเปลี่ยนค่า  $\lambda$

ถ้า  $\lambda$  มีค่าเป็น 10 การทำงานของ penalty term จะส่งผลให้ slope มีค่าน้อยลงจากเดิมอีก นั่นหมายความว่า ยิ่ง  $\lambda$  มีค่ามาก ก็จะทำให้ลดความ sensitive ของโมเดลมากขึ้นด้วย และ slope ก็ลดลงแปรผันกับค่าของ  $\lambda$



# Ridge Regression: ค่าของ $\lambda$



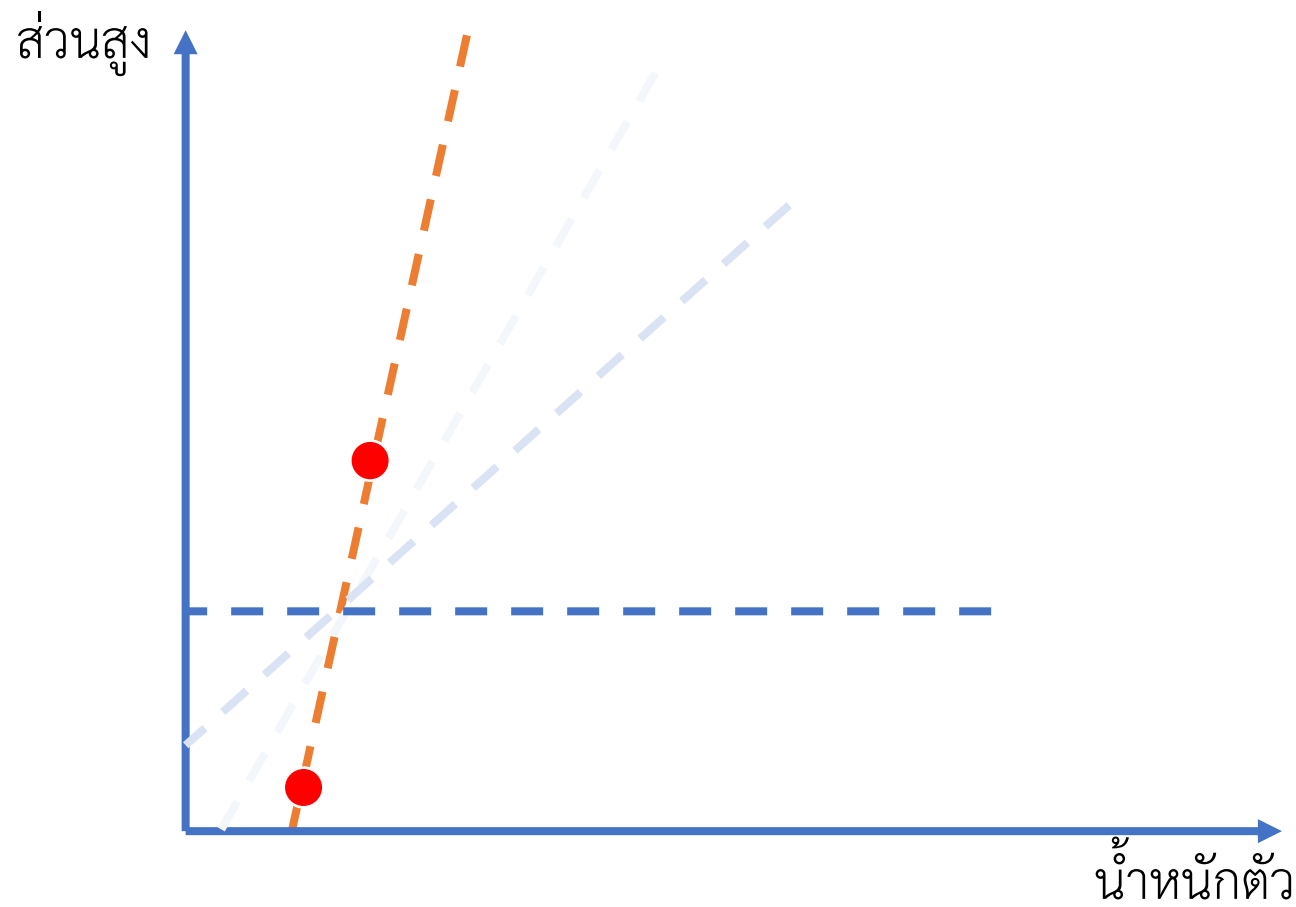
- Ridge Regression

$\lambda$  จะเป็นค่าอะไรก็ได้ตั้งแต่ 0 ไปจนถึง  $+\infty$

ทดลองเปลี่ยนค่า  $\lambda$

ถ้า  $\lambda$  มีค่าเป็น 100000000 การทำงานของ penalty term จะส่งผลให้ slope มีค่าน้อยลงจากเดิมมาก ๆ หมายความว่ายิ่ง  $\lambda$  เข้าใกล้  $+\infty$  ค่า slope ก็ลดลงจนเกือบเป็น 0 และโมเดลก็อาจใช้ไม่ได้อีก

ฉะนั้นห้ามตั้ง  $\lambda$  ใหญ่เกินไป

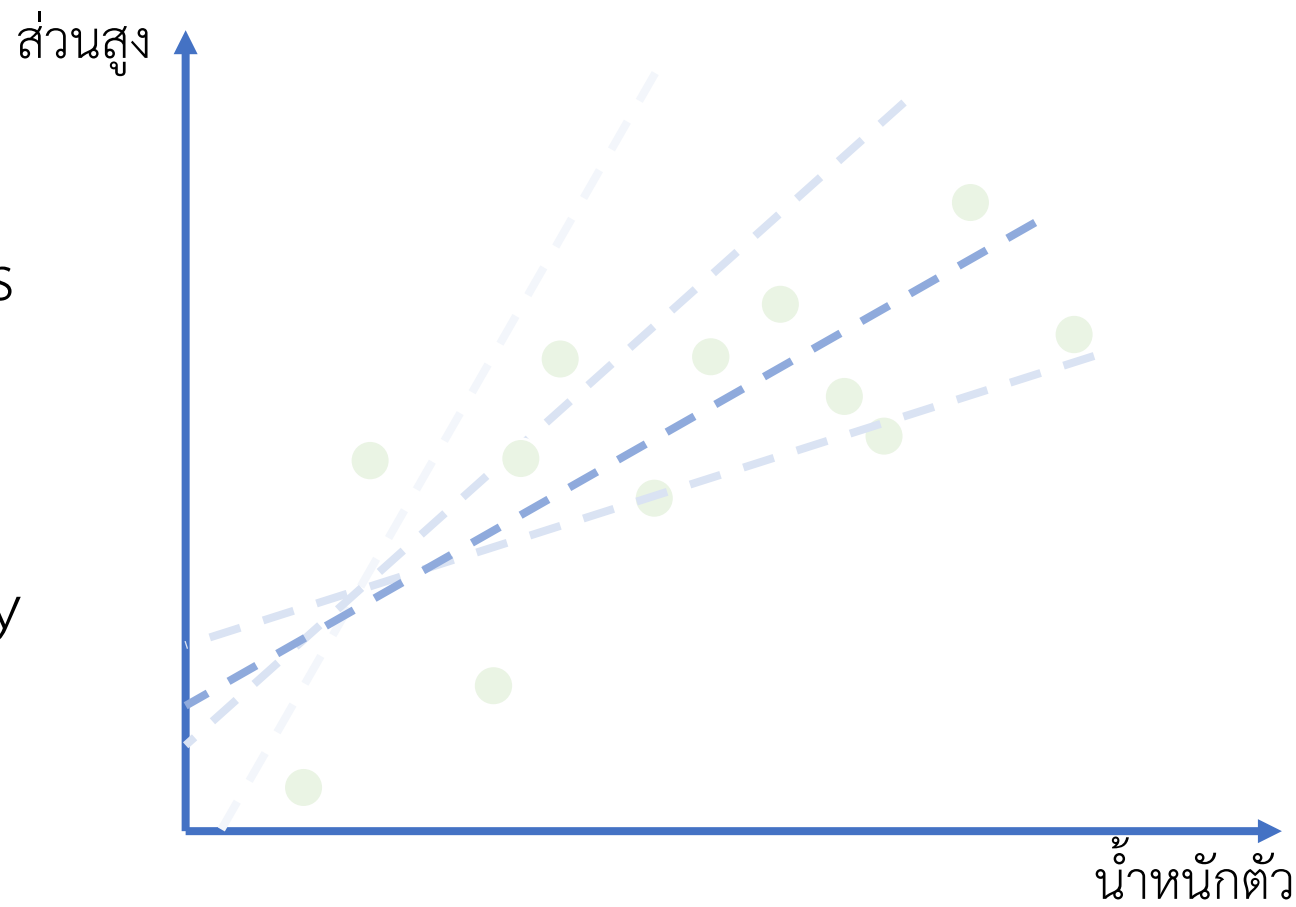


# Ridge Regression: ค่าของ $\lambda$



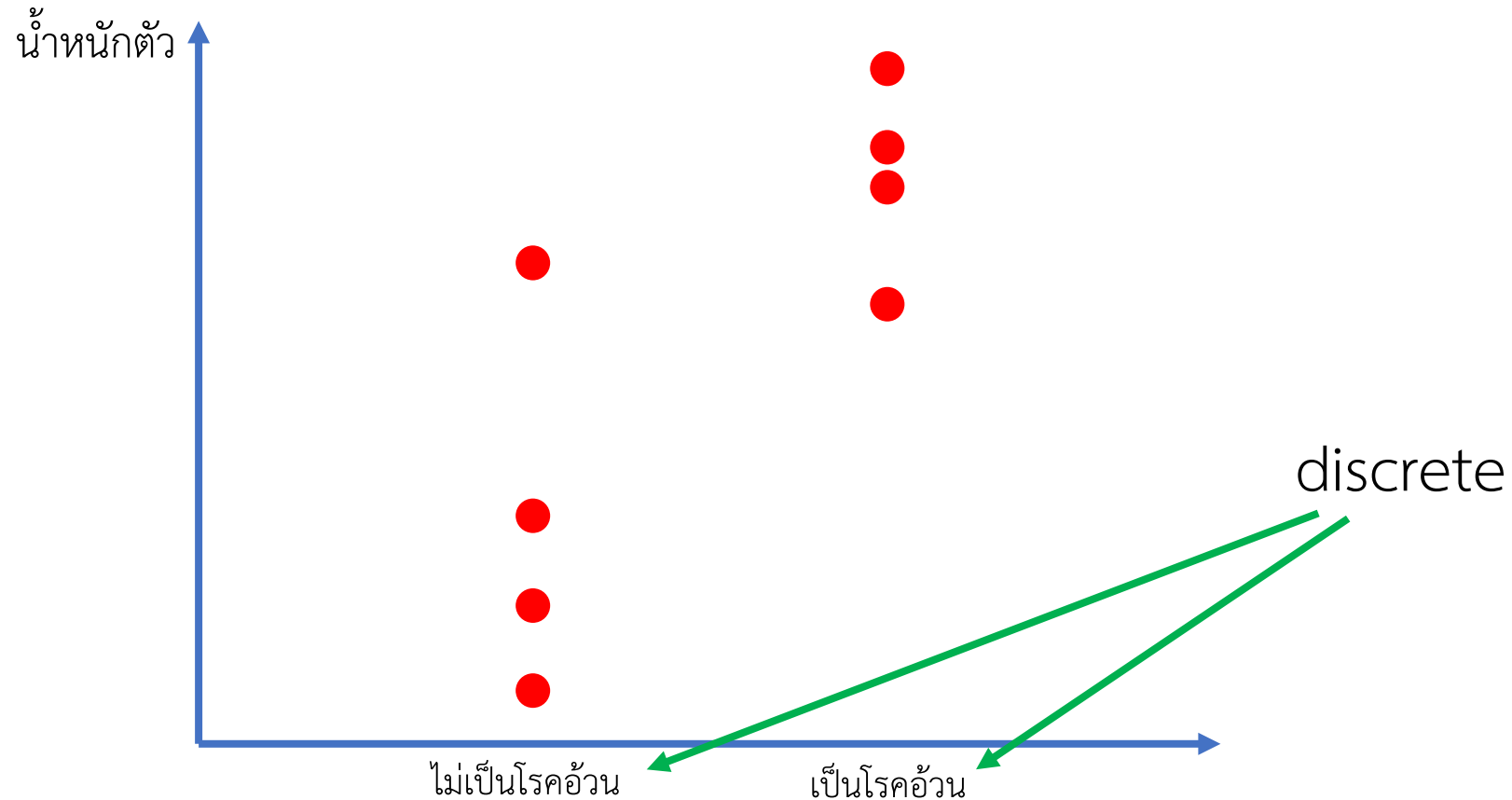
- Ridge Regression

- ปกติแล้ว การปรับค่า  $\lambda$  เพื่อเลือกค่าที่เหมาะสมที่สุด จะใช้วิธีการ 10-fold cross validation เพื่อให้ข้อมูลสลับกันเป็น Training set และ Test set
- เลือกค่า  $\lambda$  ที่ทำให้ค่าเฉลี่ยของ Accuracy จาก 10-fold cross validation ดีที่สุด
- การทำแบบนี้ทำให้เราสามารถสังเกต bias และ variance คร่าวๆ จากข้อมูลที่มีได้



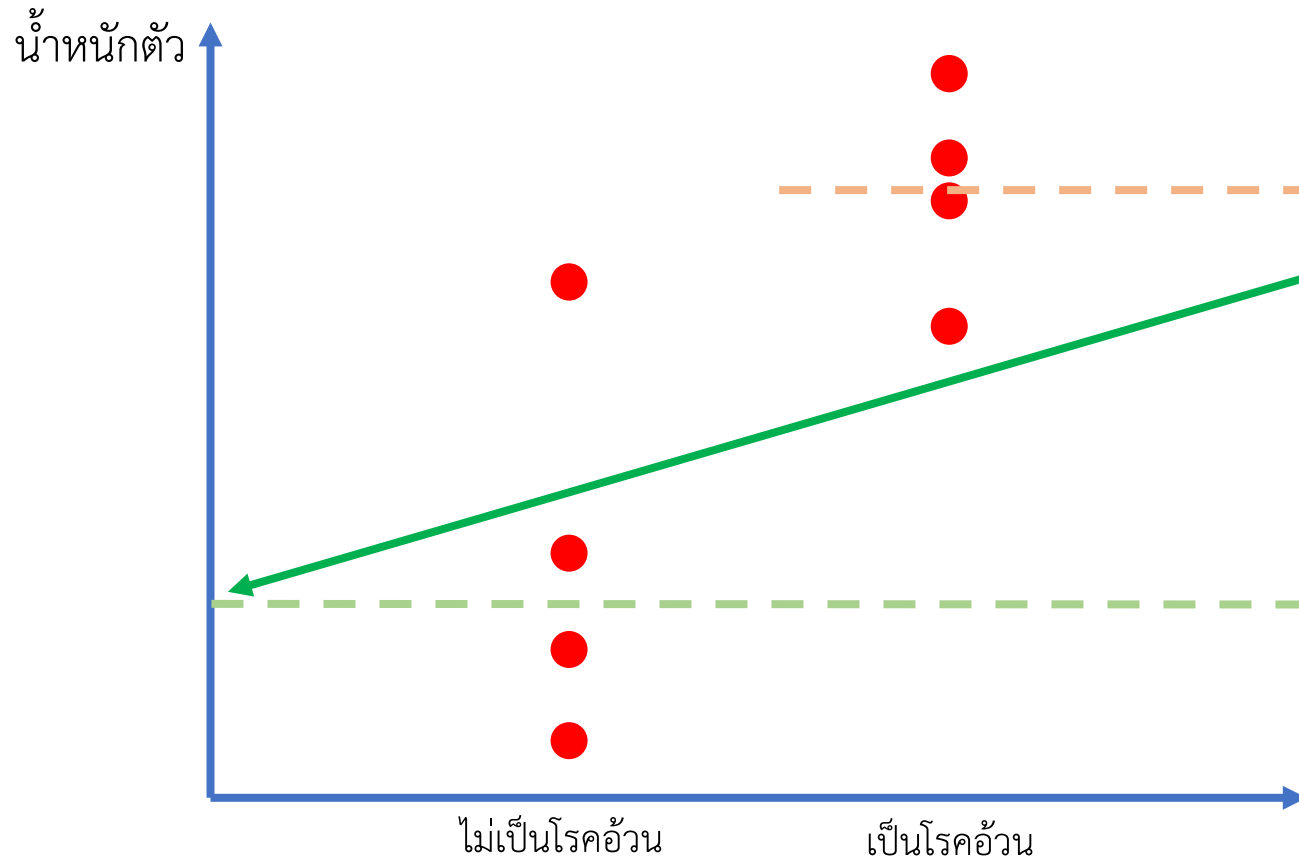
# Ridge Regression: discrete data

- Ridge Regression ยังสามารถใช้ในข้อมูลที่เป็น discrete ได้ด้วย



# Ridge Regression: discrete data

- ในกรณีตัวอย่างนี้ เราจะพยากรณ์น้ำหนักตัว โดยใช้การเป็นโรคอ้วนเป็นข้อมูลหลัก



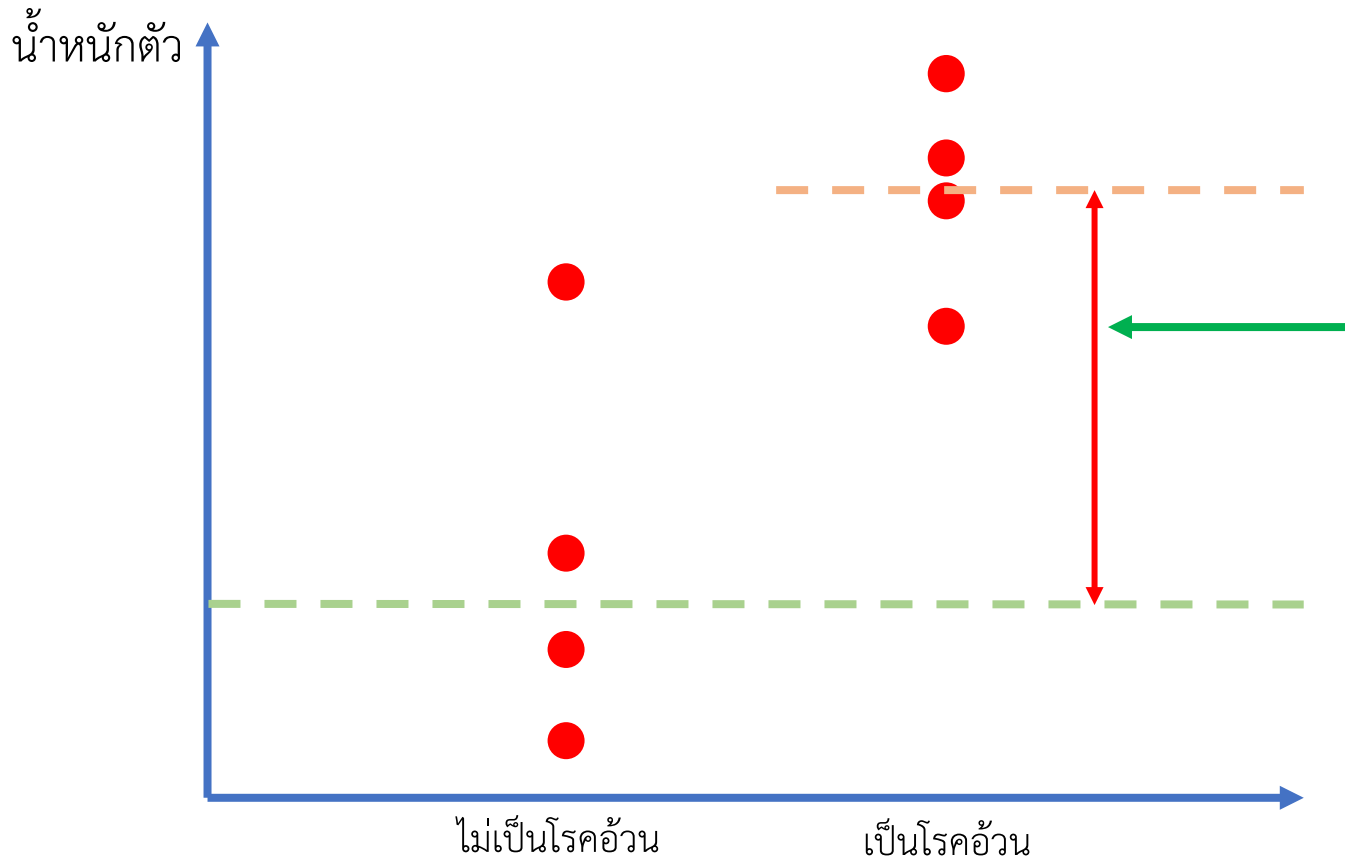
ปล่อยให้ Least squared ทำงาน (เหมือน Linear Regression)  
น้ำหนัก = จุดตัดแกน y + (slope)(โรคอ้วน?)

โดยที่

- จุดตัดแกน y จะแทนค่าเฉลี่ยของน้ำหนักของคนที่ไม่เป็นโรคอ้วน

# Ridge Regression: discrete data

- Ridge Regression ยังสามารถใช้ในข้อมูลที่เป็น discrete ได้ด้วย



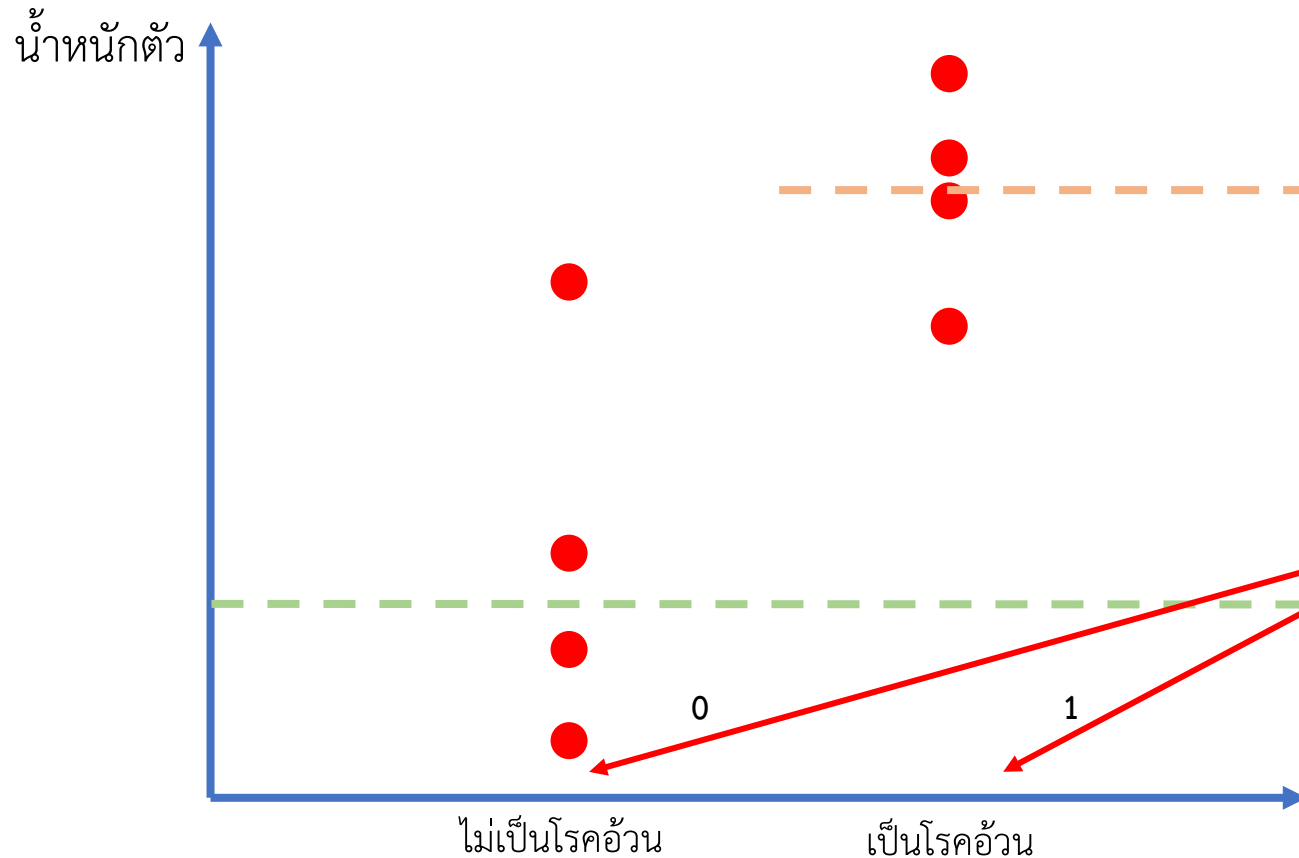
ปล่อยให้ Least squared ทำงาน (เหมือน Linear Regression)  
น้ำหนัก = จุดตัดแกน y + (slope)(โรคอ้วน?)

โดยที่

- จุดตัดแกน y จะแทนค่าเฉลี่ยของน้ำหนักของคนที่ไม่เป็นโรคอ้วน
- Slope คือ ระยะห่างระหว่างค่าเฉลี่ยของน้ำหนักของคนที่เป็นโรคอ้วนและไม่เป็นโรคอ้วน

# Ridge Regression: discrete data

- Ridge Regression ยังสามารถใช้ในข้อมูลที่เป็น discrete ได้ด้วย



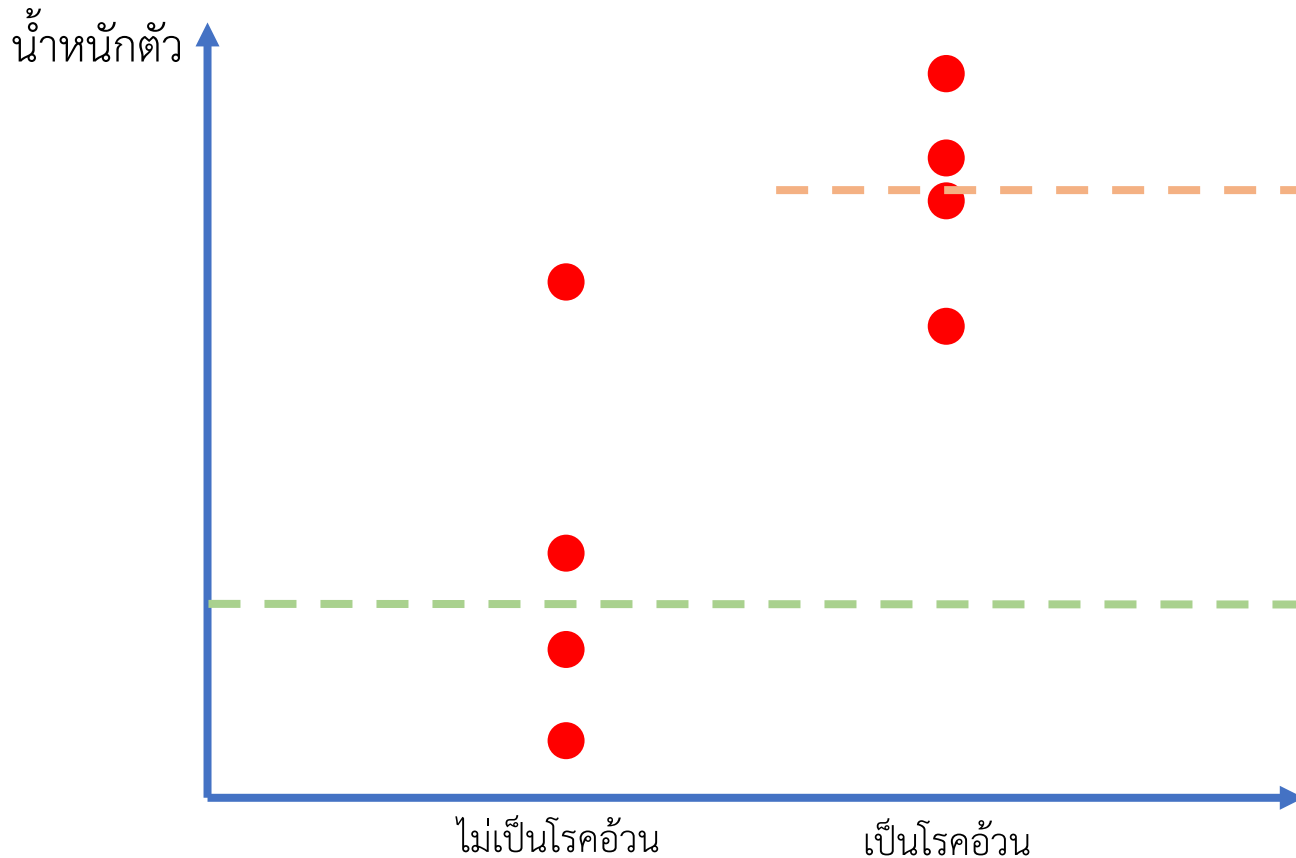
ปล่อยให้ Least squared ทำงาน (เหมือน Linear Regression)  
น้ำหนักรั่ว = จุดตัดแกน  $y$  + (slope)(โรคอ้วน?)

โดยที่

- จุดตัดแกน  $y$  จะแทนค่าเฉลี่ยของน้ำหนักรั่วของคนที่ไม่เป็นโรคอ้วน
- Slope คือ ระยะห่างระหว่างค่าเฉลี่ยของน้ำหนักรั่วของคนที่เป็นโรคอ้วนและไม่เป็นโรคอ้วน
- โรคอ้วน? แทน 0 หรือ 1 เมื่อ ตัวอย่างตัวนั้น ไม่เป็นโรคอ้วน หรือเป็นโรคอ้วน ตามลำดับ

# Ridge Regression: discrete data

- Ridge Regression ยังสามารถใช้ในข้อมูลที่เป็น discrete ได้ด้วย



ปล่อยให้ Least squared ทำงาน (เหมือน Linear Regression)

น้ำหนัก = จุดตัดแกน y + (slope)(โรคอ้วน?)

ดังนั้น สมการนี้จะต้องการค่า 2 ค่า ได้แก่จุดตัดแกน y และ slope  
ซึ่งการหา 2 ค่านี้ จะได้จาก

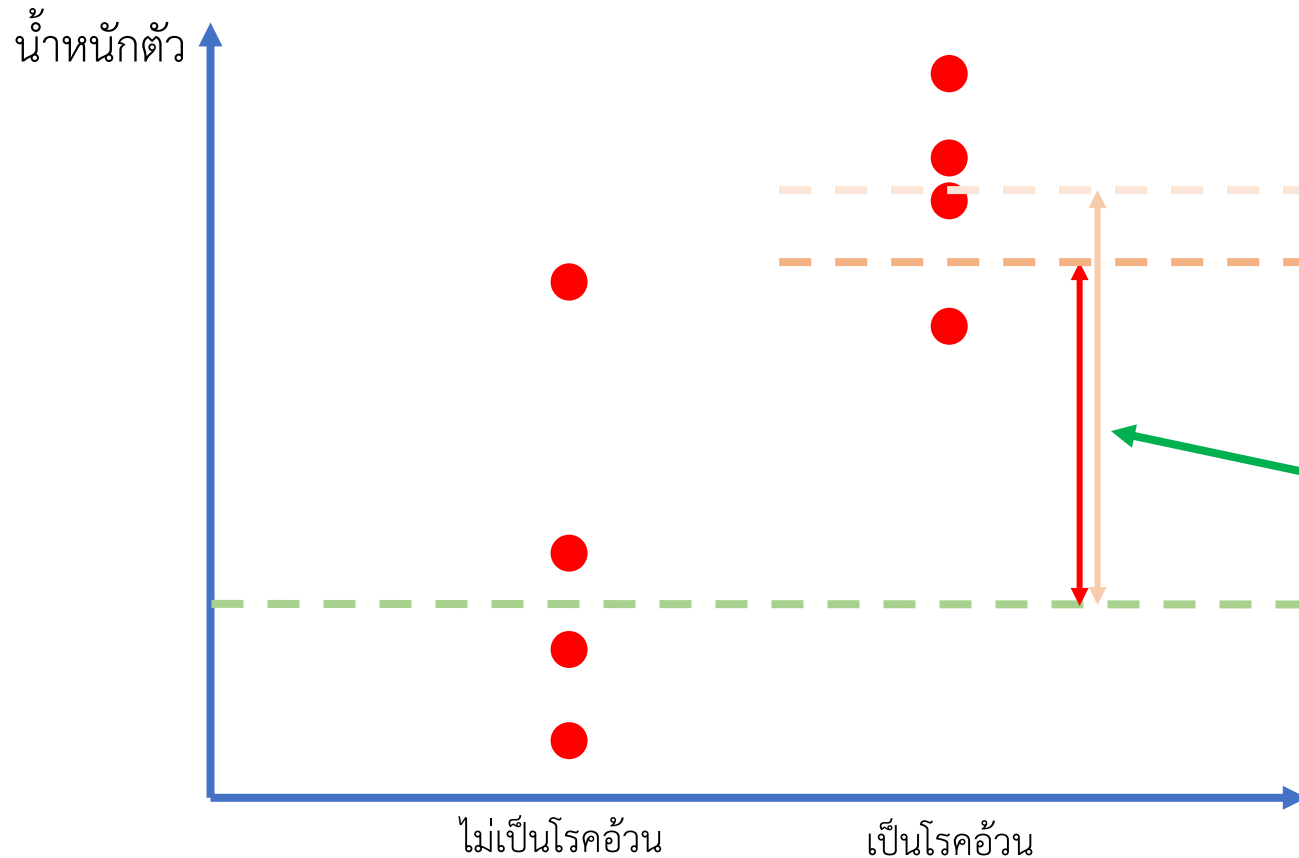
$$\min(SSR)$$

และแน่นอนว่า เมื่อเป็น Ridge Regression เราจะพยายามลด  
ความ sensitive ของโมเดลลง โดยการเพิ่ม penalty term

$$\min(SSR + \lambda(slope^2))$$

# Ridge Regression: discrete data

- Ridge Regression ยังสามารถใช้ในข้อมูลที่เป็น discrete ได้ด้วย

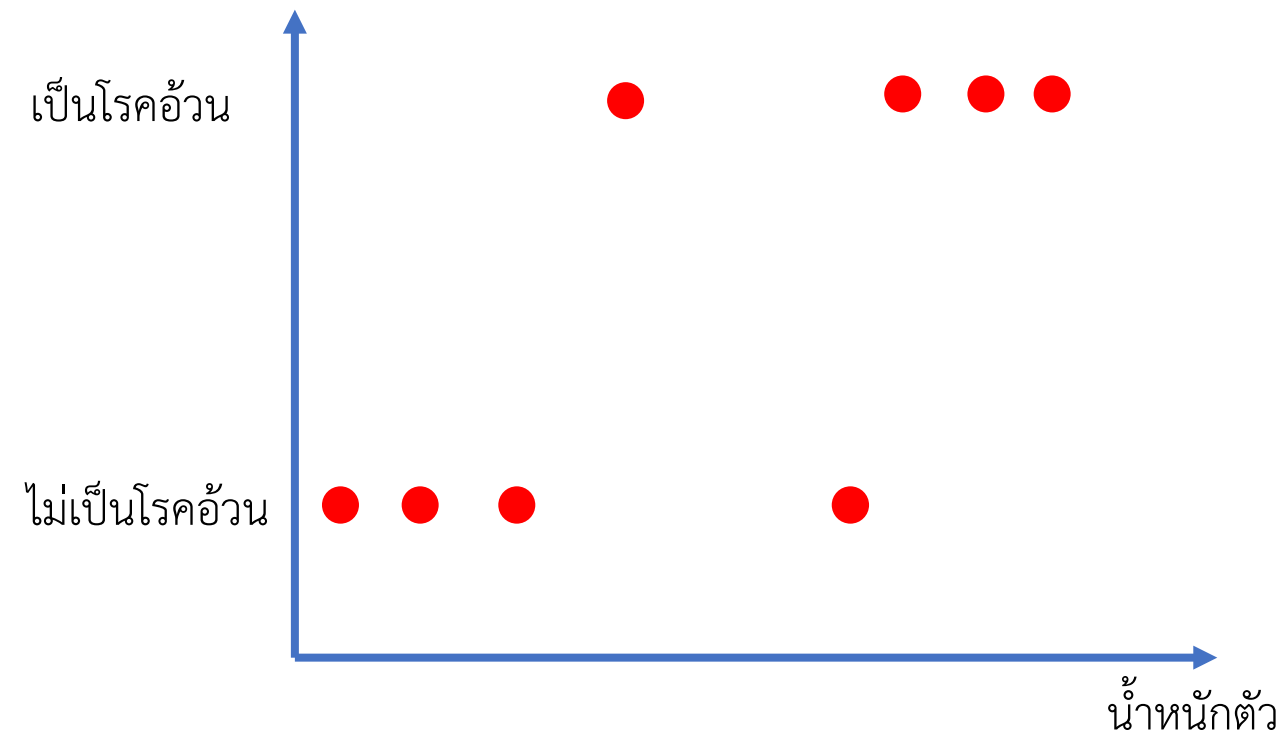


$$\min(SSR + \lambda(slope^2))$$

- จากสมการ เมื่อ  $\lambda$  เป็น 0 ก็จะไม่มีการเปลี่ยนแปลงจากเดิม
- แต่เมื่อเราเพิ่มค่า  $\lambda$  มากขึ้น ทางเดียวที่จะลดค่า  $\min(SSR + \lambda(slope^2))$  ลงได้ คือ...  
...การลดระยะห่างนี้ลง
- ยิ่งค่า  $\lambda$  มาก ทั้งสองเส้นนี้ก็จะชิดกันมากขึ้น

# Ridge Regression: Logistic Regression

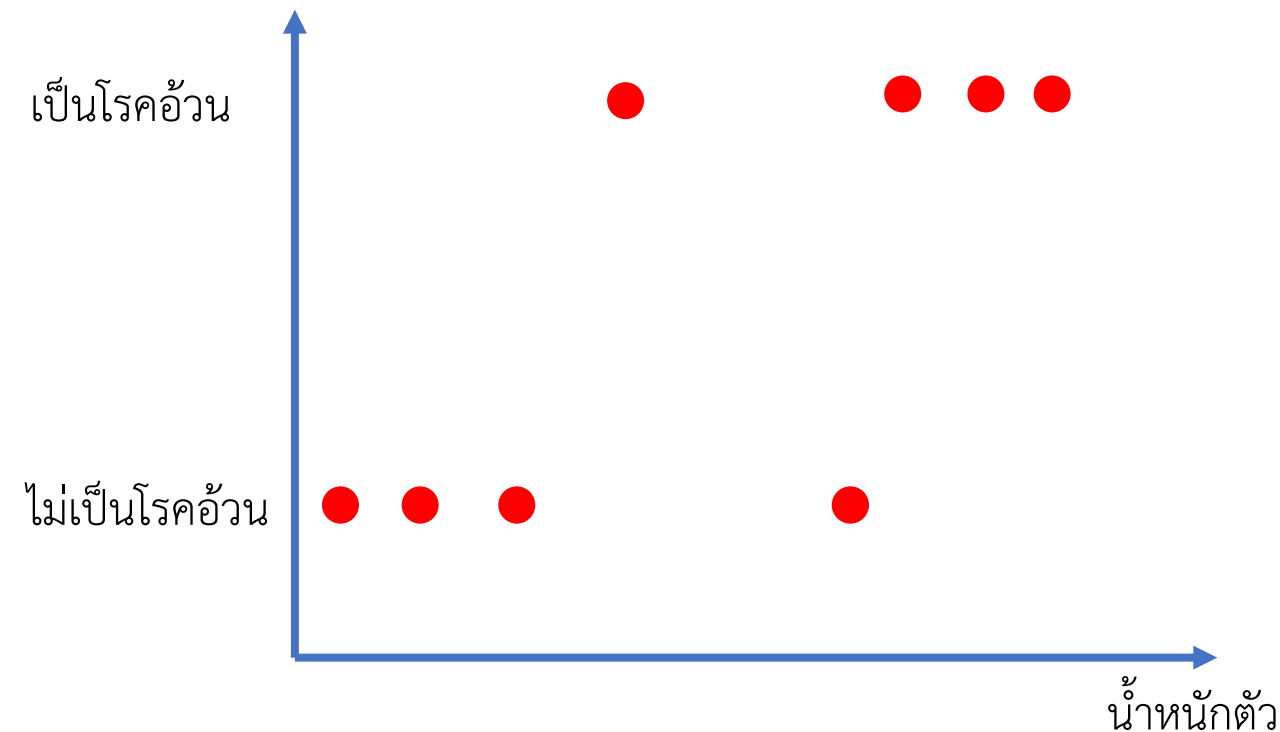
- Ridge Regression ยังสามารถประยุกต์ใช้กับ Logistic Regression ได้ด้วย



ตัวอย่างนี้จะตรงข้ามกับตัวอย่างที่แล้ว คือ ใช้  
น้ำหนักรักตัววิเคราะห์โอกาสการเป็นโรคอ้วน  
(ก่อนหน้านี้เป็น regression ใช้การเป็นโรคอ้วน ประมาณน้ำหนักรักตัว)

# Ridge Regression: Logistic Regression

- Ridge Regression ยังสามารถประยุกต์ใช้กับ Logistic Regression ได้ด้วย

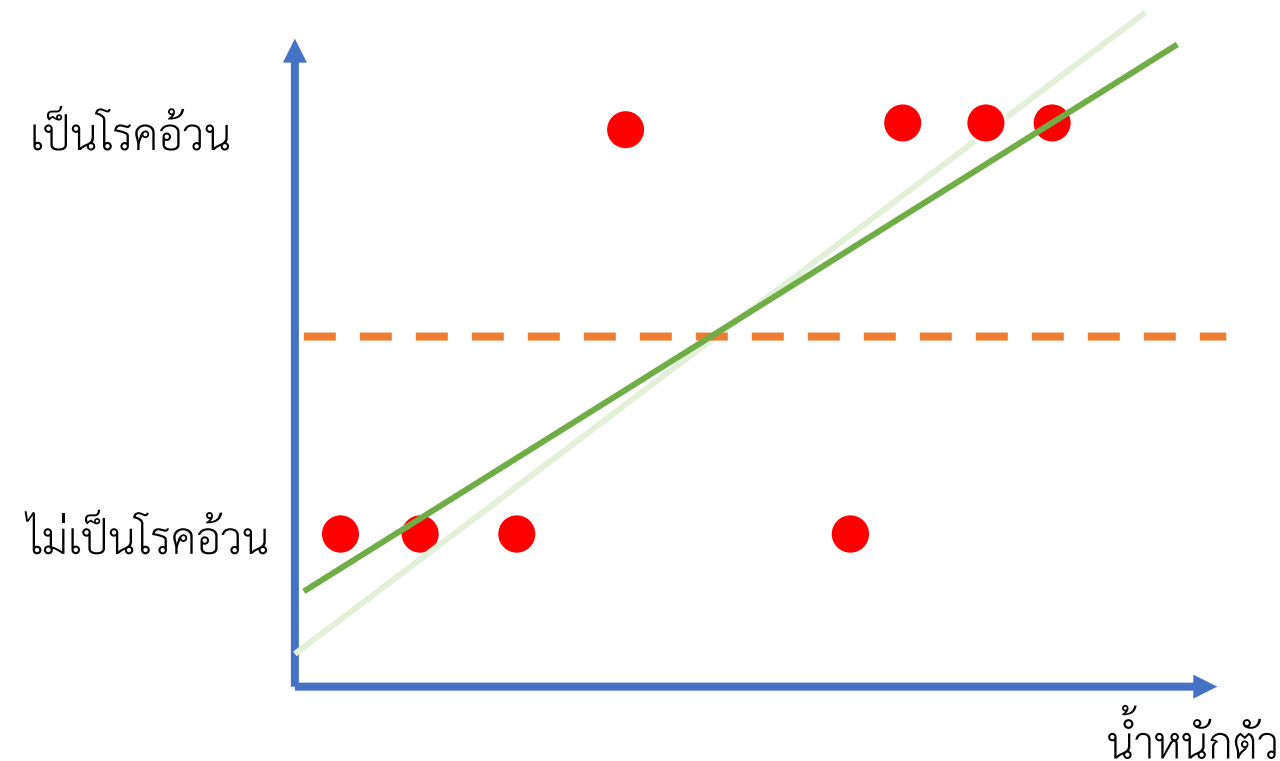


- เรารู้อยู่แล้วว่า Logistic Regression ใช้การคำนวณ sum of likelihoods ในการฟิตข้อมูล
- ไม่เคยกล่าวถึงสมการ แต่คอนเซปคือการคำนวณ likelihood ของทุกจุดมา **คูณ** กัน และเลือกเส้นที่ทำให้ค่า likelihoods มีค่ามากที่สุด
- ดังนั้น เมื่อมารวมกับ Ridge Regression ก็จะได้ว่า

$$\text{total likelihoods} + \lambda(\text{slope}^2)$$

# Ridge Regression: Logistic Regression

- Ridge Regression ยังประยุกต์ใช้ป็นงาน Classification ได้เหมือนกัน



- ถ้าใช้วิธีวาดกราฟด้วยเส้นตรง Residual ก็จะดูสูงกว่าแบบ logistic มาก แต่ก็ใช้ได้เหมือนกัน
- ผลลัพธ์จะไม่ใช่ความน่าจะเป็นเหมือน logistic แต่จะบอกได้แค่ว่าเป็นโรคอ้วนหรือไม่เป็นเท่านั้น
- ถ้าค่าที่ Predict ได้ มีค่ามากกว่า Threshold ที่กำหนดไว้ (เส้นประสีส้ม โดยมากจะอยู่ตรงกลางระหว่าง 2 คลาส เช่น ถ้ากำหนดคลาส 0 และ 1 เส้นประสีส้มจะอยู่ที่ 0.5) จะถือว่าเป็นคลาส 1 แต่ถ้ามีค่าน้อยกว่า Threshold ที่กำหนดไว้ จะถือว่าเป็นคลาส 0
- มักใช้ในงานที่มีไม่เกิน 2 คลาส

# Ridge Regression: กับข้อมูลที่ซับซ้อนขึ้น

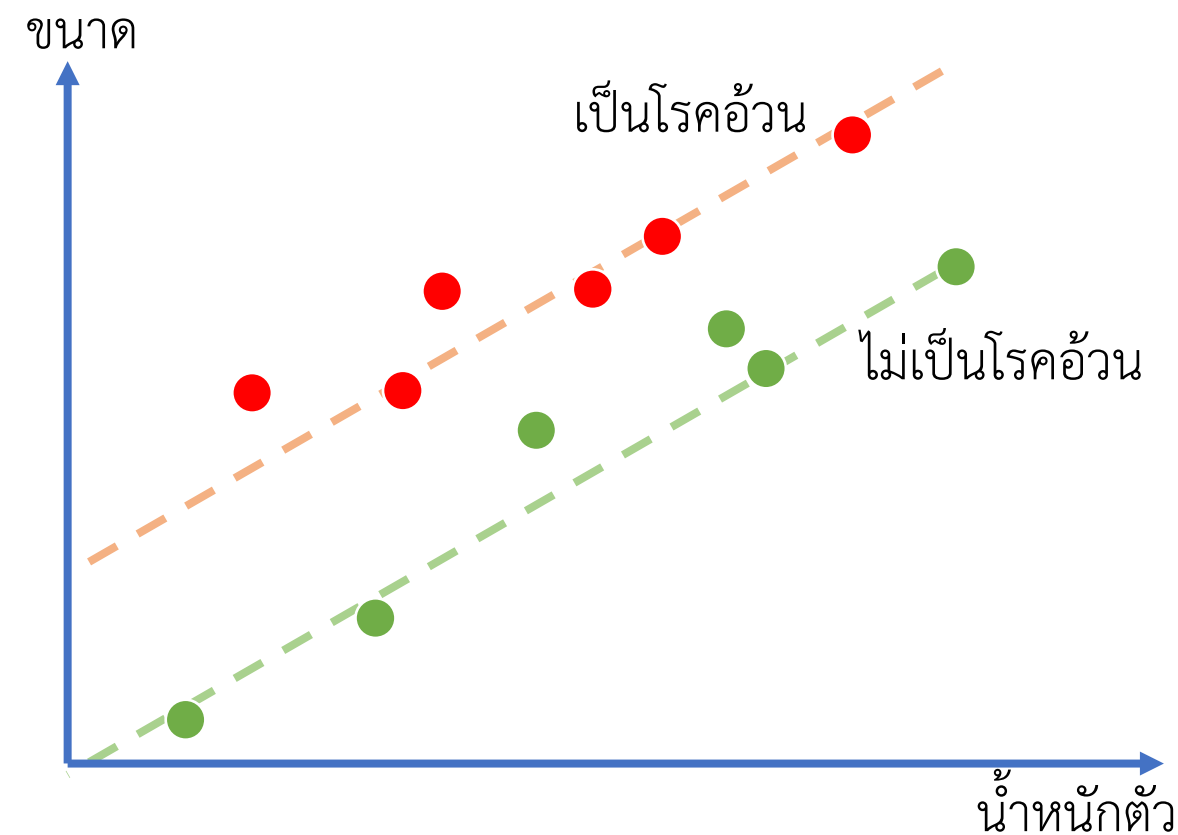


- แต่ข้อมูลจริงๆ มักไม่ได้สวยงาม
  - ตอนนี้เรามีข้อมูลที่มีมิติมากขึ้น มีการแยกการเป็นและไม่เป็นโรคอ้วน
  - เขียนสมการได้เป็น
$$size = y_{intercept} + (slope)(w) + (diff)(fat?)$$

และเราพยายามหาค่า  $y_{intercept}$ ,  $slope$ ,  $diff$  โดย

$$\min(ssr + \lambda(slope^2 + diff^2))$$

\*ถ้ามีฟีเจอร์มากกว่านี้ จะใช้ ridge regression ช่วยได้หรือไม่?



# Ridge Regression: กับข้อมูลที่ซับซ้อนขึ้น

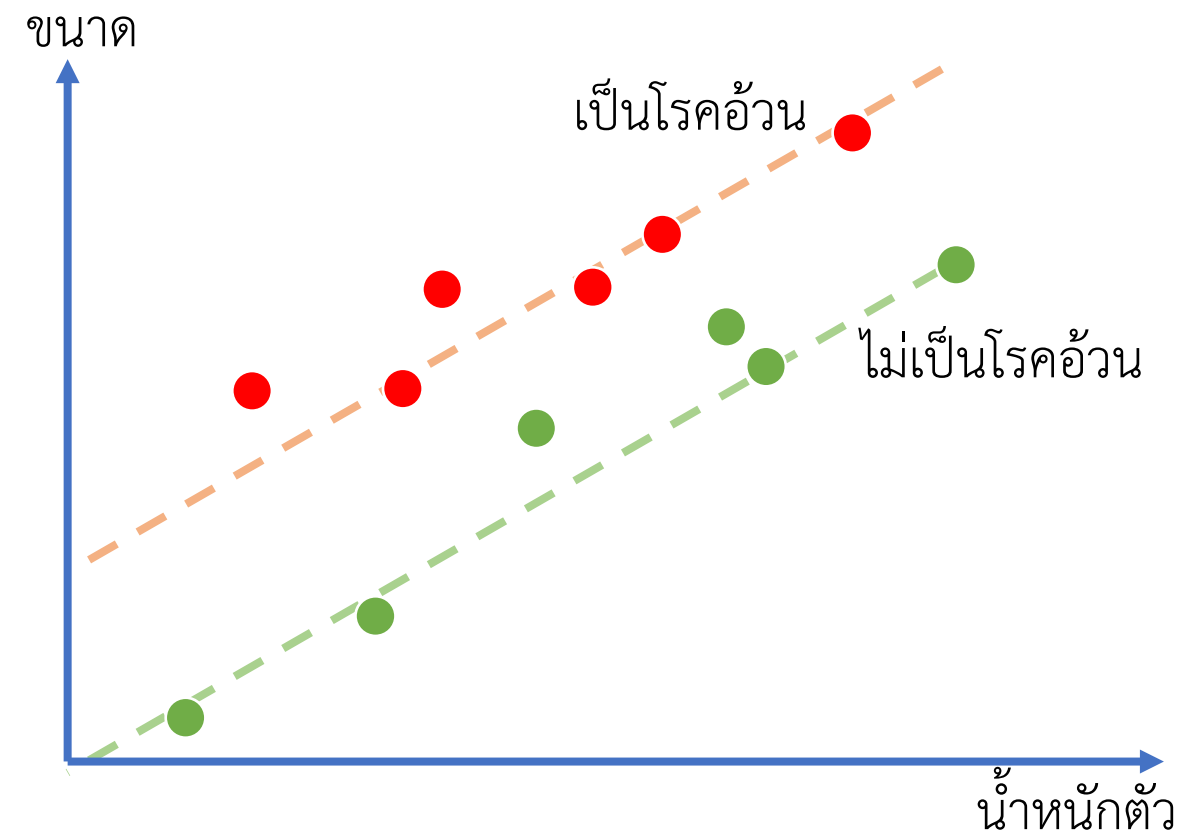


- แต่ข้อมูลจริงๆ มักไม่ได้สวยงาม
  - ตอนนี้เรามีข้อมูลที่มีมิติมากขึ้น มีการแยกการเป็นและไม่เป็นโรคอ้วน
  - เขียนสมการได้เป็น  
 $size = y_{intercept} + (slope)(w) + (diff)(fat?)$

และเราพยายามหาค่า  $y_{intercept}$ ,  $slope$ ,  $diff$  โดย

$$\min(ssr + \lambda(slope^2 + diff^2))$$

\*ถ้ามีฟีเจอร์มากกว่านี้ จะใช้ ridge regression ช่วยได้หรือไม่?  
ตอบ ได้ ไม่ว่าจะเป็นกี่มิติก็ทำได้ ...แต่ถ้าข้อมูลน้อยกว่ามิติละ?

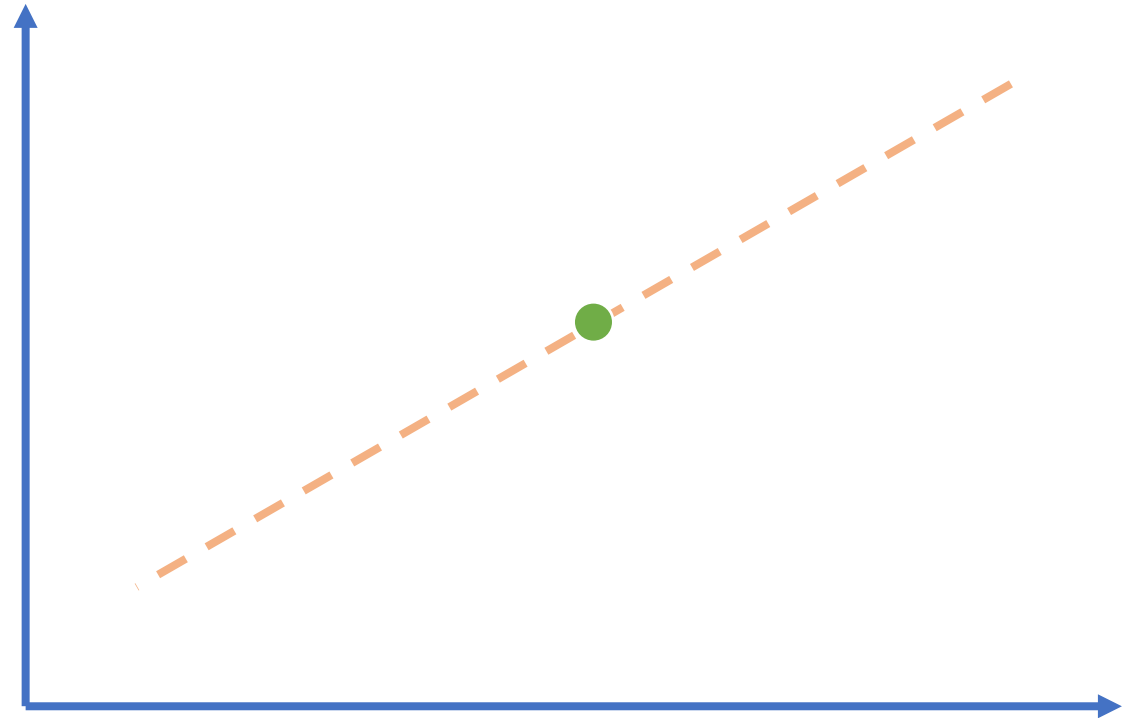


# Ridge Regression: กับข้อมูลที่ซับซ้อนขึ้น



โดยทั่วไป Regression จะต้องมียุคข้อมูลขั้นต่ำสุดอยู่

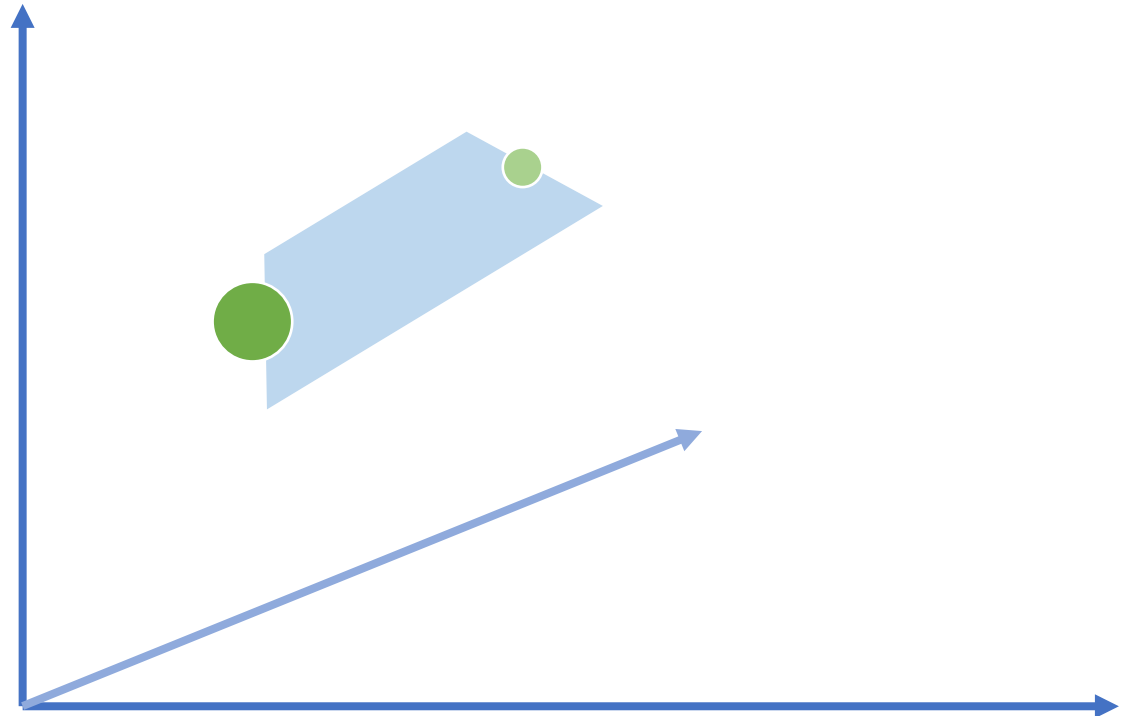
- กราฟ 2 มิติ ต้องมีข้อมูลอย่างน้อย 2 ตัว
  - ถ้ามีตัวเดียวจะเป็นดังตัวอย่างด้านขวา
  - เราจะไม่สามารถหา Fitting line ได้ เพราะ SSR เป็น 0 เท่ากันหมด



# Ridge Regression: กับข้อมูลที่ซับซ้อนขึ้น



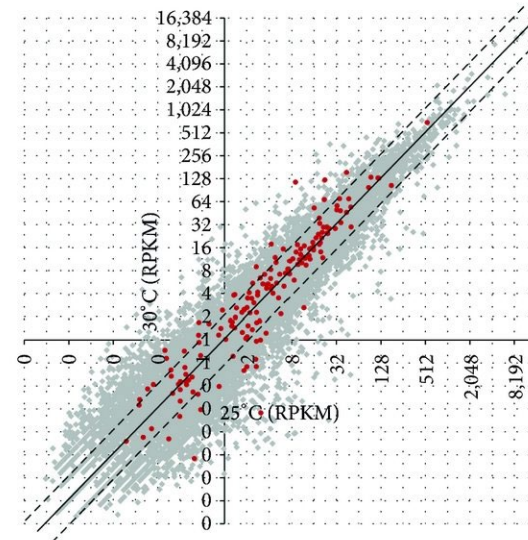
- ข้อมูลขั้นต่ำ
- กราฟ 2 มิติ ต้องมีข้อมูลอย่างน้อย 2 ตัว
- กราฟ 3 มิติ ต้องมีข้อมูลอย่างน้อย 3 ตัว
  - ตอนนี้เราไม่ได้ตัดข้อมูลด้วยเส้นแล้ว แต่เราตัดด้วยระนาบแทน
  - ถ้ามีข้อมูล 2 ตัว เราจะวางแนวระนาบได้แกนเดียว แต่มันจะสามารถหมุนรอบแกนหนึ่งได้อย่างอิสระ โดยให้ค่า SSR เป็น 0 เท่ากันหมด



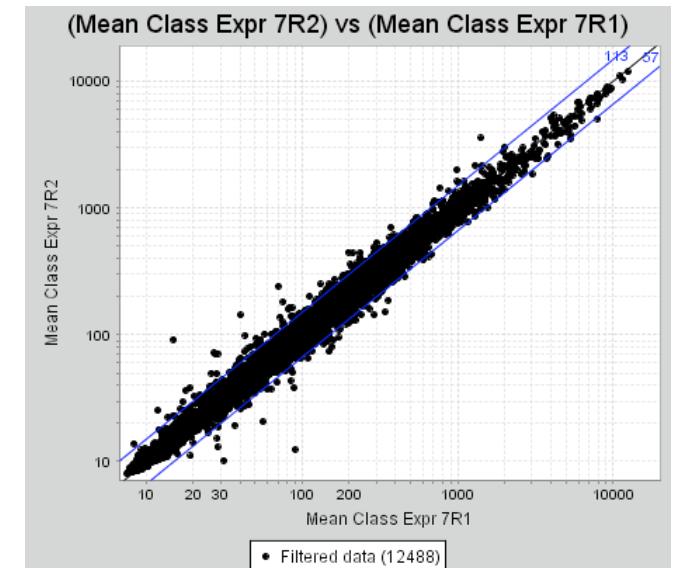
# Ridge Regression: กับข้อมูลที่ซับซ้อนขึ้น



- ข้อมูลขั้นต่ำ
- กราฟ 2 มิติ ต้องมีข้อมูลอย่างน้อย 2 ตัว
- กราฟ 3 มิติ ต้องมีข้อมูลอย่างน้อย 3 ตัว
- กราฟ 1,000,000 มิติ ต้องมีข้อมูลอย่างน้อย 1,000,000 ตัว
  - ฟังดูตลกดี แต่เรื่องจริงยิ่งกว่านิยาย



• MYB  
• 25°C versus 30°C (RPKM)



# Ridge Regression: กับข้อมูลที่ซับซ้อนขึ้น

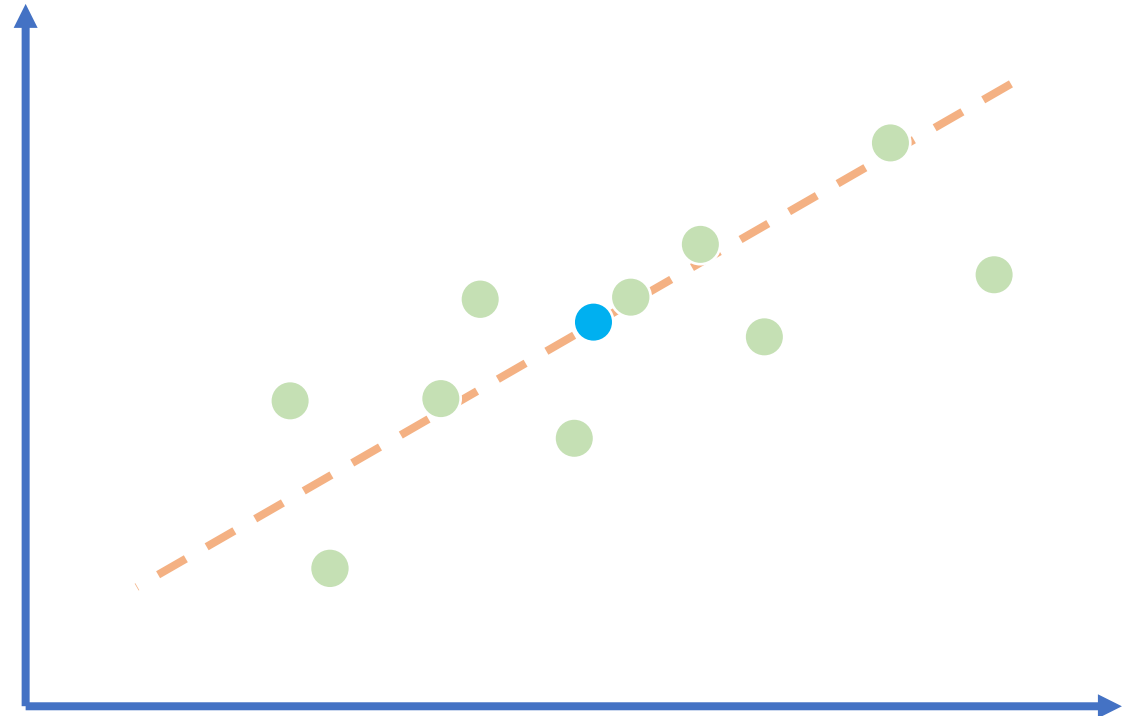


โดยทั่วไป Regression จะต้องมียุขมูลชั้นต่ำสุดอยู่

แต่ Ridge Regression ไม่ต้งมียุ

- เพราะ Ridge Regression สามารถหา Optimal penalty ที่คอยควบคุม slope ให้เหมาะสมได้
- โดยสามารถหา penalty ที่เหมาะสมจากการทำ 10-fold cross validation ได้ ทำให้ไม่จำกััดว่าต้งมียุขมูลมากกว่าหรือเท่ากับจำนวนมิติ

นี่คือความพิเศษมาก ๆ ของ Ridge Regression ที่ทำให้เราไม่ต้องกังวลเรื่องจำนวนข้อมูลและมิติ



# ตัวอย่างโค้ด: Ridge Regression

```
class sklearn.linear_model.Ridge(alpha=1.0, *, fit_intercept=True, normalize='deprecated', copy_X=True, max_iter=None, tol=0.001, solver='auto', positive=False, random_state=None)
```

## Parameters: **alpha : {float, ndarray of shape (n\_targets,)}, default=1.0**

Constant that multiplies the L2 term, controlling regularization strength. `alpha` must be a non-negative float i.e. in `[0, inf)`.

When `alpha = 0`, the objective is equivalent to ordinary least squares, solved by the `LinearRegression` object. For numerical reasons, using `alpha = 0` with the `Ridge` object is not advised. Instead, you should use the `LinearRegression` object.

If an array is passed, penalties are assumed to be specific to the targets. Hence they must correspond in number.

## **fit\_intercept : bool, default=True**

Whether to fit the intercept for this model. If set to false, no intercept will be used in calculations (i.e. `X` and `y` are expected to be centered).

## **normalize : bool, default=False**

This parameter is ignored when `fit_intercept` is set to False. If True, the regressors `X` will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use `StandardScaler` before calling `fit` on an estimator with `normalize=False`.

# ตัวอย่างโค้ด: Ridge Regression

```
class sklearn.linear_model.Ridge(alpha=1.0, *, fit_intercept=True, normalize='deprecated', copy_X=True, max_iter=None, tol=0.001, solver='auto', positive=False, random_state=None)
```

## Attributes::

**coef\_ : ndarray of shape (n\_features,) or (n\_targets, n\_features)**

Weight vector(s).

**intercept\_ : float or ndarray of shape (n\_targets,)**

Independent term in decision function. Set to 0.0 if `fit_intercept = False`.

**n\_iter\_ : None or ndarray of shape (n\_targets,)**

Actual number of iterations for each target. Available only for sag and lsqr solvers. Other solvers will return None.

*New in version 0.17.*

**n\_features\_in\_ : int**

Number of features seen during `fit`.

*New in version 0.24.*

**feature\_names\_in\_ : ndarray of shape (n\_features\_in\_,)**

Names of features seen during `fit`. Defined only when `X` has feature names that are all strings.

*New in version 1.0.*

# ตัวอย่างโค้ด: Ridge Regression

```
class sklearn.linear_model.Ridge(alpha=1.0, *, fit_intercept=True, normalize='deprecated', copy_X=True, max_iter=None, tol=0.001, solver='auto', positive=False, random_state=None)
```

```
>>> from sklearn.linear_model import Ridge
>>> import numpy as np
>>> n_samples, n_features = 10, 5
>>> rng = np.random.RandomState(0)
>>> y = rng.randn(n_samples)
>>> X = rng.randn(n_samples, n_features)
>>> clf = Ridge(alpha=1.0)
>>> clf.fit(X, y)
```

```
predict(X)
```

Predict using the linear model.

|              |                                                                                          |
|--------------|------------------------------------------------------------------------------------------|
| Parameters:: | <b>X</b> : <i>array-like or sparse matrix, shape (n_samples, n_features)</i><br>Samples. |
| Returns::    | <b>C</b> : <i>array, shape (n_samples,)</i><br>Returns predicted values.                 |

# ตัวอย่างโค้ด: Ridge Regression



สิ่งที่เกี่ยวข้องกับ Ridge ที่น่าสนใจ

**See also:**

**RidgeClassifier**

Ridge classifier.

**RidgeCV**

Ridge regression with built-in cross validation.

**KernelRidge**

Kernel ridge regression combines ridge regression with the kernel trick.

# Lasso Regression

- วิธีการ Lasso Regression เป็นวิธีที่มีแนวคิดของการลดความ Sensitive ของ โมเดลลง (ลด Slope) เหมือนกับวิธีการ Ridge Regression
- สิ่งที่แตกต่างกันในระดับทฤษฎีมีเพียงเล็กน้อยเท่านั้น นั่นคือ Penalty term

# Lasso Regression

- ถ้ายังจำกันได้ สมการที่มี penalty term ของ Ridge Regression จะเป็นดังนี้

$$\min(SSR + \lambda(slope^2))$$

- แต่สมการของ Lasso Regression จะเป็นดังนี้

$$\min(SSR + \lambda|slope|)$$

เห็นความแตกต่างหรือไม่?

# Lasso Regression

- สมการของ Ridge Regression ใช้กำลังสองของความชันในการคำนวณ นั้นทำให้ความชันส่งผลต่อสมการแบบ Exponential
- แต่สมการของ Lasso Regression ใช้ค่า Absolute ของความชัน ทำให้ความชันส่งผลต่อสมการแบบเป็นเส้นตรง (การปรับค่าความชันเพื่อ minimize สมการจะคงที่)

ทั้งสองสามารถใช้ในงานลักษณะเดียวกันได้ทั้งหมด แต่การปรับค่าความชันจะมีอุปนิสัยต่างกันเล็กน้อยเนื่องจากสมการ แต่..

# Lasso Regression

- ความแตกต่างเพียงเล็กน้อย ส่งผลต่อการใช้งานที่ยิ่งใหญ่
- การที่ Ridge Regression ใช้กำลังสองของ slope ในการคำนวณนั้น ทำให้ไม่สามารถลด slope จนกลายเป็น 0 ได้ ทำได้มากที่สุดแค่ตั้งให้เข้าใกล้ 0 เท่านั้น
  - นั่นคือเราไม่สามารถตัดข้อมูลตัวนั้นออกจากคลัง Feature ที่เรามีได้อย่างสมบูรณ์
  - ทำได้เพียงลดผลกระทบต่อระบบเท่านั้น
- แต่ Lasso Regression ใช้ absolute ของ slope ทำให้ slope ของฟีเจอร์ที่เกี่ยวข้องค่อยๆ ลดลงอย่างช้าๆ แต่ slope ของ Feature ที่ไม่เกี่ยวข้องลดลงอย่างรวดเร็ว **จนกลายเป็น 0 ได้** นั่นคือ Feature ถูกตัดออกอย่างสมบูรณ์

# Lasso Regression



- ตัวอย่าง

$$size = y_{intercept} + (slope_1)weight + (slope_2)neutrition + (slope_3)tall + (slope_4)age + (slope_5)iPhone3Price$$

## Lasso Regression

เราจะพยายามหา slope ทั้งหมดที่เหมาะสม โดยการ minimize สมการต่อไปนี้

$$\min(SSR + \lambda(|slope_1| + |slope_2| + |slope_3| + |slope_4| + |slope_5|))$$

# Lasso Regression

$$\min(SSR + \lambda(|slope_1| + |slope_2| + |slope_3| + |slope_4| + |slope_5|))$$

เราจะค่อยๆเพิ่มค่า  $\lambda$  จากนั้น slope ทุกตัวจะลดลง

1. ในกรณีที่พีเจอร์เจ้าของ slope นั้นมีความเกี่ยวข้องกับงาน slope จะลดลงช้าๆ และไม่เป็น 0
2. ในกรณีที่พีเจอร์เจ้าของ slope นั้นไม่เกี่ยวข้องกับงาน slope จะลดลงจนเป็น 0
  - เมื่อ slope กลายเป็น 0 Feature นั้นจะไม่ส่งผลต่อระบบอีกไม่ว่าจะมีค่าเป็นเท่าใด
3. ในที่สุดแล้ว หากเราเพิ่มค่า  $\lambda$  จนใหญ่มาก เข้าใกล้  $+\infty$  จะทำให้ slope ทั้งหมดกลายเป็น 0
  - ดังนั้นห้ามปรับค่า  $\lambda$  ใหญ่เกินไป
4. ในทางกลับกัน ถ้าค่า  $\lambda$  เล็กเกินไป ก็จะตัด Feature ที่ไม่จำเป็นออกได้น้อยกว่าที่ควร
  - ดังนั้นห้ามปรับค่า  $\lambda$  เล็กเกินไป

# Lasso Regression: ค่าของ $\lambda$



ใช้เทคนิคเดียวกับ Ridge Regression คือ

- ใช้วิธีการ 10-fold cross validation เพื่อให้ข้อมูลสลับกันเป็น Training set และ Test set
- เลือกค่า  $\lambda$  ที่ทำให้ค่าเฉลี่ยของ Accuracy จาก 10-fold cross validation ดีที่สุด
- การทำแบบนี้ทำให้เราสามารถสังเกต bias และ variance คร่าวๆ จากข้อมูลที่มีได้

# ตัวอย่างโค้ด: Lasso Regression

```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True, normalize='deprecated', precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

## Parameters::

**alpha : float, default=1.0**

Constant that multiplies the L1 term, controlling regularization strength. `alpha` must be a non-negative float i.e. in `[0, inf)`.

When `alpha = 0`, the objective is equivalent to ordinary least squares, solved by the [LinearRegression](#) object. For numerical reasons, using `alpha = 0` with the `Lasso` object is not advised. Instead, you should use the [LinearRegression](#) object.

**fit\_intercept : bool, default=True**

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

**normalize : bool, default=False**

This parameter is ignored when `fit_intercept` is set to False. If True, the regressors X will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use [StandardScaler](#) before calling `fit` on an estimator with `normalize=False`.

*Deprecated since version 1.0:* `normalize` was deprecated in version 1.0 and will be removed in 1.2.

# ตัวอย่างโค้ด: Lasso Regression



```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True, normalize='deprecated', precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

**precompute : bool or array-like of shape (n\_features, n\_features), default=False**

Whether to use a precomputed Gram matrix to speed up calculations. The Gram matrix can also be passed as argument. For sparse input this option is always `False` to preserve sparsity.

**copy\_X : bool, default=True**

If `True`, X will be copied; else, it may be overwritten.

**max\_iter : int, default=1000**

The maximum number of iterations.

**tol : float, default=1e-4**

The tolerance for the optimization: if the updates are smaller than `tol`, the optimization code checks the dual gap for optimality and continues until it is smaller than `tol`, see Notes below.

**warm\_start : bool, default=False**

When set to `True`, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. See [the Glossary](#).

# ตัวอย่างโค้ด: Lasso Regression

```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True, normalize='deprecated', precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

## Attributes::

**coef\_ : ndarray of shape (n\_features,) or (n\_targets, n\_features)**

Parameter vector (w in the cost function formula).

**dual\_gap\_ : float or ndarray of shape (n\_targets,)**

Given param alpha, the dual gaps at the end of the optimization, same shape as each observation of y.

**sparse\_coef\_ : sparse matrix of shape (n\_features, 1) or (n\_targets, n\_features)**

Sparse representation of the fitted `coef_`.

**intercept\_ : float or ndarray of shape (n\_targets,)**

Independent term in decision function.

**n\_iter\_ : int or list of int**

Number of iterations run by the coordinate descent solver to reach the specified tolerance.

**n\_features\_in\_ : int**

Number of features seen during `fit`.

*New in version 0.24.*

# ตัวอย่างโค้ด: Lasso Regression



```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True, normalize='deprecated', precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

```
from sklearn import linear_model  
clf = linear_model.Lasso(alpha=0.1)  
clf.fit([[0, 0, 0], [1, 1, 1], [2, 2, 2], [3, 3, 3]], [0, 1, 2, 3])  
print(clf.coef_)  
print(clf.predict([[4, 4, 4]]))
```

```
[0.85 0.    0.    ]  
[3.55]
```

# ตัวอย่างโค้ด: Lasso Regression



```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True, normalize='deprecated', precompute=False, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

```
from sklearn import linear_model  
clf = linear_model.Lasso(alpha=0.1)  
clf.fit([[1, 0, 1], [1, 0, 2], [1, 0, 3], [0, 0, 2]], [0, 0, 0, 1])  
print(clf.coef_)  
print(clf.predict([[1, 0, 1]]))
```

```
[-0.46666667  0.          0.          ]  
[0.13333333]
```

# Elastic Net Regression

- ถ้าเข้าใจ Ridge และ Lasso ตัวนี้ก็ไม่ยาก
- Elastic Net เป็นการทำงานที่ตรงตามชื่อ คือมีความยืดหยุ่นสูง
- มักใช้ในงานที่ไม่รู้จะเลือก Ridge หรือ Lasso ดี เช่น ข้อมูลเข้ามีความหลากหลาย แต่ละครึ่งก็ไม่สามารถเจาะจงได้ว่าทุกครั้งข้อมูลควรจะถูกจัดการด้วย Ridge หรือ Lasso ดี
- จะเลือกทำไม ในเมื่อเราใช้ทั้งสองตัวได้

# Elastic Net Regression

- Ridge Regression

$$\min(SSR + \lambda(\textcolor{red}{slope}^2))$$

- Lasso Regression

$$\min(SSR + \lambda|\textcolor{red}{slope}|)$$

- Elastic Net Regression

$$\min(SSR + \lambda_1(\textcolor{red}{slope}^2) + \lambda_2|\textcolor{red}{slope}|)$$

# ตัวอย่างโค้ด: Elastic Net Regression

```
class sklearn.linear_model.ElasticNet(alpha=1.0, *, l1_ratio=0.5, fit_intercept=True, normalize='deprecated', precompute=False, max_iter=1000, copy_X=True, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

## Parameters:: **alpha : float, default=1.0**

Constant that multiplies the penalty terms. Defaults to 1.0. See the notes for the exact mathematical meaning of this parameter. `alpha = 0` is equivalent to an ordinary least square, solved by the [LinearRegression](#) object. For numerical reasons, using `alpha = 0` with the [Lasso](#) object is not advised. Given this, you should use the [LinearRegression](#) object.

## **l1\_ratio : float, default=0.5**

The ElasticNet mixing parameter, with  $0 \leq \text{l1\_ratio} \leq 1$ . For `l1_ratio = 0` the penalty is an L2 penalty. For `l1_ratio = 1` it is an L1 penalty. For  $0 < \text{l1\_ratio} < 1$ , the penalty is a combination of L1 and L2.

## **fit\_intercept : bool, default=True**

Whether the intercept should be estimated or not. If `False`, the data is assumed to be already centered.

## **normalize : bool, default=False**

This parameter is ignored when `fit_intercept` is set to `False`. If `True`, the regressors `X` will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use [StandardScaler](#) before calling `fit` on an estimator with `normalize=False`.

# ตัวอย่างโค้ด: Elastic Net Regression



```
class sklearn.linear_model.ElasticNet(alpha=1.0, *, l1_ratio=0.5, fit_intercept=True, normalize='deprecated', precompute=False, max_iter=1000, copy_X=True, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

## Attributes::

**coef\_ : ndarray of shape (n\_features,) or (n\_targets, n\_features)**

Parameter vector (w in the cost function formula).

**sparse\_coef\_ : sparse matrix of shape (n\_features,) or (n\_targets, n\_features)**

Sparse representation of the fitted `coef_`.

**intercept\_ : float or ndarray of shape (n\_targets,)**

Independent term in decision function.

**n\_iter\_ : list of int**

Number of iterations run by the coordinate descent solver to reach the specified tolerance.

**dual\_gap\_ : float or ndarray of shape (n\_targets,)**

Given param alpha, the dual gaps at the end of the optimization, same shape as each observation of y.

**n\_features\_in\_ : int**

Number of features seen during `fit`.

# ตัวอย่างโค้ด: Elastic Net Regression



```
class sklearn.linear_model.ElasticNet(alpha=1.0, *, l1_ratio=0.5, fit_intercept=True, normalize='deprecated', precompute=False, max_iter=1000, copy_X=True, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

```
>>> from sklearn.linear_model import ElasticNet
>>> from sklearn.datasets import make_regression
```

```
>>> X, y = make_regression(n_features=2, random_state=0)
>>> regr = ElasticNet(random_state=0)
>>> regr.fit(X, y)
ElasticNet(random_state=0)
>>> print(regr.coef_)
[18.83816048 64.55968825]
>>> print(regr.intercept_)
1.451...
>>> print(regr.predict([[0, 0]]))
[1.451...]
```